

第 7 章

机器学习与专家系统

机器学习是人工智能中最具智能特征、最前沿的研究领域,一直受到人工智能及认知心理学家们的普遍关注,特别是近年来深度学习掀起了新的研究与应用热潮,目前已经成为人工智能应用最广泛的分支,取得了许多举世瞩目的应用成果。机器学习是专家系统的核心,已经应用到数学、物理、化学、医学、地质、气象、农业、法律、教育、交通运输、机械、艺术以及计算机科学本身,甚至渗透到政治、经济、军事等重大决策部门,产生了巨大的社会效益和经济效益。

本章首先介绍机器学习的基本概念与分类,特别是介绍监督学习、无监督学习和弱监督学习的概念,然后简要介绍线性回归、k 近邻、决策树、支持向量机和 k 均值聚类等几种常用的经典机器学习算法。最后介绍专家系统的基本概念、工作原理和建立专家系统的基本方法。

7.1 机器学习概述

机器学习是人工智能中一个重要的研究领域,一直受到人工智能及认知心理学家们的普遍关注。关于机器学习的研究,可以追溯到 20 世纪 50 年代中期。当时人们从仿生学的角度开展研究,希望搞清楚人类大脑及神经系统的学习机理。但由于受到客观条件的限制,未能如愿。之后几经波折,直到 20 世纪 80 年代才获得了蓬勃发展。

近年来,随着专家系统的发展,需要系统具有学习能力,促进了机器学习的研究,使之获得了较快的发展,研制出多种学习系统。但机器学习与计算机科学、心理学等多种学科都有密切的关系,牵涉的面比较广,因此许多理论及技术上的问题尚处于研究之中。

7.1.1 机器学习的基本概念

学习是一个有特定目的的知识获取过程,其内在行为是获取知识、积累经验、发现规律;外部表现是改进性能、适应环境、实现系统的自我完善。

机器学习(machine learning)使计算机能模拟人的学习行为,自动地通过学习来获取知识和技能,不断改善性能,实现自我完善。

作为人工智能的一个研究领域,机器学习主要研究以下三方面问题。

① 学习机理。这是对人类学习机制的研究,即人类获取知识、技能和抽象概念的天赋能力。通过这一研究,将从根本上解决机器学习中的种种问题。

② 学习方法。研究人类的学习过程,探索各种可能的学习方法,建立起独立于具体应用领域的学习算法。机器学习方法的构造是在对生物学习机理进行简化的基础上,用计算的方法进行再现。

③ 学习系统。根据特定任务的要求,建立相应的学习系统。

从计算机算法角度研究机器学习问题,与生物学、医学和生理学从生理、生物功能角度研究生物界,特别是人类学习问题,有着密切的联系。最近国际上新兴的脑机交互(brain-computer interface, BCI)就是从大脑中直接提取信号,并经过计算机处理加以应用。

一个学习系统一般应该由环境、学习、知识库、执行与评价四个基本部分组成。各部分之间的关系如图 7.1 所示,其中,箭头表示信息的流向。



图 7.1 学习系统的基本结构

“环境”指外部信息的来源,可以是系统的工作对象,也可以包括工作对象和外界条件。环境是以某种形式表达的外界信息的集合。例如,在医疗系统中,环境就是患者当前的症状、检验的数据和病历;在模式识别中,环境是要识别的图形或图像;在控制系统中,环境是被控对象。环境为系统的学习机构提供有关信息。它通过对环境的搜索取得外部信息,然后经分析、综合、类比、归纳等思维过程获得知识,并将这些知识存入知识库中。

“学习”部分将根据反馈信息决定是否要从环境中索取进一步的信息进行学习,以修改、完善知识库中的知识。这是学习系统的一个重要特征。

“知识库”用于存储由学习得到的知识,在存储时要进行适当的组织,使它既便于应用又便于维护。

“执行与评价”实际上是由“执行”与“评价”这两个环节组成的。执行环节用于处理系统面临的现实问题,即应用学到的知识求解问题,如定理证明、智能控制、自然语言处理、机器人行动规划等;评价环节用于验证、评价执行环节执行的效果,如结论的正确性等。目前对评价的处理有两种方式:一种是把评价时所需的性能指标直接建立在系统中,由系统对执行环节得到的结果进行评价;另一种是由人来协助完成评价工作。如果采用后一种方式,则图 7.1 中可略去评价环节,但环境、学习、知识库、执行等环节是不可缺少的。

上述模型是针对符号学习系统的,不能完全概括所有的机器学习系统。例如,不能概括遗传学习、神经网络学习系统等。

7.1.2 机器学习的分类

机器学习可从不同的角度,根据不同的方式进行分类。主要有以下几种分类方式。

(1) 按系统的学习能力分类

机器学习可分为监督学习、无监督学习与弱监督学习。这是当前最常用的分类方法。

监督学习在学习时需要教师的示教或训练,往往需要很大的工作量,甚至不可能实现。

无监督学习是用评价标准来代替人的监督工作,一般效果比较差。

弱监督学习则结合监督学习与无监督学习的优点,利用不完全的有标签数据进行有监督学习,同时利用大量的无标签数据进行无监督学习。弱监督学习方法主要有半监督学习、迁移学习和强化学习。

Yann LeCun 有一个非常著名的比喻:“假设机器学习是一个蛋糕,强化学习是蛋糕上的一粒樱桃,监督学习是外面的一层糖衣,无监督学习才是蛋糕的糕体”。

(2) 按学习方法分类

正如人们有各种各样的学习方法,机器学习也有多种学习方法。若按学习时所用的方法进行分类,机器学习可分为机械式学习、指导式学习、示例学习、类比学习、解释学习等。这是温斯顿在 1977 年提出的一种分类方法。

若按学习方法是否为符号表示来分类,机器学习可分为符号学习与非符号学习。本章讨论符号学习,关于非符号学习,即连接学习,则在神经网络这一章中讨论。

(3) 按推理方式分类

若按学习时所采用的推理方式进行分类,则机器学习可分为基于演绎的学习和基于归纳的学习。

基于演绎的学习是指以演绎推理为基础的学习。解释学习在其推理过程中主要用的是演绎方法,因而可将它划入基于演绎的学习这一类。

基于归纳的学习是指以归纳推理为基础的学习。示例学习、发现学习等在其学习过程中主要使用了归纳推理,因而可划入归纳学习这一类。

早期的机器学习系统一般都使用单一的推理方式,现在则趋于集成多种推理技术来支持学习。例如,类比学习就既用到演绎推理又用到归纳推理,解释学习也是这样,只是因它演绎部分所占的比例较大,所以把它归入基于演绎的学习。

(4) 按综合属性分类

随着机器学习的发展以及人们对它认识的提高,要求对机器学习进行更科学、更全面的分类。因而近年来有人提出了按学习的综合属性进行分类,它综合考虑了学习的知识表示、推理方法、应用领域等多种因素,能比较全面地反映机器学习的实际情况。用这种方法进行分类,不仅可以把过去已有的学习方法都包括在内,还能反映了机器学习的最近发展。

按照这种分类方法,机器学习可分为归纳学习、分析学习、连接学习以及遗传算法与分类器系统等。

分析学习是基于演绎和分析的学习。学习时从一个或几个实例出发,运用过去求解问题的经验,通过演绎对当前面临的问题进行求解,或者产生能更有效应用领域知识的控制性规则。分析学习的目标不是扩充概念描述的范围,而是提高系统的效率。

机器学习还有其他多种分类方法。例如,若按所学知识的表示方式分类,则机器学习可分为逻辑表示法学习、产生式表示法学习、框架表示法学习等;若按机器学习的应用领域分类,则机器学习可分为专家系统、机器人学、自然语言处理、图像识别、博弈、数学、音乐等。

7.1.3 监督学习与无监督学习

下面着重介绍按学习能力分类的机器学习方法。

1. 监督学习(有教师学习)

监督学习(supervised learning)的基本学习方法如图 7.2 所示。监督学习系统中根据“教师”提供的正确响应调整学习系统的参数和结构。简单地说,监督学习就是在已知输入和输出的情况下训练出一个模型,将输入映射到输出。典型的监督学习包括线性回归、k 近邻算法、决策树、随机森林、逻辑回归、支持向量机、BP 学习等。

监督学习是目前机器学习中最广泛使用的一类方法,已经发展出了数以百计的不同方法,占据了目前机器学习算法的绝大部分。监督学习技术通过学习大量标记的训练样本来构建预测模型,在很多领域都获得了巨大的成功,但是由于数据标注本身往往需要很高的成本,在很多任务中都很难获得全部真值标签这样的监督信息。

2. 无监督学习(无教师学习)

无监督学习(unsupervised learning)的基本学习方法如图 7.3 所示。无监督学习系统完全按照环境提供的数据的某些统计规律调节自身的参数或者结构(自组织),以表示出外部输入的某种固有特性。例如,聚类或者某种统计上的分布特征。无监督学习方法包括各种自组织学习方法,如聚类学习、自组织神经网络学习、自编码器、k 均值算法等。

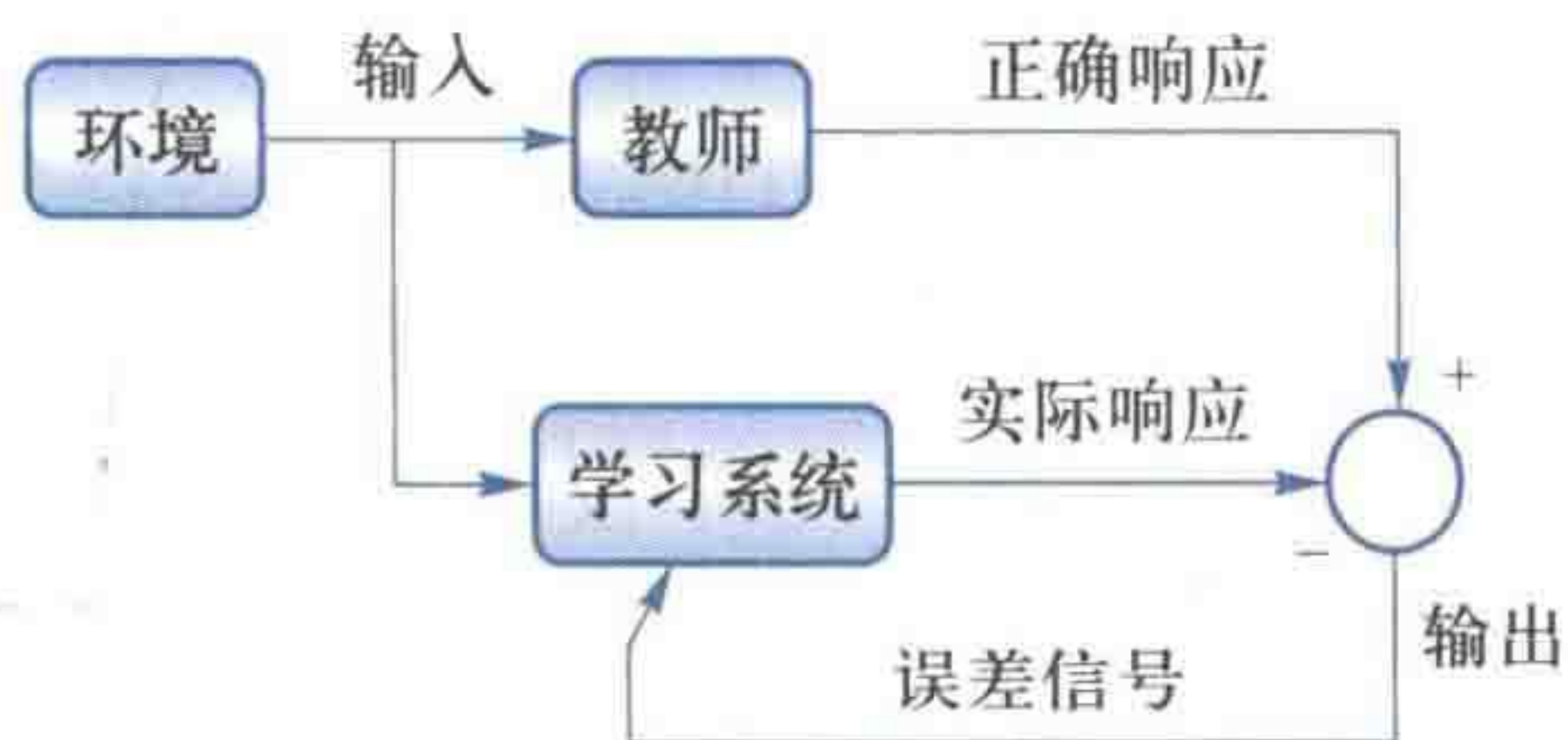


图 7.2 监督学习框图



图 7.3 无监督学习框图

无监督学习不需要人类进行数据标注,而是通过模型不断地自我认知、自我巩固,最后进行自我归纳来实现其学习过程,这对大数据分析尤为重要。但无监督学习由于缺乏标签,在实际应用中的性能往往存在很大的局限。

虽然目前无监督学习还处于研究阶段,但是机器学习未来的发展方向,正在引起越来越多的关注。2015 年,Yann LeCun,Yoshua Bengio,Geoffrey Hinton 首次合作在 *Nature* 杂志撰文对深度学习的未来展望时指出:“无监督学习对于重新点燃深度学习的热潮起到了促进作用。我们期望无监督学习在未来越来越重要,使我们能够通过观察发现世界的内在结构,而不是被告知每一个客观事物的名称。”

7.1.4 弱监督学习

针对监督学习和无监督学习各自的优缺点,研究人员提出了弱监督学习的概念。

弱监督学习中的数据标签允许是不完全的,即训练集中只有一部分数据是有标签的,而其余的数据甚至是绝大部分数据是没有标签的。

弱监督学习更接近人类的学习方式。例如,父母教一个婴儿认识猫,会指着一只小猫或者拿着一张猫的照片,告诉他这是“猫”。以后小孩遇到不同的猫或者猫的照片的时候,尽管父母不会一直告诉他们这也是“猫”,但小孩会不断地自我发现、学习、调整自己对“猫”的认识,从而最终理解并认识什么是“猫”。

父母教婴儿认识猫是监督学习,但父母给小孩看的猫或者猫的照片是不完全的标签数据。小孩看到的其他不同的猫或者猫的照片,是无标签数据,小孩会不断地自我发现、学习、调整自己对“猫”的认识是无监督学习。如果仅仅用监督学习,则要求人们一次次反复地告诉机器学习模型这是“猫”,也许要高达数万甚至数十万次。很显然,弱监督学习的模式更加接近小孩的学习方式。

弱监督学习一方面降低了人工标记的工作量,同时又可以引入人类的监督信息,在很大程度上提高无监督学习的性能,成为当前机器学习领域的重要研究方向,已经被广泛应用在自动控制、调度、金融、网络通信等领域,在认知、神经科学领域,弱监督学习也有重要研究价值。

弱监督学习涵盖的范围很广泛,可以说只要标注信息是不完全、不确切或者不精确的学习,都可以看作是弱监督学习。下面仅介绍半监督学习、迁移学习和强化学习这三种典型的弱监督学习。

(1) 半监督学习

半监督学习是一种典型的弱监督学习方法。在半监督学习中,只有少量有标注的数据,还有大量未标注的数据可供使用。因为仅仅学习这些少量有标注数据还不足以训练出好的模型,还需要用大量的无监督数据来改善模型性能。因此,半监督学习不仅最大限度地发挥有标注数据的作用,而且还从体量巨大、结构繁多的无标注数据中挖掘出隐藏的规律。半监督学习成为近年来机器学习领域比较活跃的研究方向,被广泛应用于社交网络分析、文本分类、计算机视觉和生物医学信息处理等领域。

近年来,随着大数据相关技术的飞速发展,容易收集大量的未标记数据,例如,在大量的互联网应用中,无标记的数据量是极为庞大甚至是无限的。而获取大量有标记的样本则相对较为困难,往往需要大量的人力、物力和财力。例如,在医学图像处理当中,随着医学影像技术的发展,容易获取影像数据,但是对病灶等数据的标识往往需要有经验的医生进行诊断。由于时间和精力限制,医学专家只能标注相当少的一部分图像,所以,适合采用半监督学习进行医学影像分析。

(2) 迁移学习

迁移学习侧重于将已经学习过的知识迁移应用到新的问题中。

通常所说的举一反三能力就是迁移学习。例如,学会了蛙泳,再学蝶泳就会变得相对容易;学会了中国象棋,再学习国际象棋也会变得相对容易。对于计算机来说,同样希望机器学习模型在学习到一种能力之后,稍加调整就可用于一个新的领域。人类对举一反三的理论研究要追溯到 1901 年,心理学家桑代克和伍德沃思提出了学习迁移(transfer of learning)的概念。他们主要研究了人们学习某个概念时如何对学习其他概念产生迁移,这些理论后来对教育学的发展产生了重要影响。

1990 年以来,大量研究都涉及迁移学习的概念,如自主学习、终身学习、多任务学习、知识迁移等,但没有形成一个完整的迁移学习体系。直到 2010 年,才提出了迁移学习的形式化定义,迁移学习成为机器学习中一个重要的分支领域。

随着大数据时代的到来,迁移学习变得越来越重要。现在可以很容易地获取大量的城市交通、视频监控、行业物流等不同类型的数据库,互联网也在不断产生大量的图像、文本、语音等数据。但这些数据往往都是没有标注的。而现在很多机器学习方法都是有监督学习方法,需要大量的标注数据为前提。如果能够将在标注数据上训练得到的模型,有效地迁移到这些无标注数据上,无疑具有重要的价值。

在迁移学习中,通常称有知识和数据标注的领域为源域,是要迁移的对象;而把最终要赋予知识、赋予标注的对象称作目标域。迁移学习的核心目标就是将知识从源域中迁移到目标域。目前迁移学习主要通过以下三种方式来实现。

① **样本迁移**。在源域中找到与目标域相似的数据,并赋予其更高的权重,从而完成从源域到目标域的迁移。这种方法的优点是简单且容易实现,但是权重和相似度的选择往往高度地依赖经验,使得算法的可靠性降低。

② **特征迁移**。通过特征变换将源域和目标域的特征映射到同一个特征空间当中,然后再用经典的机器学习方法来求解。这种方法的优点是对大多数方法适用,而且效果较好,但是在实际问题当中通常难以求解。

③ **模型迁移**。假设源域和目标域共享模型参数,即将在源域中通过大量数据训练好的模型应用到目标域上。从源数据中挑选出和目标数据更相似的样本来参与训练,而剔除和目标数据不相似的样本。例如,在一个千万量级的标注样本集上训练得到了一个图像分类系统,在一个新领域的图像分类任务当中,可以直接用之前训练好的模型,再加上目标域的几万张标注样本进行微调,即可以得到很高精度的模型。模型迁移是目前最主流的迁移学习方法,可以很好地利用模型之间的相似度,具有广阔的应用前景。

迁移学习可以充分利用既有模型的知识,使得机器学习模型在面临新的任务时,只需要进行少量的微调即可完成相应的任务,具有重要的应用价值。目前,迁移学习已经在机器人控制、机器翻译、图像识别、人机交互等诸多领域获得了广泛的应用。

(3) 强化学习(再励学习)

强化学习是弱监督学习的一类典型算法。强化学习算法理论的形成可以追溯到 20 世纪 70—80 年代,但是最近引起了广泛关注,特别是 2016 年 3 月,DeepMind 开发的 AlphaGo 利用强

化学习算法击败了人类世界围棋冠军,成为解决通用人工智能的关键路径。目前,强化学习算法已经在游戏、机器人等领域中取得突出成果。

强化学习(reinforcement learning, RL)的基本学习方法如图 7.4 所示。监督学习是对每个输入模式都有一个正确的目标输出,而强化学习中外部环境对系统输出结果只给出评价信息(奖励或者惩罚),而不是正确答案,学习系统通过那些受惩的动作改善自身的性能。基于遗传算法的学习方法就是一种强化学习。



图 7.4 强化学习框图

与监督学习不同,强化学习中的智能体通过尝试来发现各个动作产生的结果。因为没有标注数据告诉机器应当做哪个动作,只能通过设置合适的奖励函数,使得机器学习模型在奖励函数的引导下,自主地学习到相应的策略。

强化学习的目标就是研究智能体在与环境的交互过程中,如何学习到一种行为策略,以最大化得到的累积奖赏。强化学习就是在训练的过程中,不断地尝试,错了就扣分,对了就奖励,从而得到在各个状态环境当中最好的决策。

目前,将深度学习的感知能力和强化学习的决策能力相结合,成为深度强化学习,更接近人类的思维。

7.1.5 深度学习

传统的机器学习具有优异的特征学习能力,但在处理未加工数据时,需要设计一个特征提取器,把原始数据(如图像的像素值)转换成一个适当的内部特征表示或特征向量。深度学习是一种特征学习方法,把原始数据通过一些简单的但是非线性的模型转变成为更高层次的、更加抽象的表达。通过足够多的转换的组合,非常复杂的函数也可以被学习。深度学习的实质是通过构建具有很多隐含层的机器学习模型和海量的训练数据,来学习更有用的特征,从而提升分类或预测的准确性。

机器学习是人们一直致力研究的课题。前面介绍了机器学习中几种经典的符号学习方法,但这些方法难以学习深层次知识,限制了机器学习的应用。自从提出感知机以来,人们不断研究基于连接机制的机器学习方法。20 世纪 80 年代末期提出 BP 算法,此后提出各种各样的机器学习方法,如支持向量机(support vector machines, SVM)、Boosting、最大熵方法(如 logistic regression, LR)等。这些模型的结构基本上可以看成带有一层隐含层结点(如 SVM、Boosting),或没有隐含层结点(如 LR),所以称为浅层学习(shallow learning)方法。由于神经网络理论分析的难度大,训练方法又需要很多经验和技巧,有限样本和有限计算单元情况下对复杂函数的表示能力有限,针对复杂分类问题其泛化能力受到一定制约。这个时期浅层人工神经网络的研究进展较慢。

2006 年 7 月,加拿大多伦多大学教授 Geoffrey Hinton 和他的学生 Ruslan Salakhutdinov 在 *Science* 上发表文章,提出深度学习(deep learning, DL),通过无监督学习实现“逐层初始化”

(layer-wise pre-training), 有效克服深度神经网络在训练上的难度, 掀起了深度学习的浪潮。

与人工规则构造特征的方法相比, 深度学习利用大数据来学习特征, 更能够刻画数据丰富的内在信息。深度学习具有较多层的隐含层结点, 通过逐层特征变换, 将样本在原空间的特征表示变换到一个新特征空间, 从而使分类或预测更加容易。

第8章将在介绍人工神经网络的基础上, 介绍卷积神经网络、胶囊网络、生成对抗网络等深度学习算法。

7.2 线性回归机器学习算法

7.2.1 线性回归问题

线性回归(linear regression)是利用数理统计中回归分析的方法, 确定两种或者两种以上变量之间相互依赖的定量关系的一种统计分析方法, 属于监督学习范畴。

线性回归的模型是一个超平面, 即设系统的输入输出关系为下列线性关系:

$$y = \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n \quad (7.1)$$

其中, y 是输出变量; $x_1, \cdots, x_n$ 是输入变量; $\theta_1, \cdots, \theta_n$ 是模型的参数, 这些参数可能部分或全部未知。设对输入、输出进行 m 次观测得到的数据为 $\{x_1(i), x_2(i), \cdots, x_n(i), y(i)\}, i=1, 2, \cdots, m$ 。现在的问题是: 怎样根据这些观测数据估计系统的参数 $\theta_1, \theta_2, \cdots, \theta_n$ 。

如果模型准确, 测量数据也准确, 则不难看出, 只要 n 组测量数据, 构成下列线性方程组, 解线性方程组就可唯一地确定系统参数 $\theta_1, \theta_2, \cdots, \theta_n$ 。

$$y(i) = \theta_1 x_1(i) + \theta_2 x_2(i) + \cdots + \theta_n x_n(i) \quad i = 1, 2, \cdots, n \quad (7.2)$$

若写成矩阵向量形式, 则有

$$Y = X\theta \quad (7.3)$$

其中

$$Y = \begin{bmatrix} y(1) \\ y(2) \\ \vdots \\ y(n) \end{bmatrix}, \quad \theta = \begin{bmatrix} \theta_1 \\ \theta_2 \\ \vdots \\ \theta_n \end{bmatrix}, \quad X = \begin{bmatrix} x_1(1) & x_2(1) & \cdots & x_n(1) \\ x_1(2) & x_2(2) & \cdots & x_n(2) \\ \vdots & \vdots & \ddots & \vdots \\ x_1(n) & x_2(n) & \cdots & x_n(n) \end{bmatrix}$$

若测量数据使 X 非奇异, 即 X 的逆矩阵存在, 则系统参数为

$$\theta = X^{-1}Y \quad (7.4)$$

但是, 实际系统的模型总是近似的, 测量输入、输出数据不会绝对精确, 而且测量装置的误差、读数误差, 以及系统中存在的随机误差都会使观测数据不能准确地反映系统特性。这些误差常常是随机的, 通常用一随机变量 $e(i)$ 反映上述随机误差。这样, m 组观测数据和系统参数间的关系可表示为

$$y(i) = \theta_1 x_1(i) + \theta_2 x_2(i) + \cdots + \theta_n x_n(i) + e(i) \quad i = 1, 2, \dots, m \quad (7.5)$$

或表示为

$$Y = X\theta + \varepsilon \quad (7.6)$$

其中, $\varepsilon = [e(1) \ e(2) \ \cdots \ e(m)]^T$ 为误差向量, 又称残差。

由于残差的存在, 不同的 m 组观测数据会得出不同的参数 θ 值, 这些 θ 值显然不是系统参数的真正值。线性回归机器学习的任务就是用统计的方法, 从带有噪声的观测数据中, 按照某种准则估计出最接近实际值的参数。最小二乘参数估计方法是广泛应用于工程界的线性回归机器学习方法。

7.2.2 最小二乘法及其应用

1. 最小二乘法

最小二乘法是数学家高斯于 1795 年提出的, 当时高斯利用它确定出了天体星球的运行轨道参数。目前此法已被应用到许多领域, 成为参数估计中最基本、最成熟的方法。

下面先以一个简单的例子来介绍最小二乘法的基本原理, 然后进行一般性的讨论。

已知某输入与输出的样本数据成正比, 即 $y = \theta x$ 。显然如果数据没有误差, 则只要一个样本数据 (x_1, y_1) 即可确定系数 θ , 即 $\theta = y_1/x_1$ 。当有噪声存在时, 实际的样本输出数据为 $y = \theta x + \varepsilon$ 。最小二乘法就是使系统输出的估计值 $\hat{\theta}x(i)$ 与系统输出的实际测量值 $y(i)$ 之差的平方和最小, 即

$$J_{\min} = \sum_{i=1}^m [y(i) - \hat{\theta}x(i)]^2 \quad (7.7)$$

对于任一个参数估计值 $\tilde{\theta}$, 其残差平方和为

$$\begin{aligned} \tilde{J} &= \sum_{i=1}^m [y(i) - \tilde{\theta}x(i)]^2 \\ \frac{\partial \tilde{J}}{\partial \tilde{\theta}} &= \sum_{i=1}^m 2[y(i) - \tilde{\theta}x(i)][-x(i)] \end{aligned}$$

当 $\hat{\theta}$ 是最小二乘估计时, $\tilde{J}$ 取得最小值。根据数学分析中熟知的结论, 应有

$$\left. \frac{\partial \tilde{J}}{\partial \tilde{\theta}} \right|_{\tilde{\theta} = \hat{\theta}} = -2 \sum_{i=1}^m x(i)[y(i) - \hat{\theta}x(i)] = 0$$

因此

$$\sum_{i=1}^m x(i)y(i) - \hat{\theta} \sum_{i=1}^m x^2(i) = 0$$

所以, 最小二乘估计 $\hat{\theta}$ 为

$$\hat{\theta} = \frac{\sum_{i=1}^m x(i)y(i)}{\sum_{i=1}^m x^2(i)}$$

下面讨论一般情况。设系统由下列多元静态线性数学模型描述：

$$y = \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n + \varepsilon \quad (7.8)$$

其中, x_i 和 y 都是可以测量得到的已知量, 只是存在着系统噪声和测量噪声, 它们总的效应用随机变量 ε 来表示。

若对式(7.8)描述的线性关系代入 m 个样本数据, 则可得到 m 个方程式:

$$y(1) = \theta_1 x_1(1) + \theta_2 x_2(1) + \cdots + \theta_n x_n(1) + e(1)$$

$$y(2) = \theta_1 x_1(2) + \theta_2 x_2(2) + \cdots + \theta_n x_n(2) + e(2)$$

$$\vdots$$

$$y(m) = \theta_1 x_1(m) + \theta_2 x_2(m) + \cdots + \theta_n x_n(m) + e(m) \quad (7.9)$$

写成矩阵向量形式, 则有

$$Y = X\theta + \varepsilon \quad (7.10)$$

其中

$$Y = \begin{bmatrix} y(1) \\ y(2) \\ \vdots \\ y(m) \end{bmatrix}_{m \times 1}, \quad X = \begin{bmatrix} x_1(1) & x_2(1) & \cdots & x_n(1) \\ x_1(2) & x_2(2) & \cdots & x_n(2) \\ \vdots & \vdots & \vdots & \vdots \\ x_1(m) & x_2(m) & \cdots & x_n(m) \end{bmatrix}_{m \times n}, \quad \theta = \begin{bmatrix} \theta_1 \\ \theta_2 \\ \vdots \\ \theta_n \end{bmatrix}_{n \times 1}, \quad \varepsilon = \begin{bmatrix} e(1) \\ e(2) \\ \vdots \\ e(m) \end{bmatrix}_{m \times 1}$$

按照最小二乘法的原理, 选择参数 θ 使残差的平方和为最小, 即

$$\min J = \sum_{i=1}^m e^2(i) = \varepsilon^T \varepsilon \quad (7.11)$$

因为

$$\varepsilon = Y - X\theta$$

所以

$$\begin{aligned} J &= (Y - X\theta)^T (Y - X\theta) = (Y^T - \theta^T X^T) (Y - X\theta) \\ &= Y^T Y - Y^T X\theta - \theta^T X^T Y + \theta^T X^T X\theta \\ &= Y^T Y - 2\theta^T X^T Y + \theta^T X^T X\theta \end{aligned}$$

为了求取 $\frac{\partial J}{\partial \theta}$, 下面先给出矩阵微分的公式:

$$\frac{df(x)}{dx} = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{bmatrix} \quad (7.12)$$

$$\frac{\partial (X^T A)}{\partial X} = A \quad (7.13)$$

$$\frac{\partial (X^T A X)}{\partial X} = 2AX \quad (7.14)$$

应用上面公式得

$$\frac{\partial J}{\partial \theta} = -2X^T Y + 2X^T X \theta$$

由最小二乘估计的定义, $\hat{\theta}$ 应满足

$$\left. \frac{\partial J}{\partial \theta} \right|_{\theta=\hat{\theta}} = -2X^T Y + 2X^T X \hat{\theta} = 0$$

因此

$$X^T X \hat{\theta} = X^T Y \quad (7.15)$$

若测量值构成的矩阵 $X^T X$ 非奇异, 则

$$\hat{\theta} = (X^T X)^{-1} X^T Y \quad (7.16)$$

由上式求得的 $\hat{\theta}$ 称为参数 θ 的最小二乘估计。由于估计值 $\hat{\theta}$ 是在取得足够数据后一次计算出来的, 所以也称为一次完成法。

必须指出, 当 $X^T X$ 是奇异或接近奇异时, 方程 $X^T X \hat{\theta} = X^T Y$ 的解不唯一或解不稳定。一般通过选择合适的输入和取足够多的测量值 ($m \gg n$) 使 $X^T X$ 非奇异。

2. 最小二乘法应用

用线性回归方程建立生产过程或系统静态模型, 已在许多方面得到应用, 下面举几个例子说明。

例 7.1 合成纤维抽丝工段, 导丝盘的速度是影响丝的质量的重要参数, 它和电流周波数有重要关系, 由生产记录得到的数据如表 7.1 所示

表 7.1 合成纤维抽丝工段实验记录

周波数 x	49.2	50.0	49.3	49.0	49.0	49.5	49.8	49.9	50.2	50.2
导丝盘速度 y	16.7	17.0	16.8	16.6	16.7	16.8	16.9	17.0	17.0	17.1

选择合适的曲线模型是一件不容易的事, 需要靠人们对问题所属专业知识的了解来确定。若专业上不清楚, 可用坐标纸描出点, 从数学上加以选择。

根据表 7.1 所示的实验数据, 绘制 $x-y$ 图如图 7.5 所示。

可见, y 与 x 之间近似为线性关系, 所以, 模型结构可以选择为

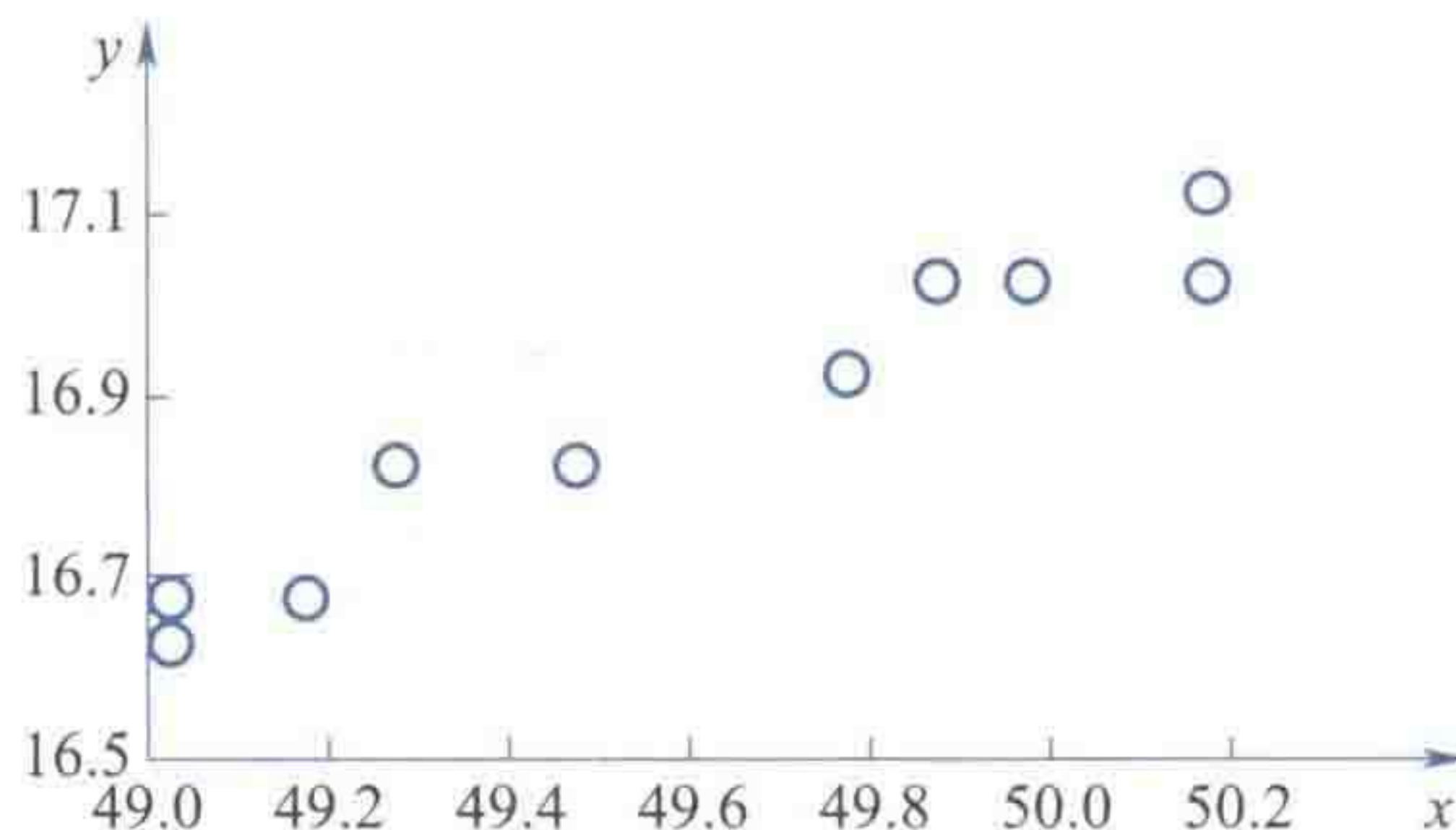


图 7.5 导丝盘的速度和电流周波数关系图

$$y = a + bx$$

由最小二乘法得

$$X^T = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 49.2 & 50.0 & 49.3 & 49.0 & 49.0 & 49.5 & 49.8 & 49.9 & 50.2 & 50.2 \end{bmatrix}$$

$$Y^T = [16.7 \quad 17.0 \quad 16.8 \quad 16.6 \quad 16.7 \quad 16.8 \quad 16.9 \quad 17.0 \quad 17.0 \quad 17.1]$$

$$\begin{bmatrix} a \\ b \end{bmatrix} = (X^T X)^{-1} X^T Y = \begin{bmatrix} 0.049 \\ 0.339 \end{bmatrix}$$

所以,导丝盘的速度和电流周波数的关系为

$$y = 0.049 + 0.339x$$

例 7.2 钢包容积和使用次数的数学模型。

出钢时盛钢水的钢包在使用过程中,由于钢液和炉渣对耐火材料的侵蚀,其容积不断增大。经过实验,钢包容积(用所盛钢水的重量表示)与使用次数的数据如表 7.2 所示。

表 7.2 钢包容积 y 和使用次数 x 的实验记录

x	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
y	6.42	8.20	9.58	9.50	9.70	10.00	9.93	9.99	10.49	10.59	10.60	10.80	10.60	10.90	10.76

作数据的散点图,如图 7.6 所示。

由图 7.6 可见, x 和 y 是非线性关系,在最初使用时,容积增加很快,随着使用次数的增加,其容积变化缓慢下来,并趋于稳定。根据其特点,可以取双曲线模型或者指数模型表示使用次数与容积增长之间的关系。这些虽然是非线性模型,但可以通过变量代换将这种非线性模型参数估计问题转换成线性模型参数估计问题。

对一般的非线性模型,应由非线性最小二乘法估计如下。

$$(1) \text{ 取双曲线模型: } \frac{1}{y} = a + b \frac{1}{x}$$

$$\text{令 } y' = \frac{1}{y}, x' = \frac{1}{x}, \text{ 则上式可以写成}$$

$$y' = a + bx'$$

将表 7.3 中的实验数据经过变换,按最小二乘法,可得到

$$y' = 0.0823 + 0.1312x'$$

因此实际模型为

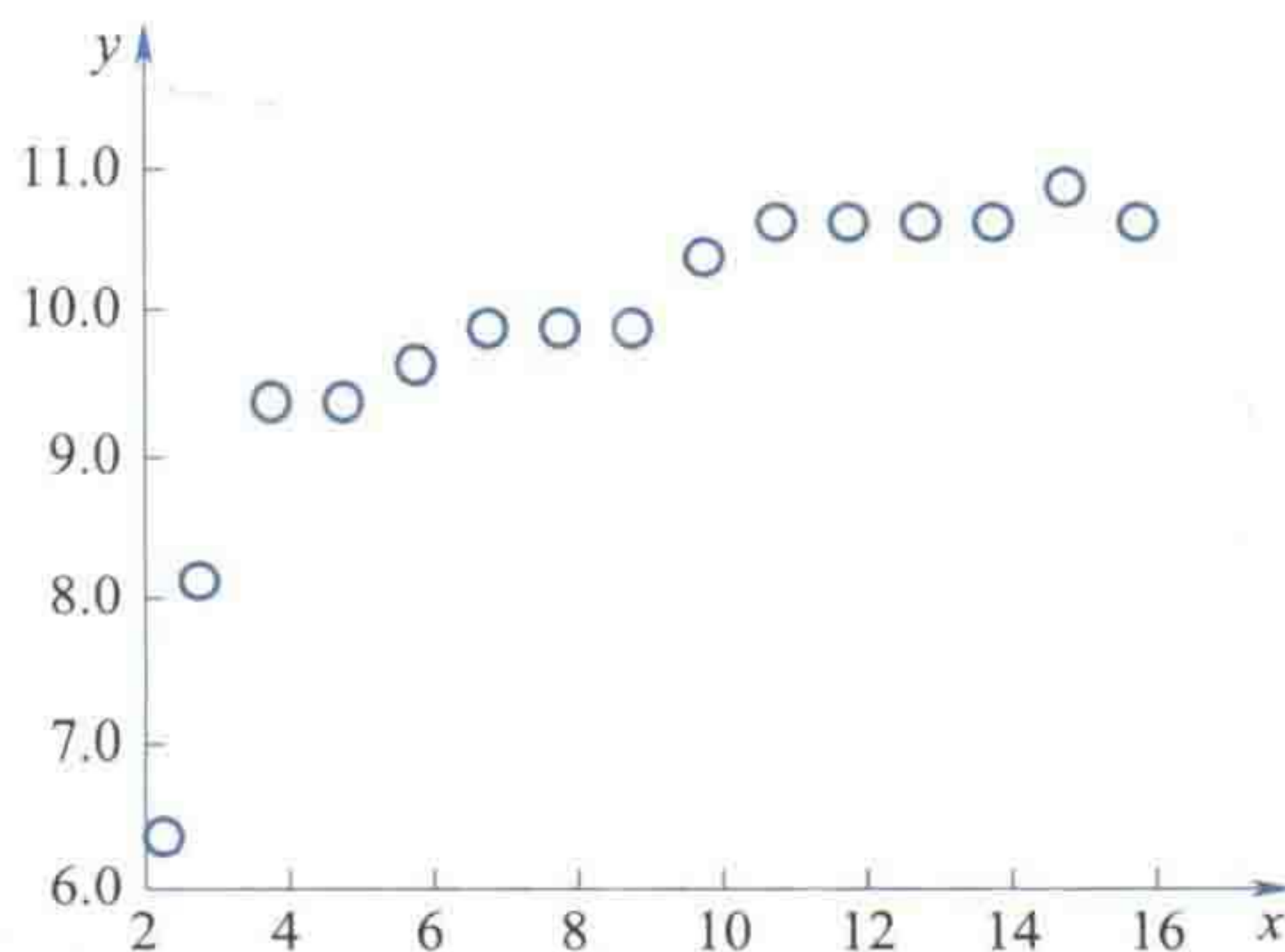


图 7.6 钢包容积与使用次数的实验数据散点图

$$\frac{1}{y} = 0.0823 + 0.1312 \frac{1}{x}$$

(2) 取指数模型: $y = ae^{\frac{b}{x}}$

两边取自然对数, 得

$$\ln y = \ln a + \frac{b}{x}$$

令 $y' = \ln y, a' = \ln a, x' = \frac{1}{x}$, 则有

$$y' = a' + bx'$$

由最小二乘法得 $a' = 2.4578, b = -1.1107$, 因此

$$y = 11.6791e^{-1.1107\frac{1}{x}}$$

对两种模型进行比较:

$$\sqrt{\sum_{i=1}^m (\varepsilon_i^{\text{双}})^2} = 1.119$$

$$\sqrt{\sum_{i=1}^m (\varepsilon_i^{\text{指}})^2} = 0.94$$

可见, 指数模型优于双曲线模型。

7.3 k 近邻机器学习算法

7.3.1 k 近邻算法原理

k 近邻(k-nearest neighbor, KNN)算法是最简单的有监督机器学习算法之一, 适用于多分类问题。KNN 分类的本质就是古语说的“近朱者赤, 近墨者黑”: 输入的待分类数据的特征接近“朱”色的特征, 就归入“赤”类; 输入的待分类数据的特征接近“墨”色的特征, 就归入“黑”类。

下面先通过一个直观的例子形象地说明 k 近邻算法。如图 7.7 所示, 如何将圆内的样本点归类为三角形或正方形? 若选取 $k=3$, 即选取训练集中距离待预测样本点最近的 3 个样本点 (如图中实线圆内)。可以看到实线圆中有 2 个样本点是三角形, 1 个样本点是正方形, 因此, 将待预测样本点归类为三角形。

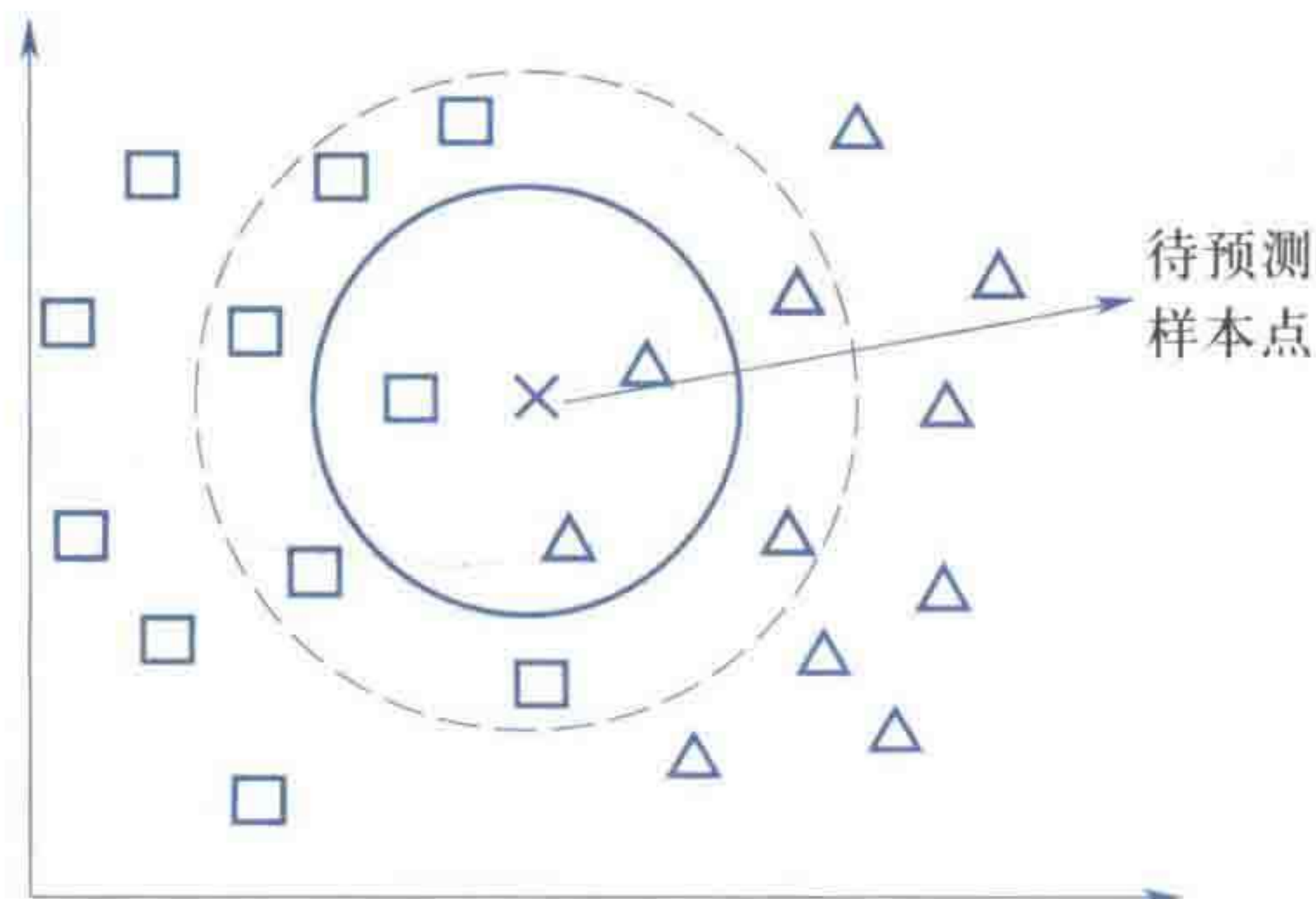


图 7.7 k 近邻算法示意

如果选取 $k=10$ 时,即选取训练集中距离待预测样本点最近的 10 个样本点(如图中虚线圆内)。可以看到虚线圆中有 6 个正方形而只有 4 个三角形,因此,将待预测样本点归类为正方形。

选取不同的 k 就得到截然不同的结果,可见, k 近邻算法对参数 k 的选择非常敏感。一般 k 选择为奇数,不超过 20。

7.3.2 k 近邻基本算法

k 近邻基本算法:给定一个训练样本集合,计算待分类样本和训练集中所有数据点的距离,将距离从小到大排列,然后取前 k 个数据点,将前 k 个数据点中数量最多的类别作为待分类样本的类别。

如何比较两个数据点之间的相似性?一般是用“相似度”“距离”来度量两个数据点之间的相似性。

设两个向量 $\mathbf{X}$ 和 $\mathbf{Y}$,每个向量维度是 n 维,表示为 $(x_1, x_2, \dots, x_i, \dots, x_n), (y_1, y_2, \dots, y_i, \dots, y_n)$,则相似度定义为

$$\text{sim}(\mathbf{X}, \mathbf{Y}) = \cos \theta = \frac{\sum_{i=1}^n x_i y_i}{\sqrt{\sum_{i=1}^n x_i^2} \sqrt{\sum_{i=1}^n y_i^2}} \quad (7.17)$$

其中,分子表示为两个向量点积,分母表示为各自的长度的乘积。

距离度量分为欧氏距离、曼哈顿距离等。欧氏距离、曼哈顿距离定义为:

欧氏距离

$$L = \sqrt{\sum_{i=1}^n (x_i - y_i)^2} \quad (7.18)$$

曼哈顿距离

$$L = \sum_{i=1}^n |x_i - y_i| \quad (7.19)$$

例如,向量 $\mathbf{A}=(1,2,2,4)$,向量 $\mathbf{B}=(1,1,1,1)$,则向量 $\mathbf{A}$ 的长度为 5,向量 $\mathbf{B}$ 的长度为 2, $\mathbf{A}$ 与 $\mathbf{B}$ 的点积为 9,则相似度为 $9/(5 \times 2)=0.9$ 。欧氏距离为 $\sqrt{11}$,曼哈顿距离为 5。

KNN 算法优点:简单且容易实现,支持多分类,并且不需要进行训练,可以直接用训练数据来实现分类。

KNN 算法的缺点:一是对参数的选择非常敏感。选取不同的参数,可能会得到完全不同的结果。二是计算量非常大,每次分类都需要计算未知数据和所有训练样本的距离,尤其当训练集非常大的时候,这个过程的计算量非常大。

7.3.3 k 近邻算法的应用举例

根据学生课程成绩、社会实践考核、计算机成绩、英语成绩,用 k 近邻算法判断这个学生就

业的企业类型。

例 7.3 已知 9 个学生的课程成绩、社会实践考核、计算机成绩、英语成绩和就业企业类型如表 7.3 所示。现在根据 10 号学生的各科成绩预测他的就业企业类型。

表 7.3 学生课程成绩和就业企业类型关系

学生序号	课程成绩	社会实践考核	计算机成绩	英语成绩	就业企业类型
1	92	65	91	85	外企
2	91	95	91	95	外企
3	65	65	80	85	私企
4	76	85	72	65	私企
5	65	65	66	65	国企
6	66	95	93	65	私企
7	68	90	80	65	私企
8	87	90	73	95	外企
9	65	75	85	85	外企
10	82	65	68	65	?

例如,利用 $\text{sim}(x,y)$ 计算 10 号(82,65,68,65)与前 9 个学生的成绩的相似度,然后将比对结果从小到大进行排序如表 7.4 所示。

表 7.4 相 似 度

学生序号	类别	10 号与其余学生成绩相似度
6	私企	0.970 9
9	外企	0.980 6
7	私企	0.980 9
3	私企	0.981 5
8	外企	0.989 4
4	私企	0.991 1
2	外企	0.993 2
1	外企	0.993 8
5	国企	0.994 8

如果 k 值取 1, 则 10 号学生就业企业类型和 6 号最相似, 所以 10 号学生就业企业类型确定为私企; 如果 k 值取 3, 前 3 个学生就业的企业中有两个是私企, 所以 10 号学生就业企业类型确定为私企; 如果 k 值取 5, 前 3 个私企 2 个外企, 10 号学生确定为私企。

7.4 决策树机器学习算法

7.4.1 决策树

决策树(decision tree)代表对象属性与对象值之间的一种映射关系, 是一类基于树结构进行决策的常见监督学习方法。

一棵决策树一般包含一个根结点、若干个内部结点和若干个叶结点。其中每个内部结点表示一个属性上的测试, 每个分支代表一个测试输出, 每个叶结点代表一种类别。

决策树学习的目的是产生一棵泛化能力强, 即处理未见示例能力强的决策树, 其基本流程遵循“分而治之”策略。

决策树的生成一般是一个递归过程。在决策树基本算法中, 有三种情形会导致递归返回:

- (1) 无须划分: 当前结点包含的样本全属于同一类别;
- (2) 无法划分: 当前属性集为空或是所有样本在所有属性上取值相同;
- (3) 不能划分: 当前结点包含的样本集合为空。

例 7.4 水果数据集的训练数据如表 7.5 所示, 每一行代表一个样本点, 从颜色、形状、大小三方面的特征来描述水果的类别。构造决策树对不同的样本进行分类。

表 7.5 水果数据集

编号	颜色	形状	大小	类别
1	红	球	一般	苹果
2	黄	弯月	一般	香蕉
3	红	球	轻	樱桃
4	绿	椭球	重	西瓜
5	橘黄	球	一般	橘子

解 利用不同的叶结点对应颜色、形状、大小等不同属性, 并分别进行测试, 进而得到最终的叶结点, 从而将所有样本的根据其属性分成不同的类别。

水果数据集决策树如图 7.8 所示。

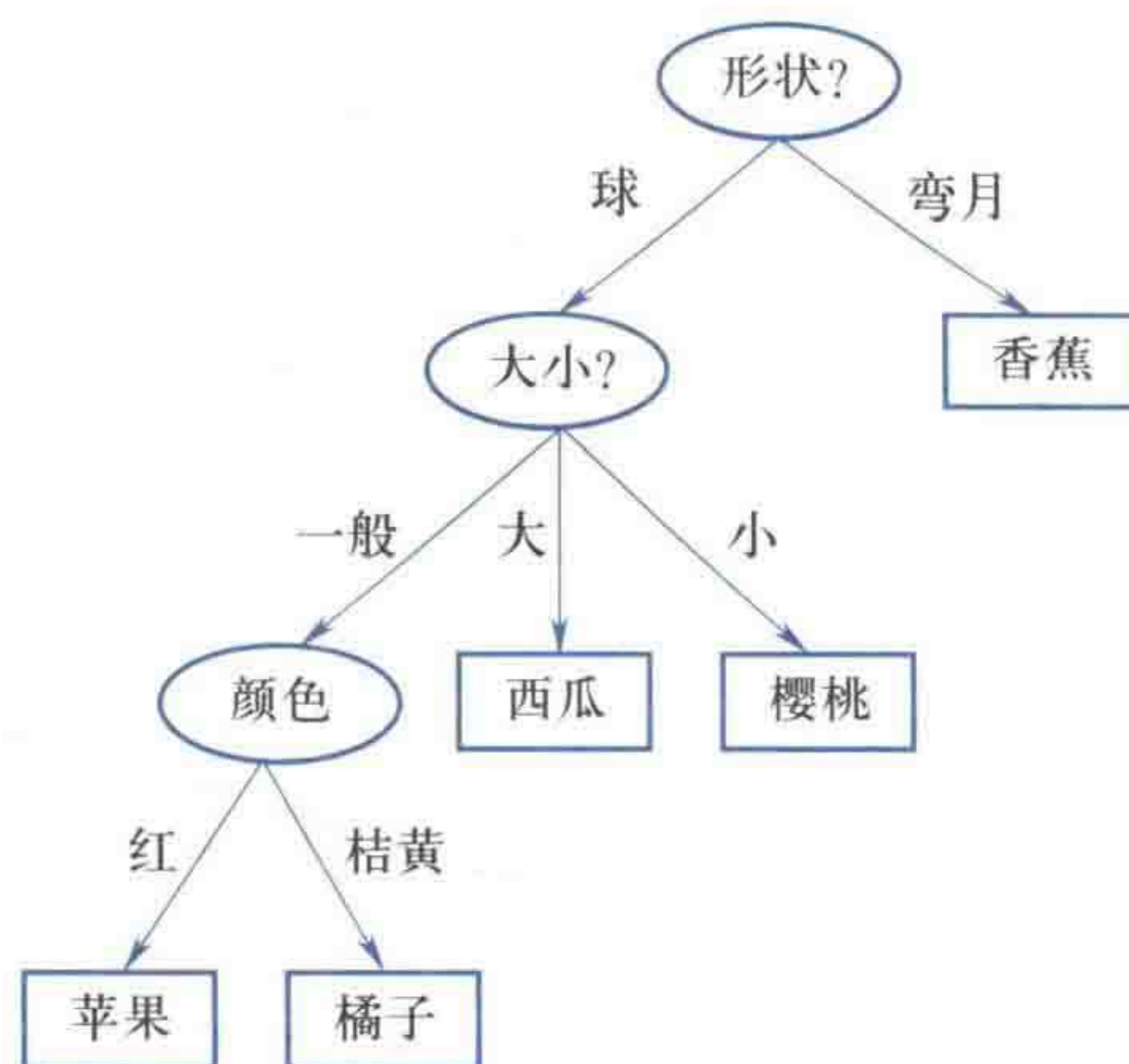


图 7.8 决策树示例

7.4.2 决策树的熵

决策树学习的关键是如何选择最优划分属性。一般而言,随着划分过程不断进行,希望决策树的分支结点所包含的样本尽可能属于同一类别,即结点的“纯度”(purity)越来越高。一般用熵(entropy)衡量结点的“纯度”。熵由公式定义:

$$H(X) = - \sum_{i=1}^n P(x_i) \log_b P(x_i) \quad (7.20)$$

其中, n 为样本个数; $p(x_i)$ 是第 i 个样本的概率; b 一般取 2、e 或者 10,这里取 2。

在信息论与概率统计中,熵表示随机变量的不确定性程度。变量的不确定程度越大,熵越大,反之越小。如果样本集合内全部为单类样本,则熵最小。

例如,投掷硬币一次,正面反面的概率都是 0.5,那么这个结果的熵可以表示为 $H(X) = -(0.5 \times \log_2 0.5 + 0.5 \times \log_2 0.5) = 1.0$ 。如果硬币正反面重量不同,一面重一面轻,比如正面概率 0.2,反面概率 0.8,那么投掷一次这样的硬币的结果熵为

$$H(X) = -(0.2 \times \log_2 0.2 + 0.8 \times \log_2 0.8) = 0.7219$$

因为反面的可能性更大,所以熵变小了,即不确定性变小了。

构建决策树的过程是通过样本集的划分,使得决策树尽快达到熵最小状态,也就是确定性状态。

7.4.3 决策树的优点

决策树是一种非线性模型,可用于解决二分类、多分类和回归问题。多决策树可以组成随机森林等树集成学习方法。决策树易于理解和实现,具有很强的解释性,通过解释容易理解决策树所表达的意义。

对于决策树,往往不需要准备大量的数据,且能够同时处理数据型和常规型属性,在相对

短的时间内能够对大型数据源做出可行高且效果良好的结果。同时,如果给定一个观察的模型,那么根据所产生的决策树很容易推出相应的逻辑表达式。

7.5 支持向量机

1995 年,Corinna Cortes 和 Vladimir Vapnik 提出了用于分类任务的支持向量机 (support vector machine, SVM),属于监督学习算法。由于 SVM 在诸多分类任务中显示出卓越性能,而且有良好的可解释性,很快成为机器学习的主流技术,掀起了机器学习的高潮。

7.5.1 支持向量机的基本思想

SVM 是一种非概率二元线性分类器,基本思想是创建一个超平面将训练样本划分成两类,并尽可能最大化两个类别之间的间隔。

图 7.9 所示一个二维平面(超平面类似讨论),二维平面上有两种不同类型的点,分别用实心点和空心点来表示。支持向量机的目标就是通过求解超平面,将不同属性的点分开。注意:在二维平面上超平面只是一条直线。

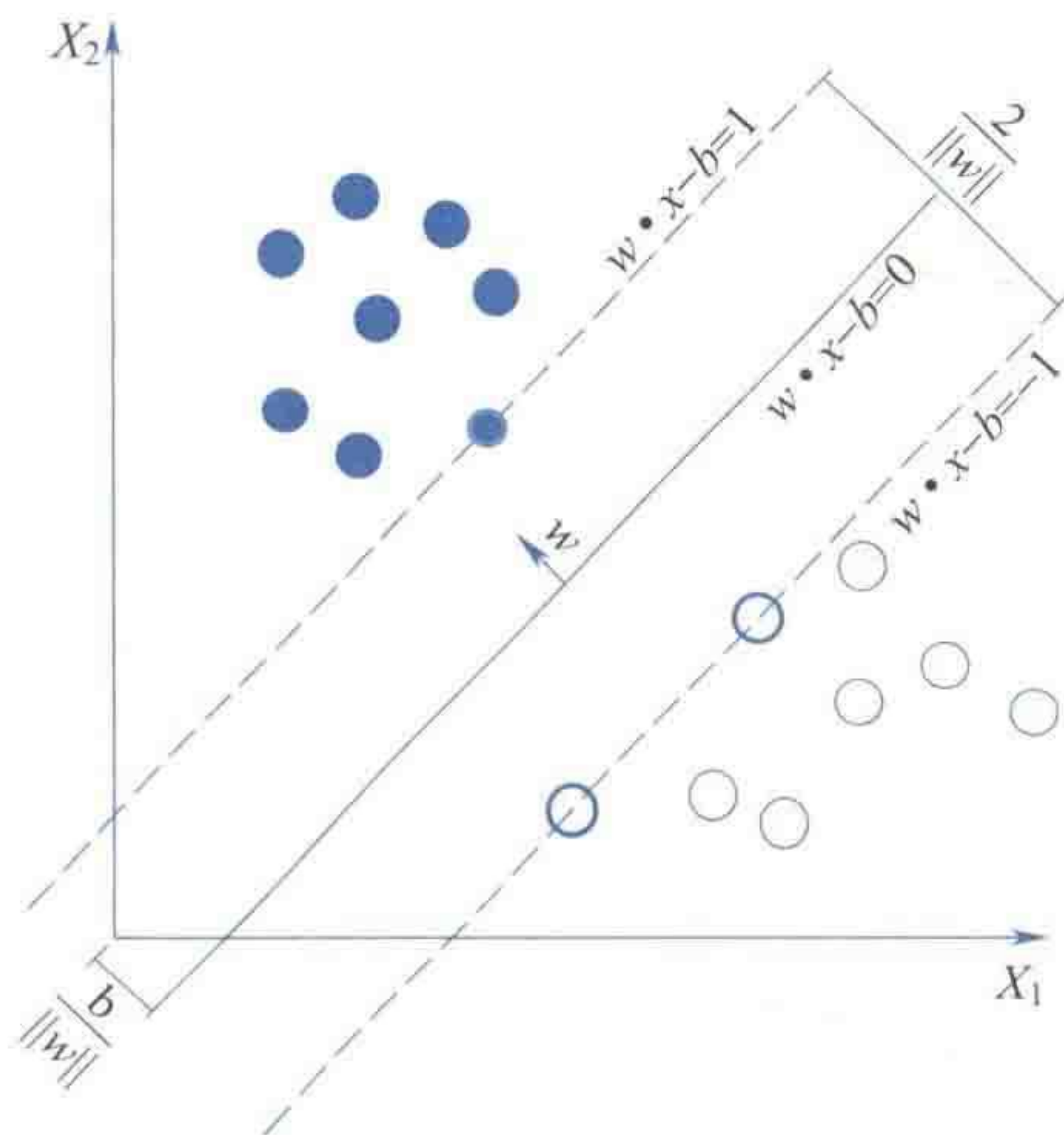


图 7.9 支持向量机

对某个数据点进行分类,当它的间隔越大时,即离超平面越远时,分类的置信度越大。对于一个包含 n 个点的数据集,可以定义它的间隔为所有 n 个点的间隔值中最小的那个。于是,为了提高分类的置信度,我们希望所选择的超平面能够最大化这个间隔值。这也就是 SVM 算法的基础,即最大间隔准则。

在图 7.9 中,距离超平面最近的这几个训练样本点被称为“支持向量”(support vector),两个异类支持向量到超平面的距离之和被称为“间隔”(margin)。支持向量机的目标就是找到具

有“最大间隔”的划分超平面。

7.5.2 线性支持向量机

给定一组训练实例,每个训练实例被标记为属于两个类别中的一个。例如,在超平面一边的数据点所对应的 y 记为 1,而在超平面另一边的数据点所对应的 y 记为 -1。

SVM 训练算法通过寻求结构化风险最小来提高学习机泛化能力,实现经验风险和置信范围的最小化,建立一个将新的实例分配给两个类别之一的模型,从而达到在统计样本量较少的情况下,亦能获得良好统计规律的目的。

SVM 模型是将实例表示为空间中的点,这样映射就使得单独类别的实例被尽可能大的间隔分开。然后,将新的实例映射到同一空间,并基于它们落在间隔的哪一侧来预测所属类别。

线性 SVM 模型是一种二类分类模型,是特征空间上间隔最大的线性分类器,即支持向量机的学习策略便是间隔最大化,最终可转化为一个凸二次规划问题的求解。

7.5.3 非线性支持向量机

线性支持向量机假设训练样本是线性可分的,即存在一个超平面能将训练样本正确分类。然而,在现实任务中,原始样本空间内可能并不存在一个能正确划分两类样本的超平面,如图 7.10(a)所示。很容易看出,实际上无法找到一个线性分类面(直线),将图中的实心样本和空心样本分开。

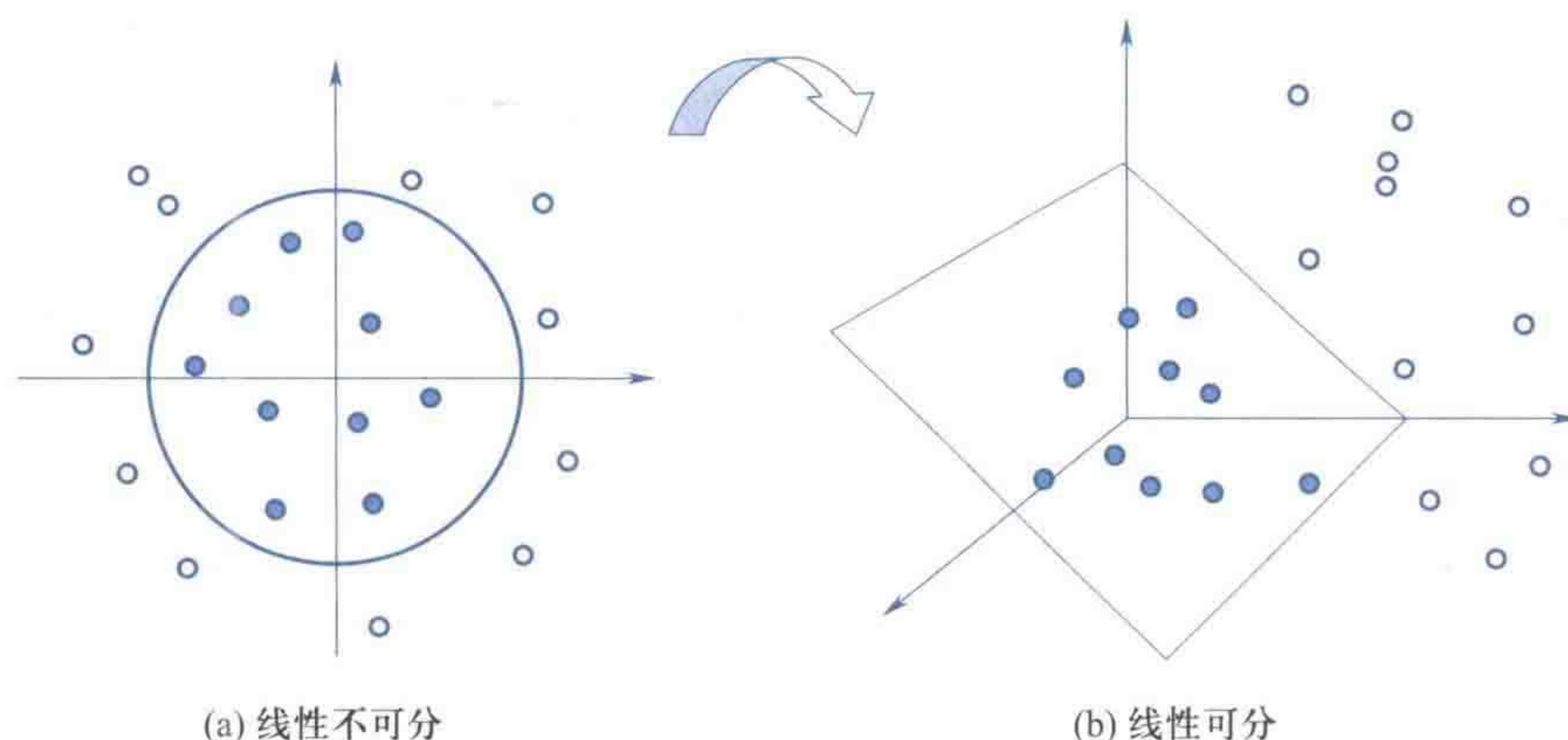


图 7.10 利用核函数方法将线性不可分映射到高维空间

为了解决这类线性不可分问题,提出了诸多解决办法,其中一个最重要的方法是核函数方法。核函数方法是通过选择一个核函数将数据映射到高维空间,使得在高维属性空间中训练数据有可能实现超平面的分割。这种核函数方法避免了在原输入空间中进行非线性曲面分割计算的困难,而是将原始空间中线性不可分的问题转换为另一个空间中线性可分的问题。如图 7.10 所示,通过核函数变换将原来在二维平面上的点映射到三维空间上,变换后显然是线

性可分的,再确定一个线性平面将图中的实心样本和空心样本分开。

由于核函数的良好性能,计算量只和支持向量的数量有关,而独立于空间的维度,在处理高维输入空间的分类时,这样的非线性扩展在计算量上并没有比原来有显著的增加。因此,核函数方法在当代的机器学习任务中有非常广泛的应用,尤其是在解决线性不可分问题时,而不仅仅局限在支持向量机算法中。

7.6 k 均值聚类算法

聚类是无监督学习中最重要的一类算法,具有非常广泛的应用。

给定一个由样本点组成的数据集,这些样本的标记信息是未知的。聚类是将样本集分为若干互不相交的子集,即样本簇。聚类算法的目标希望同一簇的样本尽可能彼此相似,即具有较高的类内相似度(intra-cluster similarity),同时不同簇的样本尽可能不同,即簇间的相似度(inter-cluster similarity)低。

可见,聚类结果事先是未知的,聚类过程仅能自动形成簇结构,簇所对应的概念语义需由使用者来把握和命名。

机器学习诞生以来,相关的研究者针对不同的问题提出了多种聚类方法,其中应用最广泛的是k 均值(k-means)聚类算法。k 均值算法无论是思想还是实现都比较简单。

k 均值聚类算法核心思想:如果将一组数据集聚成 k 类,则先随机地选取 k 个质心,然后计算每个样本到 k 个质心的距离,将对应的样本分到最近的类中,然后计算各个类的平均值,重新确定各个类的质心。迭代这个过程,直到质心不再变化。

对于给定样本集合,k 均值算法的目标函数是使聚类簇内的平方误差最小化,即

$$E = \sum_{i=1}^k \sum_{x \in C_i} \|x - \mu_i\|_2^2 \quad (7.21)$$

其中, k 是人为给定的簇的数量; μ_i 是簇 C_i 的均值向量; x 是对应的样本特征向量。

直观来看,这个误差刻画了簇内样本围绕簇均值向量的紧密程度, E 值越小则簇内样本相似度越高。

k 均值算法的求解通常采用贪心策略,通过迭代的方法实现。k 均值算法首先随机选择 k 个向量作为初始均值向量,然后是迭代过程,根据均值向量将样本划分到距离最近的均值向量所在的簇中,划分完成之后更新的均值向量,直到迭代完成。

k 均值算法对参数的选择比较敏感,不同的初始位置或者类别数量的选择往往会导致完全不同的结果。而很多情况下,我们无法事先预知样本的分布,最优参数的选择通常也非常困难。因此,k 均值算法得到的结果可能和我们的预期会有很大的不同,从而给模型学习带来很大不确定性。

k 均值算法时间复杂度近于线性,适合挖掘大规模数据集。但是,由于损失函数是 E 非凸函数,意味着我们不能保证取得的最小值是全局最小值。在实际应用中,通常k 均值达到的局

部最优已经满足需求。如果局部最优无法满足需求,简单的方法是通过不同的初始值多次计算来实现。

k 的数量是模糊的,通常可以随机选取,也可以根据一定的选取策略(如肘部法)预估 k 值等。

7.7 专家系统概述

7.7.1 专家系统的产生和发展

专家系统(expert system)的第一个里程碑是斯坦福大学费根鲍姆等人于1968年研制成功分析化合物分子结构的专家系统——DENDRAL系统,它达到了专家的水平。此后,相继建立了各种不同功能、不同类型的专家系统。MACSYMA系统是由麻省理工学院(MIT)于1971年开发成功并投入应用的专家系统,用LISP语言实现对特定领域的数学问题进行有效的处理,包括微积分运算、微分方程求解等。DENDRAL和MACSYMA系统是专家系统发展的第一阶段。这个时期专家系统的特点是:高度的专业化,专门问题求解能力强,但结构、功能不完整,移植性差,缺乏解释功能。

20世纪70年代中期,专家系统进入了第二阶段——技术成熟期,出现了一批成功的专家系统。具有代表性的专家系统是MYCIN、PROSPECTOR、AM、CASNET等系统。特别是PROSPECTOR系统,它是由斯坦福研究所开发的一个探矿专家系统。它首次实地分析华盛顿某山区一带的地质资料,发现了一个钼矿,成为第一个取得显著经济效益的专家系统。

第二阶段的专家系统的特点是:

- ① 单学科专业型专家系统;
- ② 系统结构完整,功能较全面,移植性好;
- ③ 具有推理解释功能,透明性好;
- ④ 采用启发式推理、不精确推理;
- ⑤ 用产生式规则、框架、语义网络表达知识;
- ⑥ 用限定性英语进行人机交互。

20世纪80年代以来,专家系统的研制和开发明显地趋向于商业化,直接服务于生产企业,产生了明显的经济效益。2012年以来,大数据驱动的深度学习、大语言模型等为机器学习等注入了强大动力,推动了专家系统的快速发展。但一些复杂的专家系统开发仍然存在许多问题。如2013年,IBM与世界顶级肿瘤治疗与研究机构——MD安德森癌症中心合作开发的癌症诊断与治疗专家系统Watson,用于辅助医生开展抗癌药物的临床测试,但Watson近几年进展缓慢,难以大量应用。Watson系统面临的窘境,其实也是整个专家系统现状的缩影。造成专家系统发展乏力的因素有很多,主要原因在于专家数据匮乏而昂贵,也就是知识获取成了问题。

7.7.2 专家系统的定义与特点

专家系统是基于知识的系统,它在某种特定的领域中运用领域专家多年积累的经验和专业知识,求解只有专家才能解决的困难问题。专家系统作为一种计算机系统,继承了计算机快速、准确的特点,在某些方面比人类专家更可靠、更灵活,可以不受时间、地域及人为因素的影响。

专家系统的奠基人、斯坦福大学的费根鲍姆(E. A. Feigenbaum)教授,把专家系统定义为“一种智能的计算机程序,它运用知识和推理来解决只有专家才能解决的复杂问题”。也就是说,专家系统是一种模拟专家决策能力的计算机系统。

专家系统的主要特点有:

1. 具有专家水平的专业知识

具有专家的专业水平是专家系统的最大特点。专家系统中的知识按其在问题求解中的作用可分为三个层次,即数据级、知识库级和控制级。数据级知识是指具体问题所提供的初始事实及在问题求解过程中所产生的中间结论、最终结论。数据级知识通常存放于数据库中。知识库知识是指专家的知识。这一类知识是构成专家系统的基础。控制级知识也称为元知识,是关于如何运用前两种知识的知识,如在问题求解中的搜索策略、推理方法等。专家系统具有的知识越丰富,质量越高,解决问题的能力就越强。

2. 能进行有效的推理

专家系统的核心是知识库和推理机。专家系统要利用专家知识来求解领域内的具体问题,必须有一个推理机构,能根据用户提供的已知事实,通过运用知识库中的知识,进行有效的推理,以实现问题的求解。专家系统不仅能根据确定性知识进行推理,而且能根据不确定的知识进行推理。领域专家解决问题的方法大多是经验性的,表示出来往往是不精确的,仅以一定的可能性存在。要解决的问题本身所提供的信息往往也是不确定的。专家系统的特点之一就是能综合利用这些不确定的信息和知识进行推理,得出结论。

3. 具有启发性

专家系统除能利用大量专业知识以外,还必须利用经验的判断知识来对求解的问题做出多个假设。依据某些条件选定一个假设,使推理继续进行。

4. 具有灵活性

专家系统的知识库与推理机既相互联系又相互独立。相互联系保证了推理机利用知识库中的知识进行推理以实现问题的求解;相互独立保证了当知识库作适当修改和更新时,只要推理方式不变,推理机部分就可以不变,使系统易于扩充,具有较大的灵活性。

5. 具有透明性

在使用专家系统求解问题时,不仅希望得到正确的答案,而且还希望知道得到该答案的依据。专家系统一般都有解释机构,向用户解释推理过程,回答用户“为什么(why)”“结论是如何得出的(how)”等问题。

6. 具有交互性

专家系统一般都是交互式系统,具有较好的人机界面。一方面它需要与领域专家和知识工程师进行对话以获取知识,另一方面它也需要不断地从用户那里获得所需的已知事实,并回答用户的询问。

专家系统本身是一个程序,但它与传统程序又不同。主要体现在以下几个方面。

① 从编程思想来看,传统程序是依据某个确定的算法和数据结构来求解某个确定的问题,而专家系统求解的许多问题没有可用的数学方法,而是依据知识和推理来求解,即

传统程序 = 数据结构 + 算法

专家系统 = 知识 + 推理

这是专家系统与传统程序的最大区别。

② 传统程序把关于问题求解的知识隐含于程序中,而专家系统则将知识与运用知识的过程(即推理机)分离。这种分离使专家系统具有更大的灵活性,便于修改。

③ 从处理对象来看,传统程序主要是面向数值计算和数据处理,而专家系统是面向符号处理的。传统程序处理的数据是精确的,而专家系统处理的数据和知识大多是不精确的、模糊的。

④ 传统程序一般不具有解释功能,而专家系统一般具有解释机构解释自己的行为。因为专家系统依赖于推理,它必须能够解释这个过程。

⑤ 从系统的体系结构来看,传统程序与专家系统具有不同的结构。关于专家系统的结构在后面将作专门的介绍。

⑥ 传统程序根据算法求解问题,每次都能产生正确的答案,而专家系统则像人类专家那样工作,一般能产生正确的答案,但有时也会产生错误的答案,这也是专家系统存在的问题之一。但专家系统有能力从错误中吸取教训,改进对某一问题的求解能力。

2016年3月在韩国举行的谷歌AlphaGo对决世界围棋冠军李世石的比赛中,AlphaGo连胜3局。在13日第4局比赛的78手之前,全世界的人都认为AlphaGo必赢。但此后AlphaGo犯了一个连小学生都不应该犯的错误,导致了第4局比赛的失败。其实,这正是专家系统有时会产生错误答案的特点。

7.7.3 专家系统的类型

若按专家系统的特性及功能分类,专家系统可分为10类。

1. 解释型专家系统

解释型专家系统能根据感知数据,经过分析、推理,从而给出相应解释。如化学结构说明、图像分析、语言理解、信号解释、地质解释、医疗解释等专家系统。代表性的解释型专家系统有DENDRAL、PROSPECTOR等。

2. 诊断型专家系统

诊断型专家系统能根据取得的现象、数据或事实推断出系统是否有故障,并能找出产生故

障的原因,给出排除故障的方案。这是目前开发、应用得最多的一类专家系统,如医疗诊断、机械故障诊断、计算机故障诊断等专家系统。代表性的诊断型专家系统有 MYCIN、CASNET、PUFF(肺功能诊断系统)、PIP(肾脏病诊断系统)、DART(计算机硬件故障诊断系统)等。

3. 预测型专家系统

预测型专家系统能根据过去和现在的信息(数据和经验)推断可能发生和出现的情况,例如用于天气预报、地震预报、市场预测、人口预测、灾难预测等领域的专家系统。

4. 设计型专家系统

设计型专家系统能根据给定要求进行相应的设计,例如用于工程设计、电路设计、建筑及装修设计、服装设计、机械设计及图案设计的专家系统。对这类系统一般要求在给定的限制条件下能给出最佳的或较佳的设计方案。代表性的设计型专家系统有 XCON(计算机系统配置系统)、KBVLSI(VLSI 电路设计专家系统)等。

5. 规划型专家系统

规划型专家系统能按给定目标拟定总体规划、行动计划、运筹优化等,适用于机器人动作控制、工程规划、军事规划、城市规划、生产规划等。这类系统一般要求在一定的约束条件下能以较小的代价达到给定的目标。代表性的规划型专家系统有 NOAH(机器人规划系统)、SECS(制定有机合成规划的专家系统)、TATR(帮助空军制订攻击敌方机场计划的专家系统)等。

6. 控制型专家系统

控制型专家系统能根据具体情况,控制整个系统的行为,适用于对各种大型设备及系统进行控制。为了实现对控制对象的实时控制,控制型专家系统必须具有能直接接收来自控制对象的信息,并能迅速地进行处理,及时地作出判断和采取相应行动的能力。所以控制型专家系统实际上是专家系统技术与实时控制技术相结合的产物。代表性的控制型专家系统是 YES/MVS(帮助监控和控制 MVS 操作系统的专家系统)。

7. 监督型专家系统

监督型专家系统能完成实时的监控任务,并根据监测到的现象做出相应的分析和处理。这类系统必须能随时收集任何有意义的信息,并能快速地对得到的信号进行鉴别、分析和处理。一旦发现异常,能尽快地做出反应,如发出报警信号等。代表性的监督型专家系统是 REACTOR(帮助操作人员检测和处理核反应堆事故的专家系统)。

8. 修理型专家系统

修理型专家系统是用于制定排除某类故障的规划并实施排除的一类专家系统,要求能根据故障的特点制定纠错方案,并能实施该方案排除故障。当制定的方案失效或部分失效时,能及时采取相应的补救措施。

9. 教学型专家系统

教学型专家系统主要适用于辅助教学,并能根据学生学习过程中所产生的问题进行分析、评价、找出错误原因,有针对性地确定教学内容或采取其他有效的教学手段。代表性的教学型

专家系统有 GUIDON(讲授有关细菌传染性疾病方面的医学知识的计算机辅助教学系统)。

10. 调试型专家系统

调试型专家系统用于对系统进行调试,能根据相应的标准检测出被检测对象存在的错误,并能从多种纠错方案中选出适用于当前情况的最佳方案,排除错误。

上面是根据专家系统的特性及功能对专家系统进行分类的。这种分类往往不是很确切,因为许多专家系统不止一种功能。还可以从其他的角度对专家系统进行分类。例如,可以根据专家系统的应用领域进行分类。当前专家系统主要的应用领域有:医学、计算机系统、电子学、工程、地质学、军事科学、过程控制等。

7.8 专家系统的工作原理

7.8.1 专家系统的一般结构

由专家系统的定义可知,专家系统的主要组成部分是知识库和推理机。实际专家系统的功能和结构可能彼此有些差异,但完整的专家系统一般应包括人机接口、推理机、知识库、综合数据库、知识获取机构和解释机构六部分。各部分的关系如图 7.11 所示。

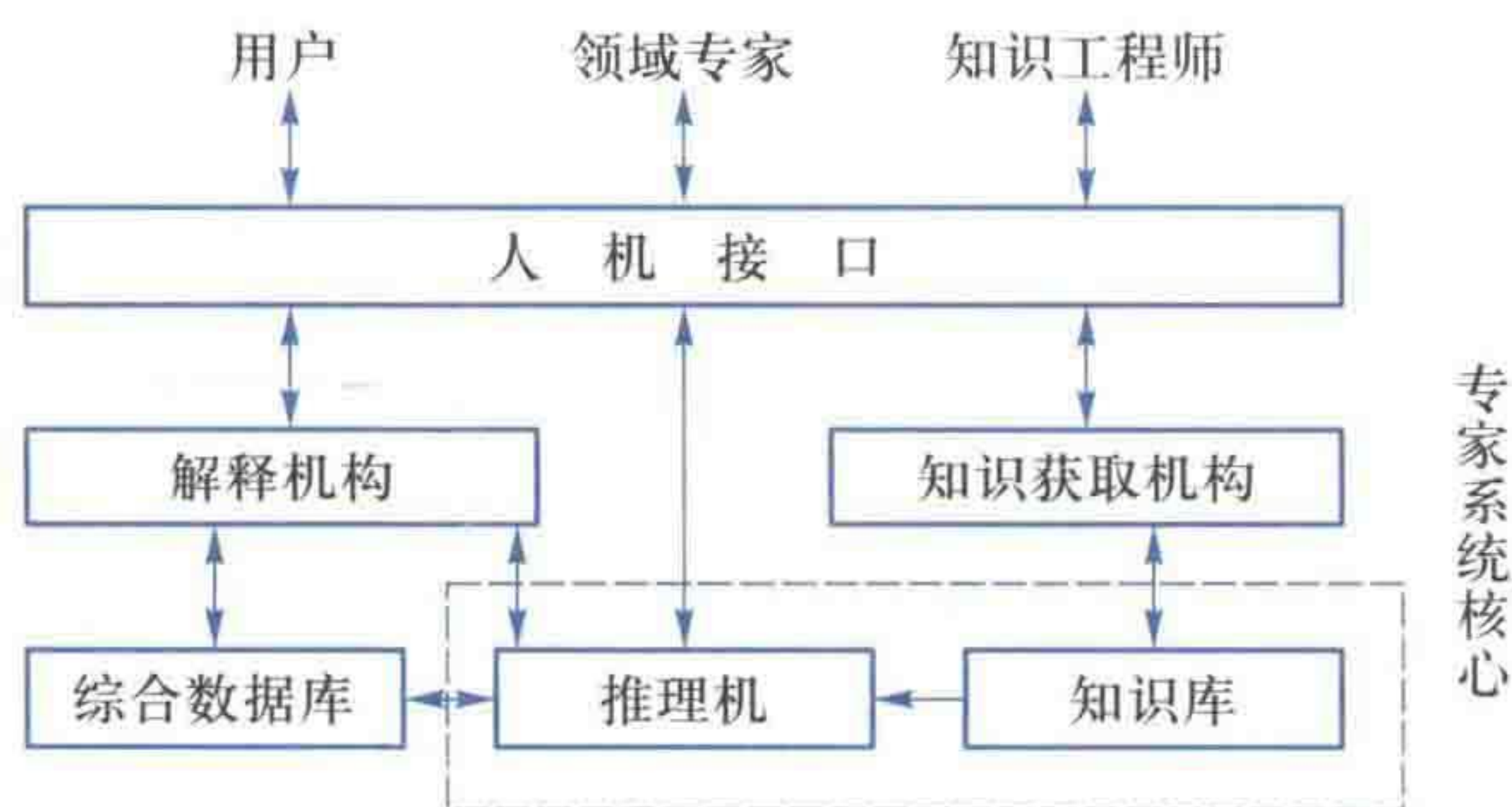


图 7.11 专家系统的一般结构

专家系统的核心是知识库和推理机,其工作过程是根据知识库中的知识和用户提供的事实进行推理,不断地由已知的事实推出未知的结论即中间结果,并将中间结果放到数据库中,作为已知的新事实进行推理,从而把求解的问题由未知状态转换为已知状态。在专家系统的运行过程中,会不断地通过人机接口与用户进行交互,向用户提问,并向用户做出解释。

下面分别对专家系统的各个部分进行简单介绍。

7.8.2 知识库

知识库(knowledge base)主要用来存放领域专家提供的有关问题求解的专门知识。知识库中的知识来源于知识获取机构,同时它又为推理机提供求解问题所需的知识。

1. 知识表示方法的选择

要建立知识库,首先要选择合适的知识表达方法。对同一知识,一般都可以用多种方法进行表示,但其效果却不同。应从能充分表示领域知识、能充分有效地进行推理、便于对知识的组织维护和管理、便于理解与实现等四个方面选择知识表达方法。本书介绍了一阶谓词逻辑、产生式、框架、模糊逻辑、状态空间、遗传编码、神经网络等知识表示方法。

2. 知识库的管理

知识库管理系统负责对知识库中的知识进行组织、检索、维护等。专家系统中任何其他部分要与知识库发生联系,都必须通过该管理系统来完成。

在进行知识库维护时,还要保证知识库的安全性。必须建立严格的安全保护措施,以防止由于操作失误等主观原因使知识库遭到破坏,造成严重的后果。一般知识库的安全保护也可以像数据库系统那样,通过设置口令验证操作者的身份,对不同操作者设置不同的操作权限等技术来实现。

7.8.3 推理机

推理机(reasoning machine)的功能是模拟领域专家的思维过程,控制并执行对问题的求解。它能根据当前已知的事实,利用知识库中的知识,按一定的推理方法和控制策略进行推理,直到得出相应的结论为止。

推理机包括推理方法和控制策略两部分。推理方法有确定性推理和不确定性推理。控制策略主要指推理方法的控制及推理规则的选择策略。推理包括正向推理、反向推理和正反向混合推理。推理策略一般还与搜索策略有关。本书第3章和第4章分别介绍了基本的确定性推理方法和不确定性推理方法。

推理机的性能与构造一般与知识的表示方法有关,但与知识的内容无关,这有利于保证推理机与知识库的独立性,提高专家系统的灵活性。

7.8.4 综合数据库

综合数据库(global database)又称动态数据库或黑板,主要用于存放初始事实、问题描述及系统运行过程中得到的中间结果、最终结果等信息。

在开始求解问题时,综合数据库中存放的是用户提供的初始事实。综合数据库的内容随着推理过程的进行而变化,推理机会根据综合数据库的内容从知识库中选择合适的知识进行推理并将得到的中间结果存放于综合数据库中。综合数据库中记录了推理过程中的各种有关信息,又为解释机构提供了回答用户咨询的依据。

综合数据库中还必须具有相应的数据库管理系统,负责对数据库中的知识进行检索、维护等。

从计算机技术角度,知识库和综合数据库都是数据库。它们所不同的是:知识库的内容在专家系统运行过程中是不改变的,只有知识工程师通过人机接口进行管理,而综合数据库在专

家系统运行过程中是动态变化的,不仅可以由用户输入数据,推理的中间结果的存入也会改变其内容。

7.8.5 知识获取机构

知识获取(knowledge acquisition)是建造和设计专家系统的关键,也是目前建造专家系统的“瓶颈”。知识获取的基本任务是为专家系统获取知识,建立起健全、完善、有效的知识库,以满足求解领域问题的需要。

知识获取通常是由知识工程师与专家系统中的知识获取机构共同完成的。知识工程师负责从领域专家那里抽取知识,并用适合的方法把知识表达出来。知识获取机构把知识转换为计算机可存储的内部形式,然后把它们存入知识库。在存储过程中,要对知识进行一致性、完整性的检测。

知识源包括专家、书本、相关数据库、实例研究和个人经验等。目前专家系统的知识源主要是领域专家,所以知识获取过程需要知识工程师与领域专家反复交流,共同合作完成,如图 7.12 所示。



图 7.12 知识获取的过程

不同专家系统的知识获取功能与实现方法差别较大,有的系统采用自动获取知识的方法,而有的系统则采用非自动或半自动知识获取的方法。

按知识获取的自动化程度划分,知识获取主要有非自动、自动和半自动三种获取模式。

1. 非自动知识获取(人工移植)

非自动知识获取也称为人工移植。在这种方式中,知识获取分两步进行,首先由知识工程师从领域专家或有关的科技文献那里获取知识,然后再由知识工程师用某种知识编辑器输入到知识库中,其工作方式如图 7.13 所示。

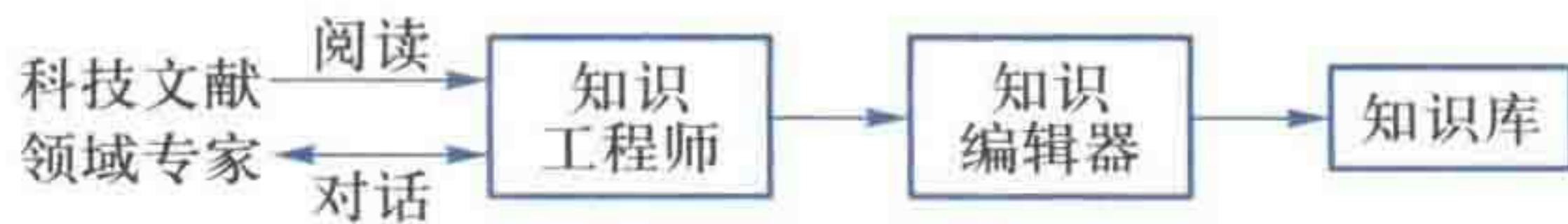


图 7.13 非自动知识获取

非自动方式是专家系统建造中用得较为普遍的一种知识获取模式。

2. 自动知识获取

所谓自动知识获取是指系统具有获取知识的能力。它不仅可以直接与领域专家对话,从专家提供的原始信息中学习得到专家系统所需的知识,而且还能从系统自身的运行实践中总结、

归纳出新的知识,发现知识中可能存在的错误,不断自我完善,建立起性能优良、知识完善的知识库。因此,它具有识别语言、文字、图像的能力、理解分析归纳的能力。具有从运行实践中学习的能力。自动知识获取的过程如图 7.14 所示。

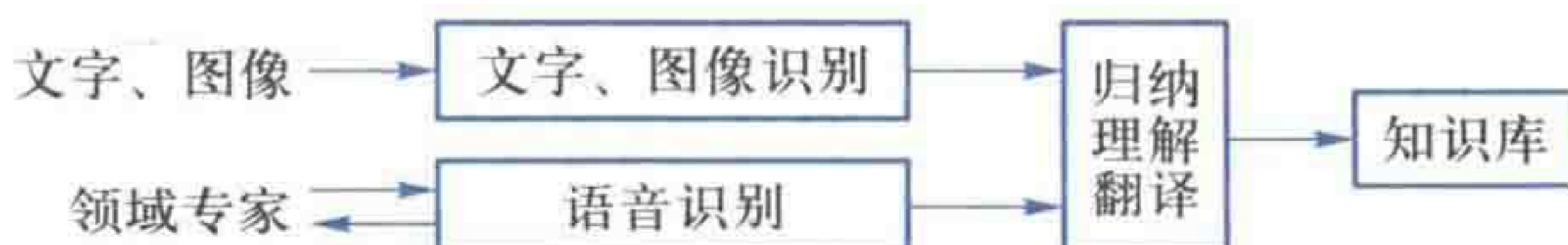


图 7.14 自动知识获取

3. 半自动知识的获取

在非自动知识获取的基础上增加了部分学习功能,使系统能从大量事例中归纳出某些知识。由于这样的系统不同于纯粹的非自动知识获取,但又没有达到完全自动知识获取的程度,因而称之为半自动知识获取。

7.8.6 人机接口

人机接口(interface)是专家系统与领域专家、知识工程师、一般用户之间进行交互的界面,由一组程序及相应的硬件组成,用于完成输入输出工作。知识获取机构通过人机接口与领域专家及知识工程师进行交互,更新、完善、扩充知识库;推理机通过人机接口与用户交互。在推理过程中,专家系统根据需要不断向用户提问,以得到相应的事实数据,在推理结束时会通过人机接口向用户显示结果;解释机构通过人机接口与用户交互,向用户解释推理过程,回答用户问题。

在输入或输出过程中,人机接口需要内部表示形式与外部表示形式的转换。在输入时,它将把领域专家、知识工程师或一般用户输入的信息转换成系统的内部表示形式,然后分别交给相应的机构去处理;输出时,它将把系统要输出的信息由内部形式转化为人们易于理解的外部形式显示给用户。

在不同的专家系统中,由于硬件、软件环境不同,接口的形式与功能有较大的差别。随着计算机硬件和自然语言理解技术的发展,有的专家系统已经可以用简单的自然语言与用户交互,但有的系统只能通过菜单方式、命令方式或简单的问答方式与用户交互。

7.8.7 解释机构

解释机构(explanator)回答用户提出的问题,解释系统的推理过程。解释机构由一组程序组成。它跟踪并记录推理过程,当用户提出的询问需要给出解释时,它将根据问题的要求分别作相应的处理,最后把解答用约定的形式通过人机接口输出给用户。

上面讨论的专家系统的一般结构只是专家系统的基本形式。实际上。在具体建造一个专家系统时,随着系统要求的不同,可以在此基础上做适当修改。

7.9 专家系统的建立

7.9.1 专家系统的一般建立过程

专家系统是人工智能中广泛应用但也正在不断发展的一个研究领域,虽然目前已建立了许多专家系统,但是尚未形成建立专家系统的一般方法。下面简单介绍专家系统的一般建立过程。

专家系统是一个计算机软件系统,但与传统程序又有区别,因为知识工程与软件工程在许多方面有很大的差别,所以专家系统的开发过程与软件工程类似,但又有区别。例如,软件工程设计目标是建立一个用于事物处理的信息处理系统,处理的对象是数据,主要功能是查询、统计、排序等,其运行机制是确定的;而知识工程的设计目标是建立一个辅助人类专家的知识处理系统,处理的对象是知识和数据,主要的功能是推理、评估、规划、解释、决策等,其运行机制难以确定。另外从系统的实现过程来看,知识工程比软件工程更强调渐进性、扩充性。因此,在设计专家系统时,软件工程设计思想及过程虽可以借鉴,但不能完全照搬。

专家系统的开发步骤一般分为问题识别、概念化、形式化、实现和测试等阶段,如图 7.15 所示。

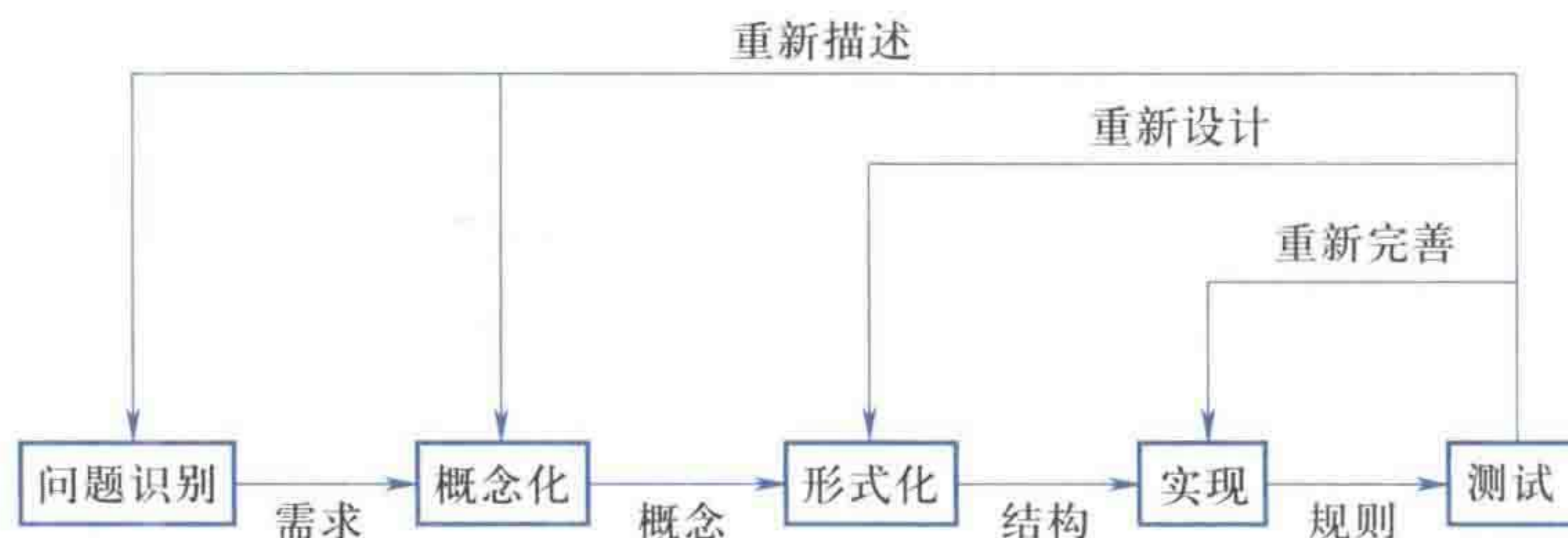


图 7.15 专家系统开发步骤

(1) 问题识别阶段。在问题识别阶段,知识工程师和专家将选定包括领域专家和知识工程师在内的参加人员,并明确各自的任务。描述问题的特征及相应的知识结构,明确问题的类型和范围。确定知识源、时间、计算设备以及经费等资源。确定问题求解的目标。

(2) 概念化阶段。主要任务是揭示描述问题所需要的关键概念、关系和控制机制,将问题划分为子任务、问题求解的策略和有关求解的约束。主要有:确定什么类型的数据有用,数据之间的关系如何,问题求解时包括哪些过程,这些过程有哪些约束。

(3) 形式化阶段。把概念化阶段概括出来的关键概念、子任务和信息流特征形式化地表示出来。究竟采用什么形式,要根据问题的性质选择适当的专家系统构造工具或适当的系统框架,确定知识表示形式和问题的求解策略,选定构造工具或系统框架。在这个阶段,知识工程师起着更积极的作用。

(4) 实现阶段。把前一阶段的形式化知识变成计算机软件,即实现知识库、推理机、人机接口和解释系统。在建立专家系统的过程中,原型系统的开发是极其重要的步骤之一。对于选定的表达方式,任何有用的知识工程辅助手段(如编辑、智能编辑或获取程序)都可以用来完成原型系统知识库。另外推理机应能模拟领域专家求解问题的思维过程和控制策略。

(5) 测试阶段。通过运行实例评价原型系统以及用于实现它的表达形式,从而发现知识库和推理机的缺陷,例如,数据获取与结论表示方法存在缺陷;推理规则有错误、不一致或不完备;控制策略不是按专家采用的“自然顺序”解决问题。测试的主要内容包括:系统结论的可靠性、获取知识的正确性、查询及推理的运行效率、解释能力以及人机交互的便利性。对人机接口的测试主要由最终用户来进行。在测试过程中,实例的选择应照顾到各个方面,要有较宽的覆盖面,既要涉及典型的情况,也要涉及边缘的情况。

7.9.2 专家系统开发环境

专家系统开发环境又称为专家系统开发工具包,它可为专家系统的开发提供多种方便的构件,如知识获取的辅助工具、适用各种不同知识结构的知识表示模式、各种不同的不确定推理机制、知识库管理系统以及各种不同的辅助工具、调试工具等。目前,国内外已有的专家系统开发环境有 AGE、KEE 等。这里只简单地介绍 AGE。

AGE 是斯坦福大学研制的一个专家系统开发环境。它是一种典型的模块组合式开发工具。AGE 为用户提供了一个通用的专家系统结构框架,并将该框架分解为许多在功能和结构上较为独立的组件部件。这些组件已预先编制成标准模块存在系统中。应用 AGE 已经开发了一些专家系统,主要用于医疗诊断、密码翻译、军事科学等方面。

可以用许多程序设计语言开发专家系统。PROLOG 和 LISP 是两种早期的人工智能程序设计语言。用 PROLOG、LISP 编写的专家系统一般难以嵌入用其他语言编写的程序。

为了克服 PROLOG 和 LISP 运行速度慢、可移植性差、解决复杂问题能力差等问题,1984 年美国航空航天局约翰逊空间中心推出 CLIPS(C language intergrated production system),它是一个基于 Rete 算法的前向推理语言,用标准 C 语言编码,具有较高移植性、扩展性、知识表达能力和较低成本等特点。

选择人工智能语言的一个重要原因是它提供了一些工具。由于可移植性、效率和速度等原因,许多专家系统工具,现在都用 Python、C/C++、Java 等语言编写。由于面向对象程序设计语言以其类、对象、继承等机制,与人工智能的知识表示、知识库等产生了自然的联系。因而,现在面向对象型语言也成为一种人工智能程序设计语言,特别是专家系统程序设计。

7.9.3 骨架系统的概念

专家系统的研制和开发是一件复杂、困难、费时的的工作。为了提高专家系统设计和开发的效率,缩短研制周期,就需要使用专家系统开发工具,以便于提供系统设计和开发的计算机辅助手段和环境。

专家系统的一个特点是知识库与系统其他部分的分离,知识库是与求解的问题领域密切相关的,而推理机等则与具体领域独立,具有通用性。为此,人们将描述领域知识的规则等从原系统中“挖掉”,只保留其知识表示方法和与领域无关的推理机等部分,就得到了一个专家系统的骨架系统,因为它保留了原有系统的主要框架。

骨架系统是由已有的成功的专家系统演化而来的。它抽出了原系统中具体的领域知识,而保留了原系统的体系结构和功能,再把领域专用的界面改为通用界面。

在骨架系统中,知识表示模式、推理机制都是确定的。利用骨架系统作为开发工具,只要将新的领域知识用骨架系统规定的模式表示出来,并装入到知识库中就可以了。

7.9.4 EMYCIN 骨架系统

EMYCIN 系统是由 MYCIN 系统抽去原有的医学领域知识,保留骨架而形成的系统。

MYCIN 系统是由斯坦福大学 1972 年开始研制的用于对细菌感染性疾病进行诊断和治疗的专家系统。MYCIN 的功能是帮助内科医生诊断细菌感染疾病,并给出建议性的诊断结果和处方。MYCIN 系统是将产生式规则从通用问题求解转移到解决专门问题的一个成功典范,在专家系统的发展中占有重要的地位,许多专家系统就是在它的基础上建立起来的。

MYCIN 系统的结构如图 7.16 所示。从图中可以看出,MYCIN 系统主要由咨询、解释和知识获取三个模块以及知识库、动态数据库组成。它采用产生式规则表达知识、目标驱动的反向推理控制策略,特别适合开发领域咨询、诊断型专家系统。

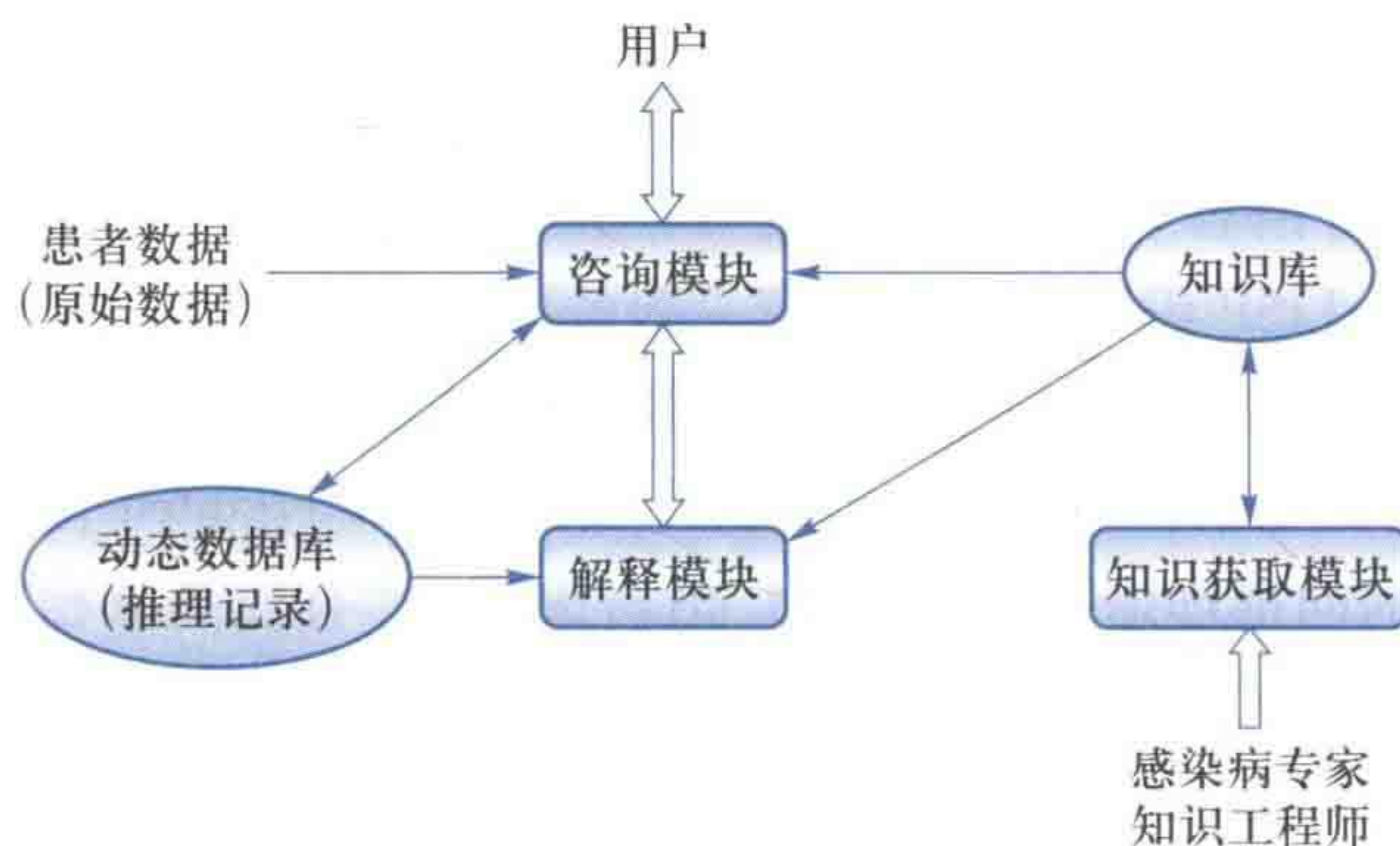


图 7.16 MYCIN 系统结构图

EMYCIN 系统具有 MYCIN 系统的全部功能,例如以下几个功能。

- (1) 解释程序。系统可以向用户解释推理过程。
- (2) 知识编辑程序及类英语的简化会话语言。EMYCIN 系统提供了一个开发知识库的环境,使开发者可以使用比 LISP 更接近自然语言的规则语言来表示知识。
- (3) 知识库管理和维护手段。EMYCIN 系统提供的开发知识库的环境,还可以在知识编辑及输入时进行语法、一致性、是否矛盾和包含等检查。

(4) 跟踪和调试功能。EMYCIN 系统还提供了有价值的跟踪和调试功能,试验过程中的状况都被记录并保留下来。

EMYCIN 系统的工作过程分两步。第一步为专家系统建立过程。在该过程中,首先知识工程师输入专家知识,知识获取和知识库构造模块把知识形式化,并对知识进行语法和语义检查,建立知识库。然后知识工程师调试并修改知识库。知识库调试正确后,一个用 EMYCIN 系统构造的专家系统即可交付使用。第二步为咨询过程。在该过程中,咨询用户提出目标假设,推理机制根据知识库中的知识进行推理,最后提出建议,作出决策,并通过解释模块向用户解释推理过程。

EMYCIN 系统已用于建造医学、地质、工程、农业和其他领域的诊断型专家系统。图 7.17 列出了借助于 EMYCIN 系统开发的一些专家系统。

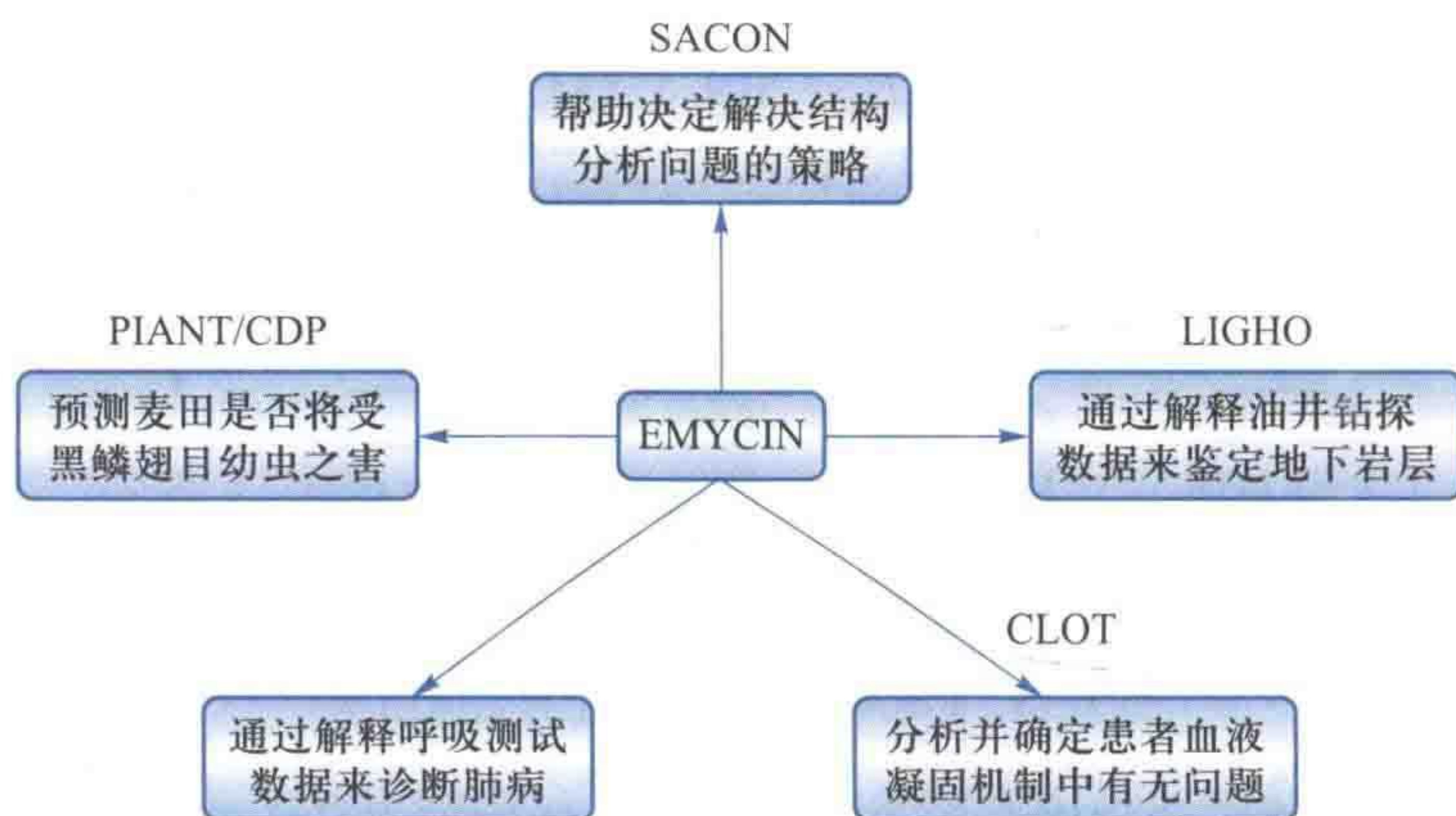


图 7.17 EMYCIN 系统的应用

7.9.5 KAS 骨架系统

KAS 骨架系统是由 PROSPECTOR 系统抽去原有的地质勘探知识而形成的。当把某个领域知识用 KAS 所要求的形式表示出来并输入到知识库中后,它就成为一个可用 PROSPECTOR 的推理机构来求解问题的专家系统。

PROSPECTOR 是美国斯坦福人工智能研究中心(SRI)于 1976 年开始研制的著名的地质勘探专家系统。该系统采用 LISP 语言编写。到 1980 年为止,PROSPECTOR 探测到价值 1 亿美元的矿物淀积层,带来了巨大的经济效益,目前它已成为世界上公认的著名专家系统之一。

PROSPECTOR 系统由推理网络、匹配器、传送器、问答系统、英语分析器、解释系统、网络编译程序和知识获取系统等组成。其系统结构如图 7.18 所示。

PROSPECTOR 系统用语义网络表达知识。知识库由模型文件库和术语文件库组成,推理机具有层次结构,采用“从顶至底”的目标驱动推理控制策略,采用似然推理、逻辑推理、上下文推理相结合的推理方法。

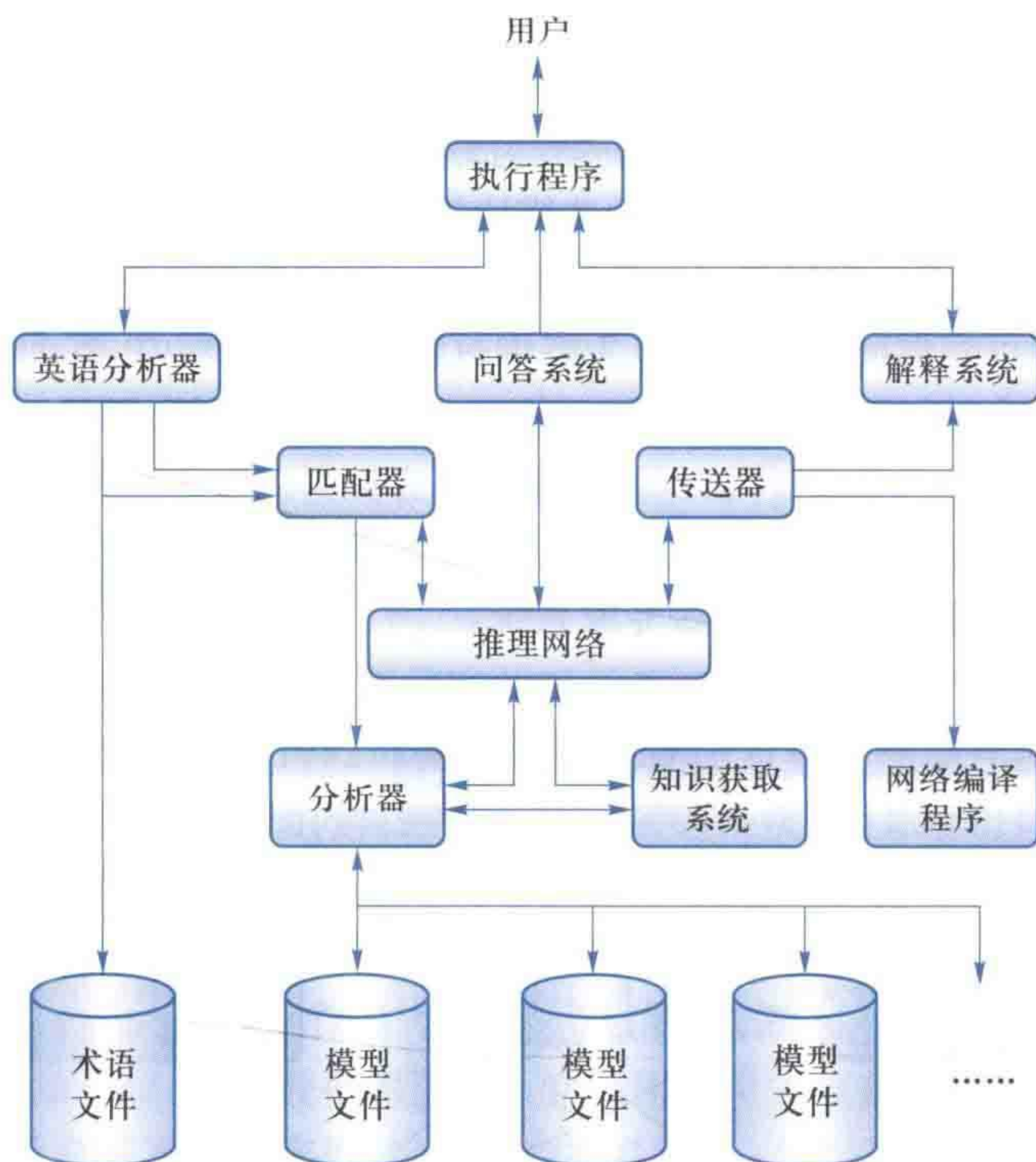


图 7.18 PROSPECTOR 系统的总体结构

KAS 骨架系统采用产生式规则和语义网络相结合的知识表达方法及启发式正反向混合推理控制策略。KAS 骨架系统在推理过程中的推理方向是不断改变的,其推理过程大致为:在 KAS 骨架系统提示下,用户以类似自然语言的形式输入信息,KAS 骨架系统对其进行语法检查并将正确的信息转换为语义网络,然后与表示成语义网络形式的规则的前提条件相匹配,从而形成一组候选目标,并根据用户输入的信息使各候选目标得到不同的评分。接着 KAS 骨架系统从这些候选的目标中选出一个评分最高的候选目标进行反向推理,只要一条规则的前提条件不能被直接证实或被否定,则反向推理就一直进行下去。当有证据表明某个规则的前提条件不可能有超过一定阈值的评分时,就放弃沿这条路线进行的推理,而选择其他的路线。

KAS 骨架系统提供了一些辅助工具,如知识编辑系统、推理解释系统、用户回答系统、英语分析器等,用来开发和测试规则和语义网络。

KAS 骨架系统具有一个功能很强的网络编辑程序和网络匹配程序。网络编辑程序可以用来把用户输入的信息转化为相应的语义网络,并可用来检测语法错误和一致性等。网络匹配程序用于分析任意两个语义网络之间的关系,看其是否具有等价、包含、相交等关系,从而决定这两个语义网络是否匹配,同时它还可以用来检测知识库中的知识是否存在矛盾、冗余等。

KAS 骨架系统适用于开发解释型专家系统。其典型应用如图 7.19 所示。



图 7.19 KAS 骨架系统的应用

以上讨论了两种典型的骨架系统。用骨架系统开发领域专家系统可以大大减少开发的工作量。但也存在一定的问题,主要问题是骨架系统只适用于建造与之类似的专家系统,因其推理机制和控制策略是固定的,所以局限性较大,灵活性差。

7.10 小 结

学习是一个有特定目的的知识获取过程,其内在行为是获取知识、积累经验、发现规律;外部表现是改进性能、适应环境、实现系统的自我完善。

机器学习使计算机能模拟人的学习行为,自动地通过学习获取知识和技能,不断改善性能,实现自我完善。机器学习主要研究学习机理、学习方法、学习系统三方面问题。

监督学习系统中根据“教师”提供的正确响应调整学习系统的参数和结构。典型的监督学习包括线性回归、k 近邻算法、逻辑回归、决策树、随机森林、支持向量机、BP 学习等。

无监督学习系统完全按照环境提供的数据的某些统计规律调节自身的参数或者结构(自组织),以表示出外部输入的某种固有特性。如聚类学习、自组织神经网络学习、自编码器、k 近邻算法等。

弱监督学习中的数据标签允许是不完全的,即训练集中只有一部分数据是有标签的,而其余的数据甚至是绝大部分数据是没有标签的。

半监督学习是一种典型的弱监督学习方法。在半监督学习中,只有少量有标注的数据,还有大量未标注的数据可供使用。

强化学习中外部环境对系统输出结果只给出评价信息(奖励或者惩罚),而不是正确答案,学习系统通过那些受惩的动作改善自身的性能。

线性回归是利用数理统计中回归分析的方法,确定两种或者两种以上变量之间相互依赖的定量关系的一种统计分析方法,属于监督学习。

参数 θ 的最小二乘估计: $\hat{\theta} = (X^T X)^{-1} X^T Y$

k 近邻基本算法:给定一个训练样本集合,计算待分类样本和训练集中所有数据点的距离,将距离从小到大排列然后取前 k 个数据点,将前 k 个数据点中数量最多的类别作为待分类样本的类别。

决策树代表对象属性与对象值之间的一种映射关系,是一类基于树结构进行决策的常见

监督学习方法。

SVM 是一种非概率二元线性分类器,基本思想是创建一个超平面将训练样本划分成两类,并尽可能最大化两个类别之间的间隔。

k 均值聚类算法:对一组数据集,如果要聚成 k 类,则随机的选取 k 个质心(k 个类),然后计算每个样本到 k 个质心的距离,将对应的样本分到最近的类中,然后计算各个类的平均值,重新确定各个类的重心。迭代这个过程,直到重心不再变化。

专家系统是基于知识的系统,用于在某种特定的领域中运用领域专家多年积累的经验和专业知识,求解需要专家才能解决的困难问题。

专家系统具有专家水平的专业知识、能进行有效的推理、具有启发性、灵活性、透明性、交互性等特点。

专家系统本身是一个程序,但它与传统程序不同。一般包括人机接口、推理机、知识库、数据库、知识获取机构和解释机构六部分。

思考题

- 7.1 什么是专家系统? 它有哪些基本特征?
- 7.2 专家系统由哪几部分组成? 各部分的功能和结构如何?
- 7.3 专家系统与传统程序有何不同和相似之处?
- 7.4 什么是学习和机器学习?
- 7.5 试述机器学习系统的基本结构,并说明各部分的作用。
- 7.6 试述机器学习的模式。机器学习有哪些重要问题需要研究?
- 7.7 专家系统设计中要注意哪些问题?
- 7.8 简述专家系统的开发过程。
- 7.9 专家系统的主要类型和主要的应用领域有哪些?
- 7.10 描述专家系统中常用的正向推理和反向推理的算法流程。

读一读

深度学习教父杰弗里·辛顿

2012年的一天,谷歌公司的实习生培训班里,来了一位头发花白的奇怪老人。在这群年轻实习生中,他显得非常特别,年龄好像很大,但好像什么都不懂。培训班上的年轻人都在想:那个老人是谁?直到有一天,有人在餐厅午餐时热情地和他打招呼“辛顿教授,我选修了你的课。你在这里做什么?”实习生们大为惊讶,这才知道原来天天和他们坐在一起的这位老实习生,竟然是“深度学习教父”、加拿大多伦多大学计算机系教授杰弗里·辛顿(Geoffrey Hinton)。

辛顿在谷歌公司参加过实习生培训,是一段颇具戏剧性的故事。原来,谷歌大脑(Google Brain)负责人杰夫·迪恩邀请辛顿到谷歌短期工作三个月。但谷歌有一些古怪的规定,进入谷歌公司的人必须经过实习生培训。这样,六十多岁的辛顿就出现在年轻实习生队伍里。

杰弗里·辛顿1947年出生于英国。他的家族有很深的学术渊源,他的曾祖父的岳父就是大名鼎鼎的乔治·布尔(George Boole),是符号逻辑领域的先驱,这是促进计算机诞生与发展的重要基础研究之一,“布尔代数”就是以他的姓命名。

辛顿很早就开始思考人的大脑。16岁时一位同学给他介绍关于记忆的理论:大脑对于事物和概念的记忆,不是存储在某个单一的地点,而是像全息照片一样,分布式存储于一个巨大神经元网络里。全息照片、分布式、神经元网络,这些概念深深启发了他,让他对神经网络产生了浓厚兴趣。很自然地,他进入剑桥大学心理学专业并于1970年本科毕业。1978年,他取得爱丁堡大学AI专业博士学位。爱丁堡大学的AI实验室,是世界上最早建立的四个AI研究基地之一。

辛顿博士毕业后,在卡内基·梅隆大学等多所欧美著名高校游学和研究,最后到加拿大多伦多大学任教,始终聚焦人工神经网络的研究。由于人工神经网络的高度复杂性,这一研究方向一度被认为是学术死路。在AI第二次进入低谷时,很多同行转到别的研究领域,但是辛顿等人仍然坚持研究。尽管辛顿在1998年被选为英国皇家学会院士,但很长一段时间里,多伦多大学计算机系私下流行着一句对新生的警告:不要去辛顿的实验室。据说辛顿有一种激励自己的特殊方法,每周发泄般大吼一次:我发现大脑是怎样工作的啦!这样的习惯,几十年一直保持下来。

2004年,辛顿终于从加拿大高等研究所(CIFAR)获得了每年50万美元的经费支持,开展“神经计算和适应感知”项目研究,将一批一流的计算机、生物学、电子工程、神经科学、物理学和心理学专家团结在一起,共同探索神经网络模拟人脑智能的新方法。

2006年7月28日,辛顿和学生题为《用神经网络降低数据维数》的论文在美国权威杂志*Science*上发表,被认为是深度学习(deep learning)领域开创性论文,很快成为AI和神经网络最热门的研究方向,深度学习甚至成为神经网络的代名词。

读一读

专家系统之父费根鲍姆

爱德华·费根鲍姆(Edward Albert Feigenbaum, 1936—), 在12岁时自学钢琴, 高中时自学微积分。1952年进入卡内基·梅隆大学电气工程系学习。本科期间, 他选修了学校里所有有关数学和物理的课程, 用了不足4年便毕业。在那里, 他学了司马贺教授的“社会科学的数学模型”课程。他在参观纽约理工大学布鲁克林分校时首次使用电脑, 并在一小时内便写了一个用牛顿法开方的程序。本科毕业后, 他选择司马贺教授当自己的博士导师。在司马贺的指导下, 他研究了一个模拟人在刺激反应环境中记忆单词时的反应的程序, 叫作EPAM(基本识别和存储设备系统的缩写), 并以此为题完成了他的博士论文。

费根鲍姆最早提出了“知识工程”(knowledge engineering)的概念, 并使知识工程成为人工智能领域中取得实际成果最丰富、影响最大的一个分支, 成为人工智能研究的一个历史性突破。他于1965年和遗传学系主任、诺贝尔奖得主莱德伯格(Joshua Lederberg)等人合作, 开发出了世界上第一个专家系统DENDRAL, 不仅验证了费根鲍姆关于知识工程的理论的正确性, 还为专家系统软件的发展和应用开辟了道路, 应用遍及各个领域。费根鲍姆领导的研究小组后来又为医学、工程和国防等部门研制成功一系列实用的专家系统, 其中尤以医学专家系统方面的成果最为突出。例如, 用于帮助医生诊断传染病和提供治疗建议的著名专家系统MYCIN等。

费根鲍姆有句名言:“知识中蕴藏着力量”。这句话和培根的名言“知识就是力量”意义相近, 但似乎更确切些: 知识只有被人所发掘和掌握时, 才能生成力量。

爱德华·费根鲍姆获1994年度图灵奖。

第 8 章 人工神经网络与深度学习

2024 年 10 月 8 日,瑞典皇家科学院宣布 2024 年诺贝尔物理学奖授予美国普林斯顿大学约翰·霍普菲尔德和加拿大多伦多大学杰弗里·辛顿,表彰他们从 20 世纪 80 年代起就开展了与物理学相关的人工神经网络的重要工作,使用人工神经网络进行机器学习的基础性发现和发明。

人工神经网络(artificial neural network, ANN)是一个用大量简单处理单元经广泛连接而组成的人工网络,是对人脑或生物神经网络若干基本特性的抽象和模拟。神经网络理论为机器学习等许多问题的研究提供了一条新的思路,目前已经在模式识别、机器视觉、联想记忆、自动控制、信号处理、软测量、决策分析、智能计算、组合优化问题求解、数据挖掘等方面获得成功应用。神经网络的研究已经获得许多成果,提出了大量的神经网络模型和算法。

本章着重介绍最基本、最典型、应用最广泛的 BP 神经网络和 Hopfield 神经网络及其在模式识别、联想记忆、组合优化问题求解等方面的应用,并为后面章节介绍深度学习奠定基础。

8.1 神经元与神经网络

8.1.1 生物神经元结构

现代人的大脑内约有 10^{11} 个神经元,每个神经元与其他神经元之间约有 1 000 个连接,这样,大脑内约有 10^{14} 个连接。如果将一个大脑中所有神经细胞的轴突和树突依次连接起来,拉成一条直线,则可以从地球连到月亮,再从月亮连到地球。人的智能行为就是由如此高度复杂的组织产生的。浩瀚的宇宙中,也许只有包含数千亿颗星球的银河系的复杂性能够与大脑相比。早在 1904 年,生物学家就已经发现了神经元的结构。

从生物控制与信息处理的角度看,生物神经元结构如图 8.1 所示。

神经元的主体部分为细胞体。细胞体(soma)由细胞核、细胞质、细胞膜等组成。每个细胞体都有一个细胞核(cell nucleus),埋藏在细胞体之中,进行呼吸和新陈代谢等许多生化过程。神经元还包括树突和一条长的轴突,由细胞体向外伸出的最长的一条分支称为轴突即神经纤维。轴突末端部分有许多分支,叫轴突末梢。典型的轴突长 1 cm,是细胞体直径的 100 倍。一

个神经元通过轴突末梢与 10 到 10 万个其他神经元相连接。轴突是用来传递和输出信息的,其端部的许多轴突末梢为信号输出端子,将神经冲动传给其他神经元。由细胞体向外伸出的其他许多较短的分支称为树突。树突相当于细胞的输入端,树突的全长各点都能接收其他神经元的冲动。神经冲动只能由前一级神经元的轴突末梢传向下一级神经元的树突或细胞体,不能作反方向的传递。

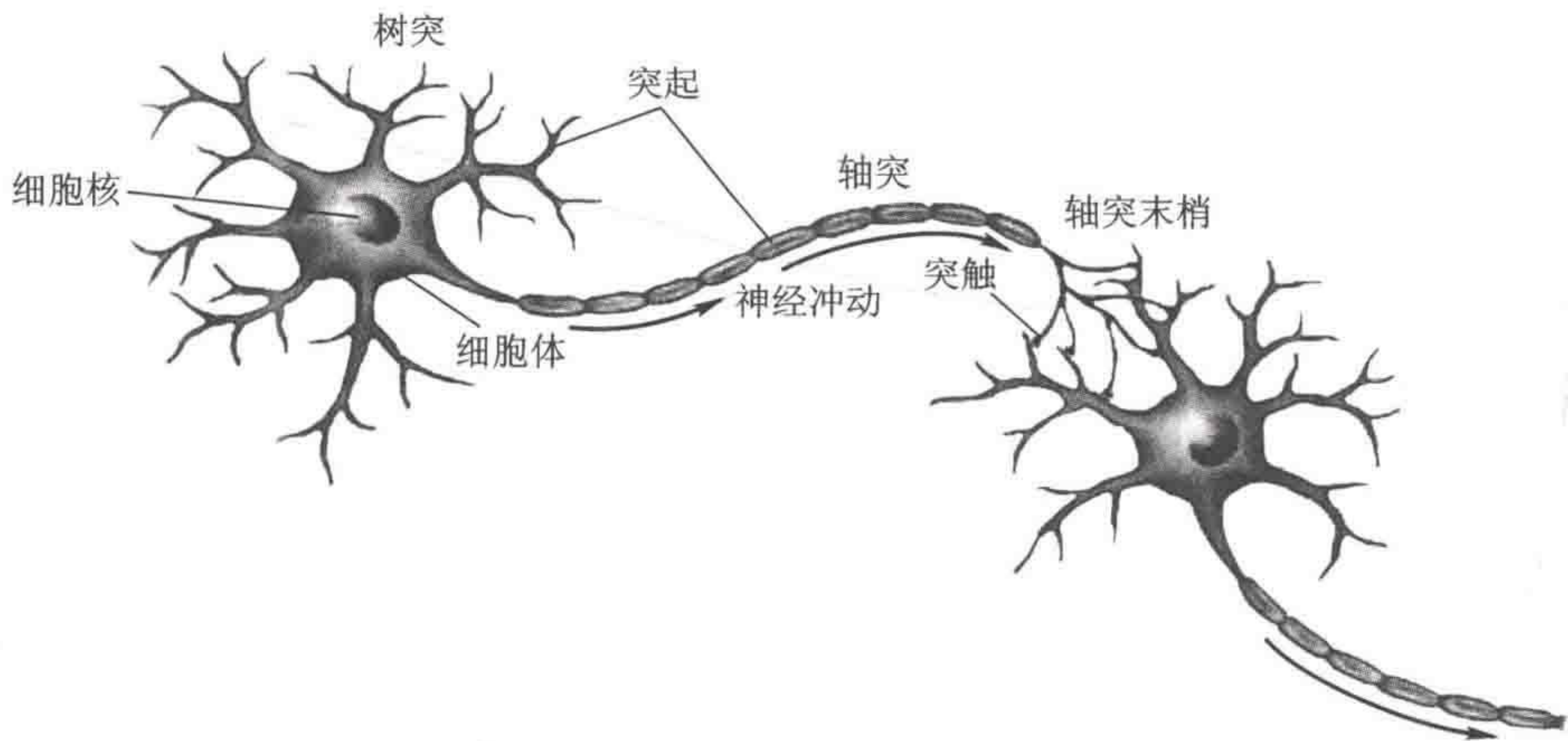


图 8.1 神经元构造

神经元具有两种常规工作状态:兴奋与抑制,即满足“0-1”律。当传入的神经冲动使细胞膜电位升高超过阈值时,细胞进入兴奋状态,产生神经冲动并由轴突输出;当传入的冲动使膜电位降低于阈值时,细胞进入抑制状态,没有神经冲动输出。

8.1.2 神经元数学模型

早在 1943 年,美国神经生理学家麦卡洛克(W. S. McCulloch)和数学家皮茨(W. Pitts)就提出了神经元的数学模型(M-P 神经元模型),从此开创了神经科学理论研究的时代,其影响深远,是当前兴起的深度学习大模型的源头。从 20 世纪 40 年代开始,根据神经元的结构和功能不同,先后提出的神经元模型有几百种之多。下面介绍神经元的一种所谓标准、统一的数学模型,它由三部分组成,即加权求和、线性动态系统和非线性函数映射,如图 8.2 所示。

图 8.2 中, $y_i(t)$ 为第 i 个神经元的输出, θ_i 为第 i 个神经元的阈值($i=1,2,\dots,N$); $u_k(t)$ ($k=1,2,\dots,M$)为外部输入; a_{ij}, b_{ik} 为权值。

加权求和

$$v_i(t) = \sum_{j=1}^N a_{ij} y_j(t) + \sum_{k=1}^M b_{ik} u_k(t) - \theta_i \quad (8.1)$$

将式(8.1)记为矩阵形式

$$\mathbf{V}(t) = \mathbf{A}\mathbf{Y}(t) + \mathbf{B}\mathbf{U}(t) - \boldsymbol{\theta} \quad (8.2)$$

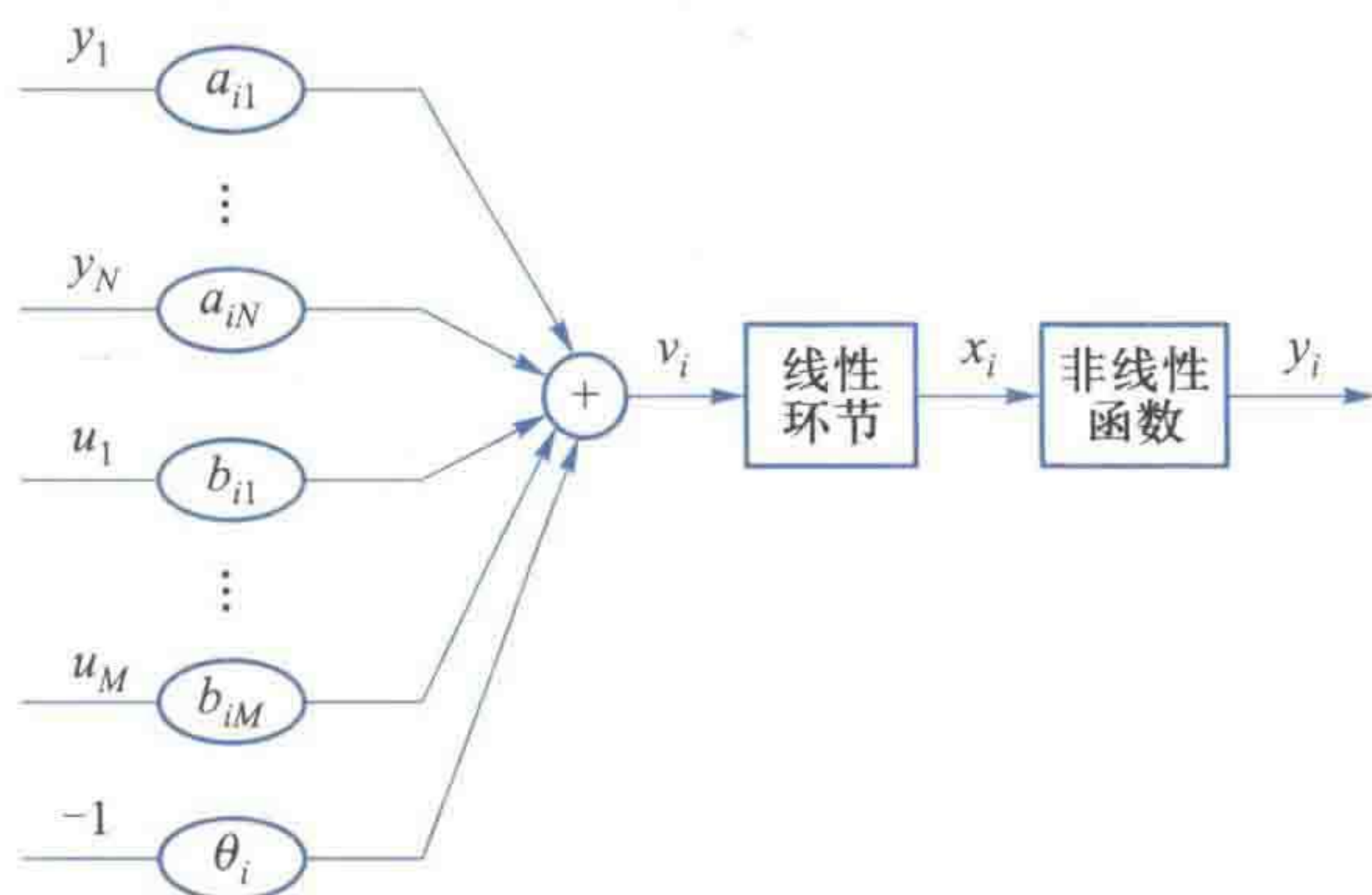


图 8.2 神经元数学模型

其中, $\mathbf{A}=[a_{ij}]_{N \times N}$, $\mathbf{B}=[b_{ik}]_{N \times M}$, $\mathbf{V}=[v_1, \dots, v_N]^T$, $\mathbf{Y}=[y_1, \dots, y_N]^T$, $\mathbf{U}=[u_1, \dots, u_M]^T$, $\boldsymbol{\theta}=[\theta_1, \dots, \theta_N]^T$ 。

线性环节的传递函数描述为

$$X_i(s) = H(s) V_i(s) \quad (8.3)$$

其中, $H(s)$ 通常取为 1 、 $\frac{1}{s}$ 、 $\frac{1}{Ts+1}$ 、 e^{-Ts} 及其组合等。

神经元输出 $y_i(t)$ 与输入 $x_i(t)$ 之间的非线性函数关系最常用的有以下两种。

(1) 阶跃函数

$$f(x_i) = \begin{cases} 1 & x_i \geq 0 \\ 0 & x_i < 0 \end{cases} \quad (8.4)$$

或

$$f(x_i) = \begin{cases} 1 & x_i \geq 0 \\ -1 & x_i < 0 \end{cases} \quad (8.5)$$

(2) S 型函数

它具有平滑和渐近性, 并保持单调性, 是最常用的非线性函数。最常用的 S 型函数为 Sigmoid 函数

$$f(x_i) = \frac{1}{1 + e^{-\alpha x_i}} \quad (8.6a)$$

其中, α 可以控制其斜率。

对于需要神经元输出在 $[-1, 1]$ 区间时, S 型函数可以选为双曲线正切函数 (hyperbolic tangent function)

$$f(x_i) = \frac{1 - e^{-\alpha x_i}}{1 + e^{-\alpha x_i}} \quad (8.6b)$$

Sigmoid 函数的缺点是在输入的绝对值大于某个阈值后, 过快进入饱和状态 (即函数值趋

于1或者-1,而不再有显著的变化),出现梯度消失情况,即梯度会趋于0,在实际模型训练中会导致模型收敛缓慢,性能不够理想。因此,在一些现代网络结构中,逐渐为 ReLU 等类模型的激活函数取代。

(3) ReLU 函数

ReLU(rectified linear units)是近年来深度学习研究中广泛使用的一个激活函数。ReLU 不是一个光滑曲线,而是一个很简单分段线性函数,即

$$f(x_i) = \begin{cases} 0 & x_i < 0 \\ x_i & x_i \geq 0 \end{cases} \quad (8.7)$$

ReLU 函数尽管形式简单,但在实际应用中并没有饱和问题,运算速度快,收敛效果好,在卷积神经网络等深度神经网络中效果很好。

8.1.3 神经网络的结构与工作方式

1. 神经网络的结构

神经网络是由众多简单的神经元连接而成的一个网络。尽管每个神经元结构、功能都不复杂,但神经网络的行为并不是各单元行为的简单相加,网络的整体动态行为则是极为复杂的,可以组成高度非线性动力学系统,从而描述很多复杂的物理系统,表现出一般复杂非线性系统的特性(如不可预测性、不可逆性、多吸引子、可能出现混沌现象等)和作为神经网络系统的各种性质。神经网络具有大规模并行处理能力和自适应、自组织、自学习能力以及分布式存储等特点,在许多领域得到了成功的应用,展现了非常广阔的应用前景。

众多的神经元的轴突和其他神经元或者自身的树突相连接,构成复杂的神经网络。根据神经网络中神经元的连接方式可以划分为不同类型的结构。目前人工神经网络主要有前馈型和反馈型两大类。

(1) 前馈型

前馈型神经网络中,各神经元接受前一层的输入,并输出给下一层,没有反馈。前馈网络可分为不同的层,第*i*层只与第*i*-1层输出相连,输入与输出的神经元与外界相连。后面着重介绍的 BP 神经网络就是一种前馈型神经网络。

(2) 反馈型

在反馈型神经网络中,存在一些神经元的输出经过若干个神经元后,再反馈到这些神经元的输入端。最典型的反馈型神经网络是 Hopfield 神经网络。它是全互联神经网络,即每个神经元和其他神经元都相连。

2. 神经网络的工作方式

当满足兴奋条件时,神经网络中的神经元就会改变为兴奋状态,当不满足兴奋条件时,它就会改变为抑制状态。如果神经网络中各个神经元同时改变状态,则称为同步(synchronous)工作方式;如果神经网络中神经元是一个一个地改变状态,即当某个神经元改变状态时,其他神经元保持状态不变,称为异步(asynchronism)工作方式。神经网络在不同的工作方式下的性

能有些差异。

8.1.4 神经网络的学习

神经网络方法是一种知识表示方法和推理方法。神经网络知识表示方法与谓词、产生式、框架、语义网络等完全不同。谓词、产生式、框架、语义网络等方法知识的显式表示。例如,在产生式系统中知识独立地表示为一条规则。神经网络知识表示是一种隐式的表示方法。在这里,将某一问题的若干知识通过学习表示在同一网络中。

神经网络的学习是指调整神经网络的连接权值或者结构,使输入输出具有需要的特性。

1944 年赫布(Hebb)提出了改变神经元连接强度的 Hebb 学习规则。由于 Hebb 学习规则的基本思想很容易被接受,得到了较为广泛的应用,至今仍在各种神经网络模型的研究中起着重要的作用。但近年来神经科学的许多发现都表明,Hebb 学习规则并没有准确反映神经元在学习过程中突触变化的基本规律。

Hebb 学习规则:当某一突触两端的神经元同时处于兴奋状态,那么该连接的权值应该增强。用数学方式描述调整权值 w_{ij} 的方法为

$$w_{ij}(k+1) = w_{ij}(k) + \alpha y_i(k) y_j(k) \quad (\alpha > 0) \quad (8.8)$$

其中, $w_{ij}(k+1)$ 为权值的下一步值; $w_{ij}(k)$ 为权值的当前值。

8.2 BP 神经网络及其学习算法

8.2.1 BP 神经网络的结构

BP 神经网络(back-propagation neural network)就是多层前向网络,其结构如图 8.3 所示。

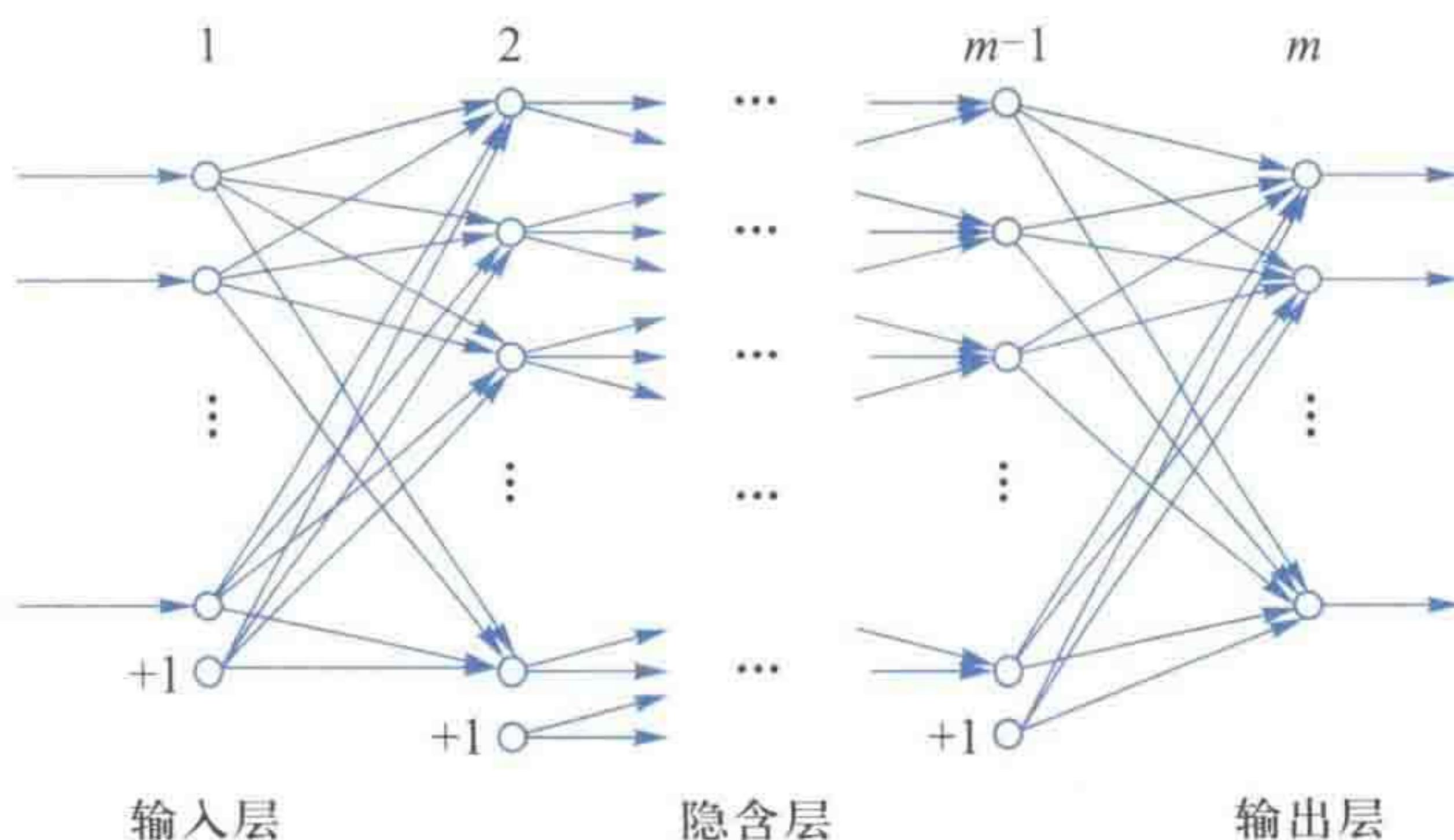


图 8.3 BP 神经网络结构

设 BP 神经网络具有 m 层。第一层称为输入层,最后一层称为输出层,中间各层称为隐含层。标上“+1”的圆圈称为偏置结点。没有其他单元连向偏置单元(bias units)。偏置单元没有

输入,它的输出总是+1。输入层起缓冲存储器的作用,把数据源加到网络上,因此,输入层的神经元的输入与输出关系一般是线性函数。隐含层中各个神经元的输入与输出关系一般为非线性函数。隐含层 k 与输出层中各个神经元的非线性输入与输出关系记为 $f_k(k=2, \dots, m)$ 。由第 $k-1$ 层的第 j 个神经元到第 k 层的第 i 个神经元的连接权值为 w_{ij}^k 。并设第 k 层中第 i 个神经元输入的总和为 u_i^k ,输出为 y_i^k ,则各变量之间的关系为

$$\begin{aligned} y_i^k &= f_k(u_i^k) \\ u_i^k &= \sum_j w_{ij}^{k-1} y_j^{k-1} \quad (k=2, \dots, m) \end{aligned} \quad (8.9)$$

当 BP 神经网络输入数据 $\mathbf{X}=[x_1, x_2, \dots, x_{p_1}]^T$ (设输入层有 p_1 个神经元),从输入层依次经过各隐含层结点,可得到输出数据 $\mathbf{Y}=[y_1^m, y_2^m, \dots, y_{p_m}^m]^T$ (设输出层有 p_m 个神经元)。因此,可以把 BP 神经网络看成是一个从输入到输出的非线性映射。

给定 N 组输入输出样本为 $\{X_{si}, Y_{si}\}, i=1, 2, \dots, N$ 。如何调整 BP 神经网络的权值,使 BP 神经网络输入为样本 X_{si} 时,神经网络的输出为样本 Y_{si} 。这就是 BP 神经网络的学习问题。可见, BP 学习算法是一种有教师学习算法。

要解决 BP 神经网络的学习问题,关键是解决以下两个问题。

第一,是否存在一个 BP 神经网络能够逼近给定的样本或者函数。

下述定理可以回答这个问题。

BP 定理:给定任意 $\varepsilon > 0$,对于任意的连续函数 f ,存在一个三层前向神经网络,可以在任意 ε 平方误差精度内逼近 f 。

BP 定理虽然说明只要用三层 BP 网络就可以逼近一个任意连续函数,但可能需要大量的隐含层神经元个数。为了减少隐含层神经元个数,可以采用多层 BP 神经网络结构。对多层 BP 神经网络,如何合理地选取 BP 网络的隐含层数及隐含层的神经元个数,目前尚无有效的理论和方法。

第二,如何调整 BP 神经网络的权值,使 BP 神经网络的输入与输出之间的关系与给定的样本相同。BP 学习算法给出了具体的调整算法。

8.2.2 BP 学习算法

BP 学习算法最早是由 Werbos 在 1974 年提出的。辛顿(Hinton)、鲁梅尔哈特(Rumelhart)等于 1985 年发展了 BP 学习算法,实现了明斯基(Minsky)多层感知机的设想。

BP 学习算法是通过反向学习过程使误差最小,因此选择目标函数为

$$\min J = \frac{1}{2} \sum_{j=1}^{p_m} (y_j^m - y_{sj})^2 \quad (8.10)$$

即选择神经网络权值使期望输出 y_{sj} 与神经网络实际输出 y_j^m 之差的平方和最小。

这种学习算法实际上是求目标函数 J 的极小值,约束条件是式(8.9),可以利用非线性规划中的“最快下降法”,使权值沿目标函数的负梯度方向改变,因此,神经网络权值的修正量为

$$\Delta w_{ij}^{k-1} = -\varepsilon \frac{\partial J}{\partial w_{ij}^{k-1}} \quad (\varepsilon > 0) \quad (8.11)$$

其中, ε 为学习步长, 一般小于 0.5。

下面推导 BP 学习算法。求解公式(8.11)的困难是求偏导数 $\frac{\partial J}{\partial w_{ij}^{k-1}}$ 。由复合函数的链式求导法则, 得

$$\frac{\partial J}{\partial w_{ij}^{k-1}} = \frac{\partial J}{\partial u_i^k} \frac{\partial u_i^k}{\partial w_{ij}^{k-1}} = \frac{\partial J}{\partial u_i^k} \frac{\partial}{\partial w_{ij}^{k-1}} \left(\sum_j w_{ij}^{k-1} y_j^{k-1} \right) = \frac{\partial J}{\partial u_i^k} y_j^{k-1}$$

记

$$d_i^k = \frac{\partial J}{\partial u_i^k} \quad (k = 2, \dots, m)$$

则

$$\Delta w_{ij}^{k-1} = -\varepsilon d_i^k y_j^{k-1} \quad (k = 2, \dots, m) \quad (8.12)$$

下面推导计算 d_i^k 的公式

$$d_i^k = \frac{\partial J}{\partial u_i^k} = \frac{\partial J}{\partial y_i^k} \frac{\partial y_i^k}{\partial u_i^k} = \frac{\partial J}{\partial y_i^k} f'_k(u_i^k)$$

下面分两种情况求 $\frac{\partial J}{\partial y_i^k}$ 。

① 对输出层(第 m 层)的神经元, 即 $k=m, y_i^k=y_i^m$, 由误差定义式得

$$\frac{\partial J}{\partial y_i^k} = \frac{\partial J}{\partial y_i^m} = y_i^m - y_{si} \quad (8.13)$$

则

$$d_i^m = (y_i^m - y_{si}) f'_m(u_i^m) \quad (8.14)$$

② 若 i 为隐单元层 k , 则有

$$\frac{\partial J}{\partial y_i^k} = \sum_l \frac{\partial J}{\partial u_l^{k+1}} \frac{\partial u_l^{k+1}}{\partial y_i^k} = \sum_l d_l^{k+1} w_{li}^k$$

则

$$d_i^k = f'_k(u_i^k) \sum_l d_l^{k+1} w_{li}^k \quad (8.15)$$

综上所述, BP 学习算法可以归纳为

$$\Delta w_{ij}^{k-1} = -\varepsilon d_i^k y_j^{k-1} \quad (8.16a)$$

$$d_i^m = (y_i^m - y_{si}) f'_m(u_i^m) \quad (8.17a)$$

$$d_i^k = f'_k(u_i^k) \sum_l d_l^{k+1} w_{li}^k \quad (k = m-1, \dots, 2) \quad (8.18a)$$

若取 $f_k(\cdot)$ 为 S 型函数, 即

$$y_i^k = f_k(u_i^k) = \frac{1}{1 + e^{-u_i^k}}$$

则

$$\frac{\partial y_i^k}{\partial u_i^k} = f'_k(u_i^k) = \frac{e^{-u_i^k}}{(1 + e^{-u_i^k})^2} = y_i^k(1 - y_i^k)$$

则 BP 学习算法可以归纳为

$$\Delta w_{ij}^{k-1} = -\varepsilon d_i^k y_j^{k-1} \quad (8.16b)$$

$$d_i^m = y_i^m(1 - y_i^m)(y_i^m - y_{si}) \quad (8.17b)$$

$$d_i^k = y_i^k(1 - y_i^k) \sum_l d_l^{k+1} w_{li}^k \quad (k = m - 1, \dots, 2) \quad (8.18b)$$

从以上公式可以看出, 求第 k 层的误差信号 d_i^k , 需要上一层的 d_l^{k+1} 。因此, 误差函数的求取是一个始于输出层的反向传播的递归过程, 所以称为反向传播 (back propagation, BP) 学习算法。通过多个样本的学习, 修改权值, 不断减少偏差, 最后达到满意的结果。

8.2.3 BP 学习算法的实现

BP 学习算法的程序框图如图 8.4 所示。

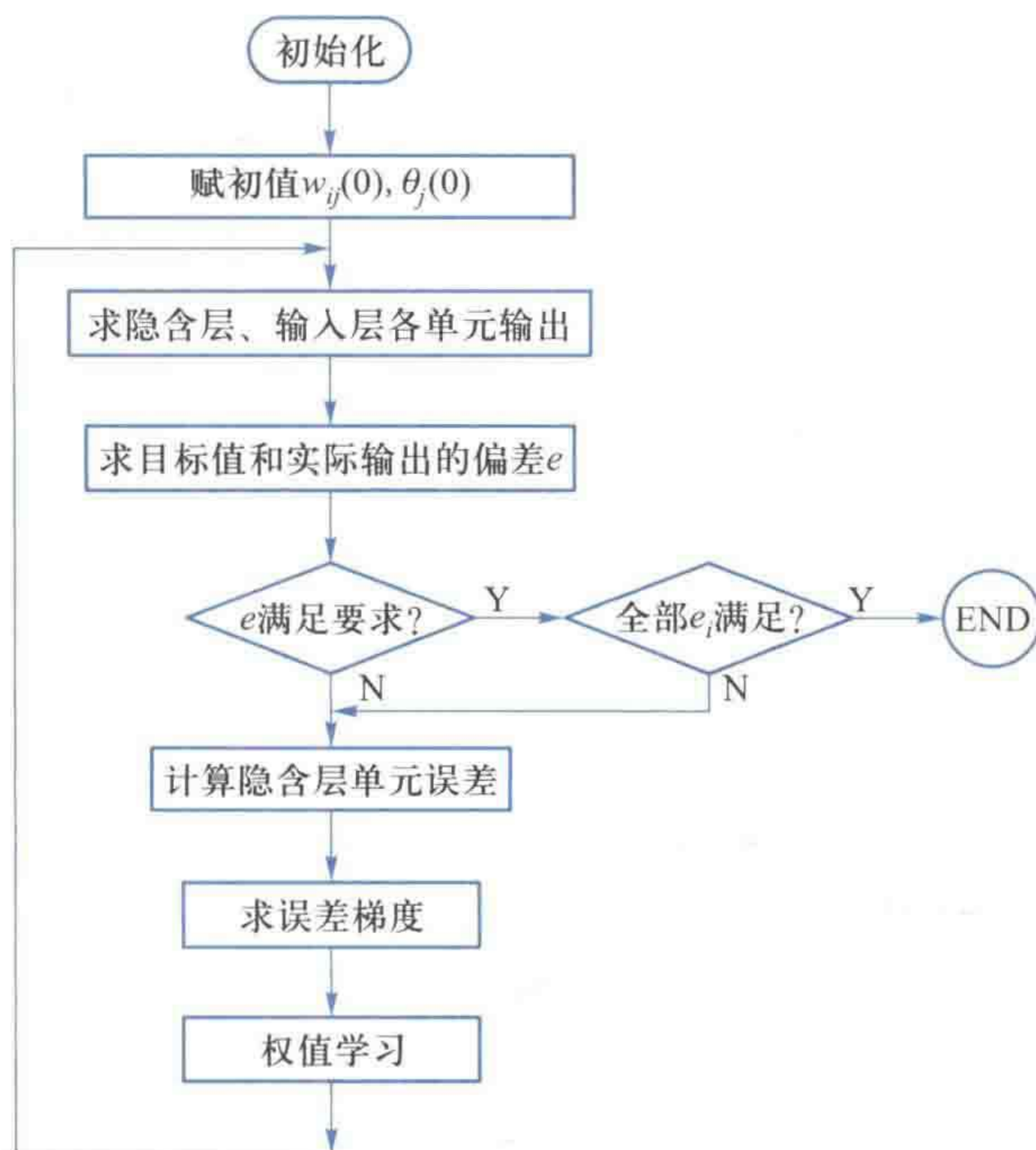


图 8.4 BP 算法程序框图

在 BP 算法实现时,还要注意下列问题。

① 训练数据预处理。预处理过程包含一系列的线性的特征比例变换,将所有的特征变换到 $[0,1]$ 或者 $[-1,1]$ 上,使得在每个训练集上,每个特征的均值为0,并且具有相同的方差。预处理过程也称为尺度变换(scaling),或者规格化(normalization)。

② 后处理过程。当应用神经网络进行分类操作时,通常将输出值编码成所谓的名义变量,具体的值对应类别标号。在一个两类分类问题中,可以仅使用一个输出,将它编码成一个二值变量(例如+1、-1)。当具有更多的类别时,应当为每个类别分配一个代表类别决策的名义输出值。例如,对于一个三类分类问题,可以设置三个名义输出,每个名义输出取值为 $\{+1,-1\}$,对应的各个类别决策为 $\{+1,-1,-1\}$, $\{-1,+1,-1\}$, $\{-1,-1,+1\}$ 。利用阈值可以将神经网络的输出值变换成为合适的名义输出值。

③ 初始权值的影响及设置。和所有梯度下降算法一样,初始权值对 BP 神经网络的最终解有很大的影响。虽然全部设置为0显得比较自然,但从式(8.18)可以看出这将导致很不理想的结果。如果输出层的权值全部为0,则反向传播误差也将为0,输出层前面的权值将不会改变。因此,一般以一个均值为0的随机分布设置 BP 神经网络的初始权值。

8.3 BP 神经网络在模式识别中的应用

模式识别主要研究用计算机模拟生物、人的感知,对模式信息(如图像、文字、语音等)进行识别和分类。传统人工智能的研究部分地显示了人脑的归纳、推理等智能。但是,对于人类底层的智能,如视觉、听觉、触觉等方面,现代计算机系统的信息处理能力还不如一个幼儿园的孩子。

神经网络模型模拟了人脑神经系统的特点:处理单元的广泛连接,并行分布式信息储存、处理,自适应学习能力等。神经元网络的研究为模式识别开辟了新的研究途径。与传统的模式识别方法相比,神经网络方法具有较强的容错能力、自适应学习能力、并行信息处理能力。

例 8.1 设计一个三层 BP 网络对数字0至9进行分类。训练数据如图 8.5 所示,测试数据如图 8.6 所示。

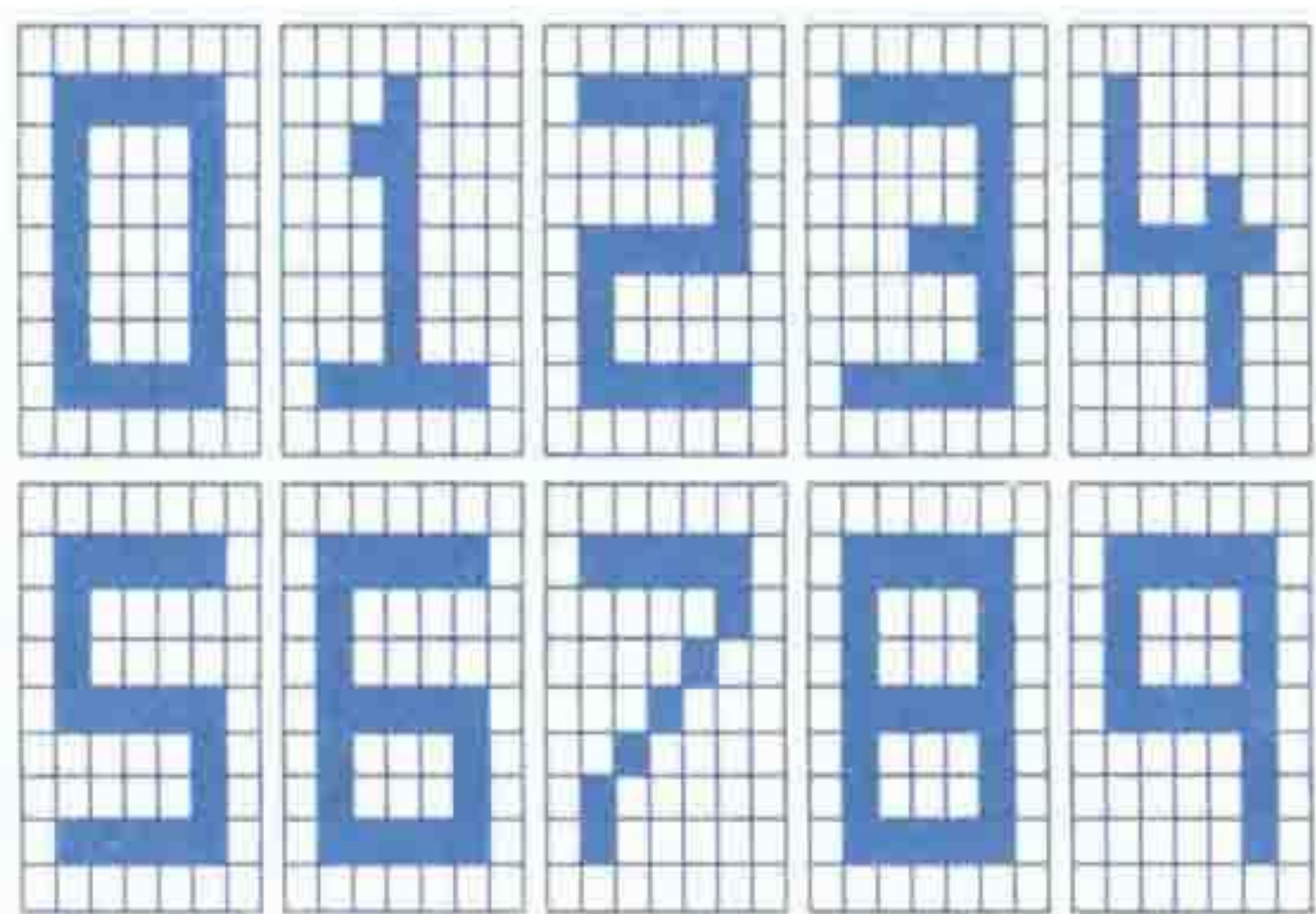


图 8.5 数字分类训练数据

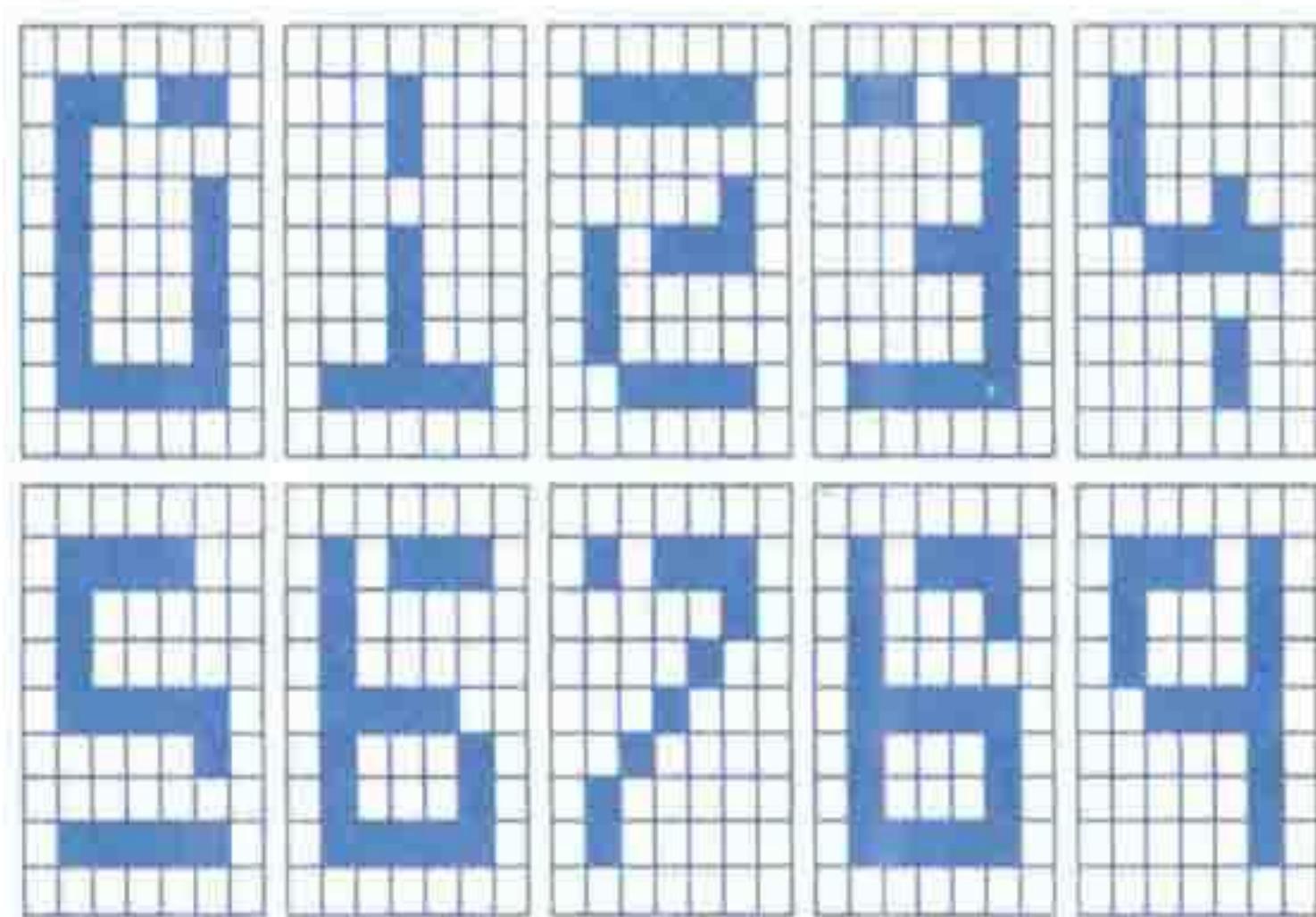


图 8.6 数字分类测试数据

解 该分类问题有 10 类,且每个目标向量应该是这 10 个向量中的一个。目标值由数字 1 至 9 的 9 个向量中的一个表示,0 是由所有结点的输出全为 0 来表示。每个数字用 9×7 的网

格表示,灰色像素代表0,黑色像素代表1。将网格表示为0或者1的长位串。位映射由左上角开始向下直到网格的整个一行,然后重复其他列。

例如,数字“1”的网格的数字串为{0,0,0,0,0,0,0,0,0,0;0,0,0,0,0,0,0,1,0;0,0,1,0,0,0,0,1,0;0,1,1,1,1,1,1,0;0,0,0,0,0,0,0,1,0;0,0,0,0,0,0,0,1,0;0,0,0,0,0,0,0,0,0}。

选择网络结构为63-6-9。9×7个输入结点,对应上述网格的映射。9个输出结点对应10种分类。使用的学习步长为0.3。训练1000个周期,如果输出结点的值大于0.9,则取为1,如果输出结点的值小于0.1,则取为0。

当训练成功后,对如图8.6所示测试数据进行测试。图8.6所示测试数据都有一个或者多个位丢失。测试结果表明:除了8以外,所有被测的数字都能够被正确地识别。对于数字8,第8个结点的输出值为0.41,而第6个结点的输出值为0.53,表明第8个测试数据是模糊的,可能是数字6,也可能是数字8,但也不完全确信是6或8两者之一。实际上,人识别这个数字时也会发生这种错误。

8.4 Hopfield 神经网络及其改进

从20世纪60年代初到80年代初,神经网络的研究处于“冰河期”,到了20世纪80年代中期,美国加州理工学院生物物理学家霍普菲尔德(J. J. Hopfield)在神经网络建模及应用方面的开创性成果,在世界范围内重新掀起了神经网络的研究热潮。

1982年和1984年,霍普菲尔德先后提出离散型Hopfield神经网络和连续型Hopfield神经网络,引入“计算能量函数”的概念,给出了网络稳定性判据,尤其是给出了Hopfield神经网络的电子电路实现,为神经计算机的研究奠定了基础,同时开拓了神经网络用于联想记忆和优化计算的新途径,从而有力地推动了神经网络的研究。这两种模型是目前最重要的神经优化计算模型。

8.4.1 离散型 Hopfield 神经网络

Hopfield神经网络(HNN)是全互联反馈神经网络,它的每一个神经元都和其他神经元相连接。

具有 N 个神经元的离散型Hopfield神经网络HNN,可由一个 $N \times N$ 阶矩阵 $\mathbf{w} = [w_{ij}]_{N \times N}$ 和一个 N 维列向量 $\boldsymbol{\theta} = [\theta_1, \theta_2, \dots, \theta_N]^T$ 所唯一确定,记为 $HNN = (\mathbf{w}, \boldsymbol{\theta})$,其中, w_{ij} 为从第 j 个神经元的输出到第 i 个神经元的输入之间的连接权值,表示神经元 i 与 j 的连接强度,且 $w_{ji} = w_{ij}$, $w_{ii} = 0$; θ_i 表示神经元 i 的阈值。若用 $v_i(k)$ 表示 k 时刻神经元所处的状态,那么神经元 i 的状态随时间变化的规律(又称演化律)为二值硬限器(binary hardlimiters):

$$v_i(k+1) = \begin{cases} 1 & u_i(k) \geq 0 \\ 0 & u_i(k) < 0 \end{cases} \quad (8.19)$$

或者为双极硬限器:

$$v_i(k+1) = \begin{cases} 1 & u_i(k) \geq 0 \\ -1 & u_i(k) < 0 \end{cases} \quad (8.20)$$

其中

$$u_i(k) = \sum_{\substack{j=1 \\ j \neq i}}^n w_{ij} v_j(k) - \theta_i \quad (1 \leq i \leq N) \quad (8.21)$$

Hopfield 神经网络可以是同步工作方式,也可以是异步工作方式。即神经元更新既可以同步(并行)进行,也可以是异步(串行)进行。在同步进行时,神经网络中所有神经元的更新同时进行。在异步进行时,在同一时刻只有一个神经元更新,而且这个神经元在网络中每个神经元都更新之前不会再次更新。在异步更新时,神经元的更新顺序可以是随机的。

Hopfield 神经网络中的神经元相互作用,不断演化。如果神经网络在演化过程中,从某一时刻开始,神经网络中的所有神经元的状态不再改变,则称该神经网络是稳定的。

Hopfield 神经网络是高维非线性动力学系统,可能有若干个稳定状态。从任一初始状态开始运动,总可以达到某个稳定状态。这些稳定状态可以通过改变各个神经元之间的连接权值得到。

稳定性是 Hopfield 神经网络的最重要的特性。下面分析 Hopfield 神经网络的稳定性。

Hopfield 神经网络是一个多输入多输出带阈值的二态非线性动力学系统,所以,类似于李雅普诺夫稳定性分析方法,在 Hopfield 神经网络中,也可以构造一种 Lyapunov 函数,在满足一定的参数条件下,该函数值在网络运行过程中不断降低,最后趋于稳定的平衡状态。Hopfield 引入这种能量函数作为网络计算求解的工具,因此,常常称它为计算能量函数。

离散型 Hopfield 神经网络的计算能量函数定义为

$$E = -\frac{1}{2} \sum_{i=1}^N \sum_{\substack{j=1 \\ j \neq i}}^N w_{ij} v_i v_j + \sum_{i=1}^N \theta_i v_i \quad (8.22)$$

其中, v_i, v_j 是各个神经元的输出。

下面考察第 m 个神经元的输出变化前后,计算能量函数 E 值的变化。设 $v_m = 0$ 时的计算能量函数值为 E_1 。

$$E_1 = -\frac{1}{2} \sum_{i=1}^N \sum_{\substack{j=1 \\ j \neq i}}^N w_{ij} v_i v_j + \sum_{i=1}^N \theta_i v_i \quad (8.23)$$

将 $i=m$ 项分出来,并注意到 $v_m = 0$ 。

$$E_1 = -\frac{1}{2} \sum_{\substack{i=1 \\ i \neq m}}^N \sum_{\substack{j=1 \\ j \neq i}}^N w_{ij} v_i v_j + \sum_{\substack{i=1 \\ i \neq m}}^N \theta_i v_i \quad (8.24)$$

类似地,当 $v_m = 1$ 时的计算能量函数值为 E_2 ,则有

$$E_2 = -\frac{1}{2} \sum_{\substack{i=1 \\ i \neq m}}^N \sum_{\substack{j=1 \\ j \neq i}}^N w_{ij} v_i v_j + \sum_{\substack{i=1 \\ i \neq m}}^N \theta_i v_i - \sum_{\substack{j=1 \\ j \neq m}}^N w_{mj} v_j + \theta_m \quad (8.25)$$

当神经元状态由 0 变为 1 时,计算能量函数 E 值的变化量 ΔE 为

$$\Delta E = E_2 - E_1 = - \left(\sum_{\substack{j=1 \\ j \neq m}}^N w_{mj} v_j - \theta_m \right) \quad (8.26)$$

由于此时神经元的输出由 0 变为 1,因此满足神经元兴奋条件,即

$$\sum_{\substack{j=1 \\ j \neq m}}^N w_{mj} v_j - \theta_m > 0 \quad (8.27)$$

由式(8.26),得 $\Delta E < 0$ 。

当神经元状态由 1 变为 0 时,计算能量函数 E 值的变化量 ΔE 为

$$\Delta E = E_1 - E_2 = \sum_{\substack{j=1 \\ j \neq m}}^N w_{mj} v_j - \theta_m \quad (8.28)$$

由于此时神经元的输出由 1 变为 0,因此

$$\sum_{\substack{j=1 \\ j \neq m}}^N w_{mj} v_j - \theta_m < 0 \quad (8.29)$$

也得 $\Delta E < 0$ 。

综上所述,神经元状态变化时总有 $\Delta E < 0$,这表明神经网络在运行过程中能量将不断降低,最后趋于稳定的平衡状态。

关于离散型 Hopfield 神经网络的稳定性,早在 1983 年就由科恩(Cohen)与葛劳斯伯格(S. Grossberg)给出了稳定性的证明。Hopfield 等又进一步证明,只要连接权值构成的矩阵是非负对角元的对称矩阵,该网络就具有串行稳定性;若该矩阵为非负定矩阵,则该网络就具有并行稳定性。

8.4.2 连续型 Hopfield 神经网络及其 VLSI 实现

离散型 Hopfield 神经网络中的神经元与生物神经元的差别较大,因为生物神经元的输入与输出是连续的,而且生物神经元存在时滞。1984 年, Hopfield 又提出一种连续时间神经网络模型,在这种网络中,神经元的状态可以取 0 至 1 之间的任一实数值。

连续型 Hopfield 神经网络的电子线路实现如图 8.7 所示。

其中,每一个神经元由电阻 R_i 和电容 C_i 以及具有饱和非线性特性的运算放大器模拟,输出 v_i 同时还反馈至其他神经元,但不反馈自身。 u_i 表示神经元 i 的膜电位状态, v_i 表示它的输出, C_i 表示细胞膜输入电容, R_i 表示细胞膜的传递电阻,电阻 R_i 和电容 C_i 并联模拟了生物神经元输出的时间常数,而输出 v_i 对 u_j ($j=1,2,\dots,N$) 的影响则模拟了神经元之间互连的突触特性,运算放大器模拟神经元的非线性特性。

由基尔霍夫电流定律得,连续型 Hopfield 神经网络动力学系统方程为

$$\frac{1}{R_i} u_i + C_i \frac{du_i}{dt} = I_i + \sum_{j=1}^N w_{ij} v_j \quad (8.30)$$

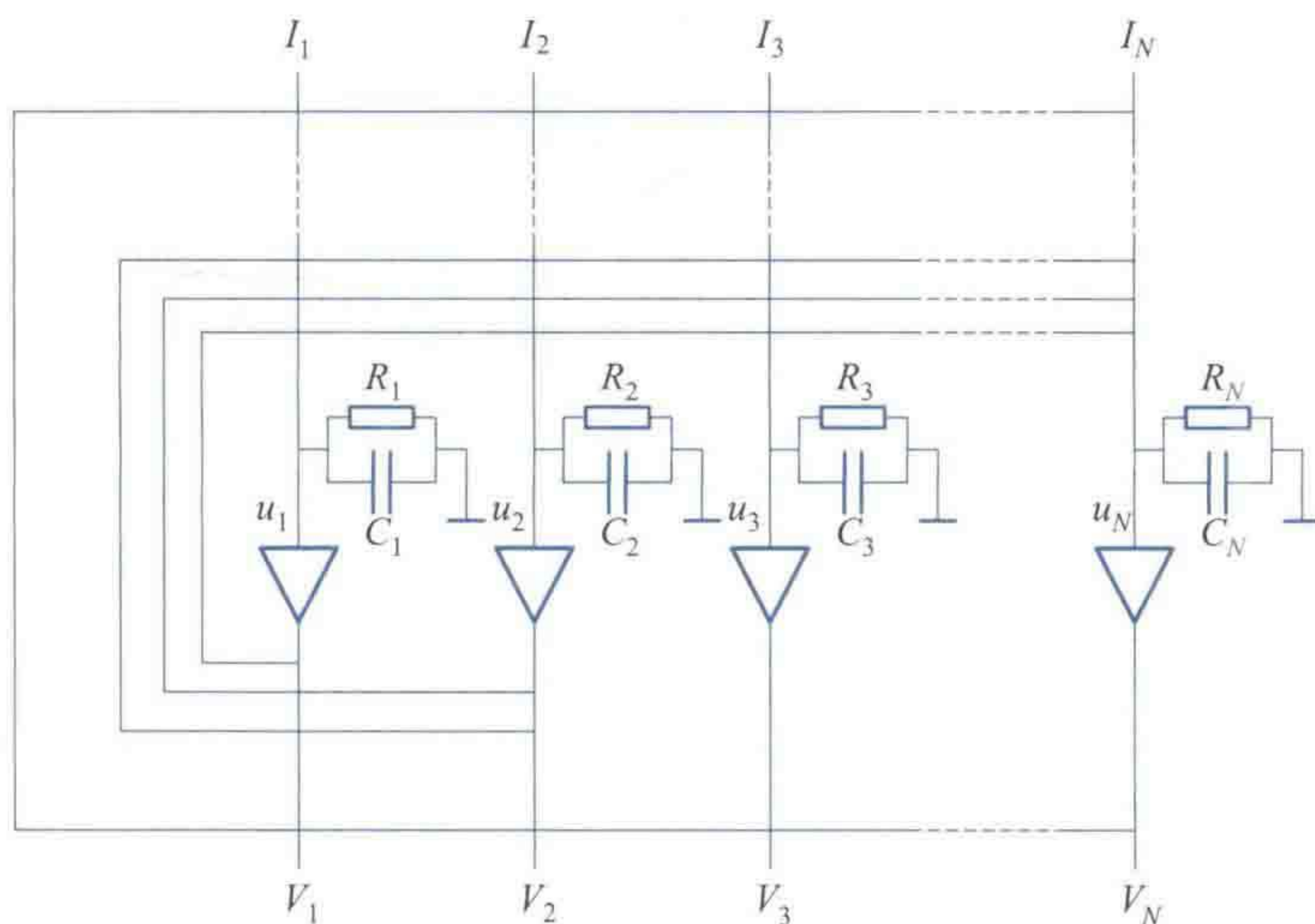


图 8.7 连续型 Hopfield 神经网络电子线路

$$v_i = f(u_i) = \frac{1}{1 + e^{-\frac{2u_i}{u_0}}} \quad (i = 1, 2, \dots, N)$$

其中, I_i 为施加在第 i 神经元上的偏置电流, 表示系统外部的输入; $w_{ij} = \frac{1}{R_{ij}}$ 模拟神经元 j 与 i 之间互连的突触特性; $f(u_i)$ 是放大器的非线性饱和特性, 近似于 S 型函数。

连续型 Hopfield 神经网络模型, 在简化了生物神经元性质的同时, 重点突出了以下重要特性。

- ① 神经元作为一个输入输出变换, 其传输特性具有 S 特性。
- ② 细胞膜具有时空整合作用。
- ③ 神经元之间存在大量的兴奋和抑制性连接, 这种连接主要是通过反馈来实现的。
- ④ 具有既代表产生动作电位的神经元, 又代表按渐近方式工作的神经元的能力。

因此, 连续型 Hopfield 神经网络准确地保留了生物神经网络的动态和非线性特征, 有助于理解大量神经元之间的协同作用是怎样产生巨大的计算能力的。

连续型 Hopfield 神经网络的计算能量函数 $E(t)$ 定义为

$$E(t) = -\frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N w_{ij} v_i(t) v_j(t) - \sum_{i=1}^N v_i(t) I_i + \sum_{i=1}^N \frac{1}{R_i} \int_0^{v_i(t)} f^{-1}(v) dv \quad (8.31)$$

定理 8.1 对于连续型 Hopfield 神经网络, 若 $f^{-1}(\cdot)$ 为单调递增的连续函数, $C_i > 0$, $w_{ij} = w_{ji}$, 则 $\frac{dE(t)}{dt} \leq 0$; 当且仅当 $\frac{dv_i(t)}{dt} = 0$ 时 ($1 \leq i \leq N$), $\frac{dE(t)}{dt} = 0$ 。

证明

$$\frac{dE(t)}{dt} = \sum_{i=1}^N \frac{\partial E(t)}{\partial v_i(t)} \frac{dv_i(t)}{dt} \quad (8.32)$$

而

$$\begin{aligned}
 \frac{\partial E(t)}{\partial v_i(t)} &= -\frac{1}{2} \sum_{j=1}^N w_{ij} v_j(t) - \frac{1}{2} \sum_{j=1}^N w_{ji} v_j(t) - I_i + \frac{1}{R_i} f^{-1}(v_i) \\
 &= -\frac{1}{2} \sum_{j=1}^N (w_{ji} - w_{ij}) v_j(t) - \left[\sum_{j=1}^N w_{ij} v_j(t) + I_i - \frac{u_i(t)}{R_i} \right] \\
 &= -\frac{1}{2} \sum_{j=1}^N (w_{ji} - w_{ij}) v_j(t) - C_i \frac{du_i(t)}{dt} \\
 &= -\frac{1}{2} \sum_{j=1}^N (w_{ji} - w_{ij}) v_j(t) - C_i \frac{df^{-1}(v_i)}{dt} \\
 &= -\frac{1}{2} \sum_{j=1}^N (w_{ji} - w_{ij}) v_j(t) - C_i \frac{df^{-1}(v_i)}{dv_i} \frac{dv_i}{dt}
 \end{aligned} \tag{8.33}$$

将式(8.33)代入式(8.32),得

$$\begin{aligned}
 \frac{dE(t)}{dt} &= \sum_{i=1}^N \frac{\partial E(t)}{\partial v_i(t)} \frac{dv_i(t)}{dt} \\
 &= -\frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N (w_{ji} - w_{ij}) v_j(t) \frac{dv_i(t)}{dt} - \sum_{i=1}^N C_i \frac{df^{-1}(v_i)}{dv_i} \left(\frac{dv_i}{dt} \right)^2
 \end{aligned} \tag{8.34}$$

由于 $w_{ij} = w_{ji}$, $C_i > 0$ 和 $f^{-1}(\cdot)$ 为单调递增, 即有 $\frac{df^{-1}(v_i)}{dv_i} > 0$, 所以从式(8.34)得

$$\frac{dE(t)}{dt} = - \sum_{i=1}^N C_i \frac{df^{-1}(v_i)}{dv_i} \left(\frac{dv_i}{dt} \right)^2 \leq 0 \tag{8.35}$$

从式(8.35)可见, 若 $\frac{dv_i}{dt} = 0$, 则 $\frac{dE(t)}{dt} = 0$; 反之, 若 $\frac{dE(t)}{dt} = 0$, 则必须满足 $\frac{dv_i}{dt} = 0$ 。

证毕。

上面的定理表明, Hopfield 神经网络的能量函数的值随着时间的推移总是在不断地减小, 神经网络趋于某一平衡状态, 平衡点就是 $E(t)$ 的极小值点, 反之亦然。这说明 Hopfield 神经网络的演变过程就是在 $[0, 1]^N$ 空间内寻找极小值稳定点(吸引子)的过程, 并在达到这些点后稳定下来。因此, 这种神经网络同样具有自动求极小值的计算功能。

无论是离散型 Hopfield 神经网络, 还是连续型 Hopfield 神经网络, 虽然理论上都能够收敛到极值点, 但一般都不能收敛到最优点。为了解决这个问题, 下面介绍随机神经网络。

8.4.3 随机神经网络

在 Hopfield 神经网络中, 神经元状态为 1 是根据其输入是否大于阈值确定的, 是确定性的。而在随机神经网络中, 神经元状态为 1 是随机的, 服从一定的概率分布。例如, 服从玻耳兹曼 (Boltzmann)、高斯 (Gaussian)、柯西 (Cauchy) 分布等, 从而构成玻耳兹曼机、高斯机、柯西机等

随机机。

下面主要以玻耳兹曼机为例,介绍随机神经网络。

1985 年,加拿大多伦多大学教授辛顿(Hinton)等人借助统计物理学的概念和方法,提出了一种随机 Hopfield 神经网络,称为 Boltzmann 机神经网络模型,也称为 Boltzmann 机。它是离散型 Hopfield 神经网络的一种变形,通过对离散型 Hopfield 神经网络加以扰动,使其以概率的形式表达,而网络的模型方程不变,只是输出值类似于 Boltzmann 分布,以概率分布取值。Boltzmann 机是按 Boltzmann 概率分布动作的神经网络。

离散型 Hopfield 神经网络的输出为

$$v_i(k+1) = \text{sgn} \left[\sum_{\substack{j=1 \\ j \neq i}}^N w_{ij} v_j(k) - \theta_i \right] \quad (8.36)$$

对于 Boltzmann 机,设内部状态为

$$I_i = \sum_{\substack{j=1 \\ j \neq i}}^N w_{ij} v_j(k) - \theta_i \quad (8.37)$$

神经元 i 输出值为 0 和 1 时的概率分别为 $p_i(0)$ 和 $p_i(1)$,即

$$p_i(1) = \frac{1}{1 + e^{-I_i/T}} \quad (8.38a)$$

$$p_i(0) = 1 - p_i(1) \quad (8.38b)$$

其中, T 为类似温度的扰动。

从式(8.38)可见,当 T 较大时, $p_i(0)$ 和 $p_i(1)$ 几乎相等;当 T 较小时,两概率值将由系统内部状态决定。因此,在系统运行时,先将 T 置于较大值以使系统能够越过能量较大的状态来避免陷入局部极小,然后将 T 值逐渐减小,从而使系统最终收敛到全局能量最小的状态。这与模拟退火算法的机制几乎一样。应用模拟退火算法的 Hopfield 神经网络,实际上就是一种最基本的 Boltzmann 机。由式(8.38a)可知, I_i 越大, $p_i(1)$ 越大,即神经元变为 1 的可能性越大。

设神经网络由 N 个神经元构成,每个神经元服从二态规律,即只取 0,1 两种状态,且假定神经元之间的连接权矩阵是对称的。在网络中,当神经元的输入加权和发生变化时,将引起神经元状态的更新,这种更新在各个神经元之间是非同步的,可用概率分析方法描述。设 $\Delta E_i = E(v_i=0) - E(v_i=1)$,那么,神经元状态 v_i 为 1 的概率 p_i 服从玻耳兹曼分布。

与 HNN 相类似,Boltzmann 的能量函数定义为

$$E = -\frac{1}{2} \sum_i \sum_{j \neq i} w_{ij} v_i v_j + \sum_i v_i \theta_i \quad (8.39)$$

其中, v_i 和 θ_i 分别是神经元 i 的状态和阈值; w_{ij} 为连接权。首先按下式计算神经元 i 状态转换时网络能量的变化

$$\Delta E_i = \sum_j w_{ij} v_j - \theta_i \quad (8.40)$$

然后,依据 ΔE_i 的大小,将神经元 i 以概率

$$p_i = \frac{1}{1 + \exp\left(-\frac{\Delta E_i}{T}\right)} \quad (8.41)$$

改变为状态“1”。

上式中, T 为温度。此式表明: 在很高的温度下 ($T \gg 1$), p_i 将接近于 0.5, 神经元行为具有随机性; 在同一温度下, ΔE_i 较大时, 状态为“1”的概率增大; 在温度很低时, 随机性将趋于消失, 此时玻耳兹曼机性能与确定性网络相同。

8.5 Hopfield 神经网络的应用

Hopfield 神经网络的许多应用都是利用 Hopfield 神经网络能够收敛到一个稳定状态这个特性。例如, 利用 Hopfield 神经网络的稳定状态能够实现联想记忆、优化等应用。下面简单介绍基于 Hopfield 神经网络的联想记忆与优化方法。

8.5.1 Hopfield 神经网络在联想记忆中的应用

如果将网络的一个稳态作为一个记忆样本, 那么以后当给这个网络一个适当的激励时, 网络能够收敛到和输入模式最为相似的样本模式。初态朝稳态的收敛过程便是寻找记忆样本的过程。初态可以认为是给定样本的部分信息, 网络改变的过程可以认为是从部分信息找到全部信息, 从而实现了联想记忆的功能。

Hopfield 神经网络联想记忆过程, 就是非线性动力学系统朝某个稳定状态运行的过程。这就需要调整连接权值使得所要记忆的样本作为 Hopfield 神经网络的能量局部极小点。Hopfield 神经网络联想记忆过程分为学习和联想两个阶段。在给定样本的条件下, 调整连接权值, 使得存储的样本成为 Hopfield 神经网络的稳定状态, 这就是学习阶段。联想是指在已经调整好权值不变的情况下, 给出部分不全或者受了干扰的信息, 按照动力学规则改变神经元的状态, 使神经网络最终变到某个稳定状态。

实现 Hopfield 神经网络联想记忆的关键是网络到达记忆样本能量函数极小点时, 决定网络的神经元间连接权值 w_{ij} 和阈值 θ_i 等参数。下面介绍按照 Hebb 学习规则设计 Hopfield 神经网络的连接权值。

设给定 m 个样本 $x^{(k)}$ ($k=1, 2, \dots, m$), $x_i^{(k)}$ 表示第 k 个样本中的第 i 个元素。记 w_{ij} 是神经元 i 到神经元 j 的权值。

当神经元输出 $x_i \in \{-1, +1\}$ 时:

$$w_{ij} = \begin{cases} \sum_{k=1}^m x_i^{(k)} x_j^{(k)} & i \neq j \\ 0 & i = j \end{cases} \quad (8.42a)$$

或者

$$\begin{aligned} w_{ij}(k) &= w_{ij}(k-1) + x_i^{(k)} x_j^{(k)} \quad (k=1, 2, \dots, m) \\ w_{ij}(0) &= 0 \quad w_{ii} = 0 \end{aligned} \quad (8.42b)$$

当神经元输出 $x_i \in \{0, 1\}$ 时:

$$w_{ij} = \begin{cases} \sum_{k=1}^m (2x_i^{(k)} - 1)(2x_j^{(k)} - 1) & i \neq j \\ 0 & i = j \end{cases} \quad (8.43a)$$

或者

$$\begin{aligned} w_{ij}(k) &= w_{ij}(k-1) + (2x_i^{(k)} - 1)(2x_j^{(k)} - 1) \quad (k=1, 2, \dots, m) \\ w_{ij}(0) &= 0 \quad w_{ii} = 0 \end{aligned} \quad (8.43b)$$

显然,按照上面公式设计的网络连接权值满足对称条件。可以证明,按照上面公式设计网络连接权值时,Hopfield 神经网络的稳定状态是给定样本。

依据上述算法的联想记忆功能,可用于模式识别。但当样本多且彼此相近时,容易引起混淆。在网络结构与参数一定的条件下,要保证联想功能的正确实现,网络所能存储的最大的样本数,称为网络的记忆容量。网络的记忆容量不仅与神经元个数有关,还与连接权值的设计、要求的联想范围大小、样本的性质等有关。当网络要求存储的样本模式是两两正交时,可以有最大的记忆容量。

例 8.2 设计基于 Hopfield 神经网络的分类器。

当人看见苹果和橘子的时候,虽然和以前见过的不完全一样,但通过自联想能力仍然能够识别。利用 Hopfield 神经网络的联想特性,能够设计苹果和橘子的分类器,如图 8.8 所示。输送带将苹果和橘子传送给外形、质地、质量三个传感器检测,Hopfield 神经网络根据传感器检测结果识别,如果识别结果是苹果则执行器就将这个苹果放进苹果筐,否则放进橘子筐。

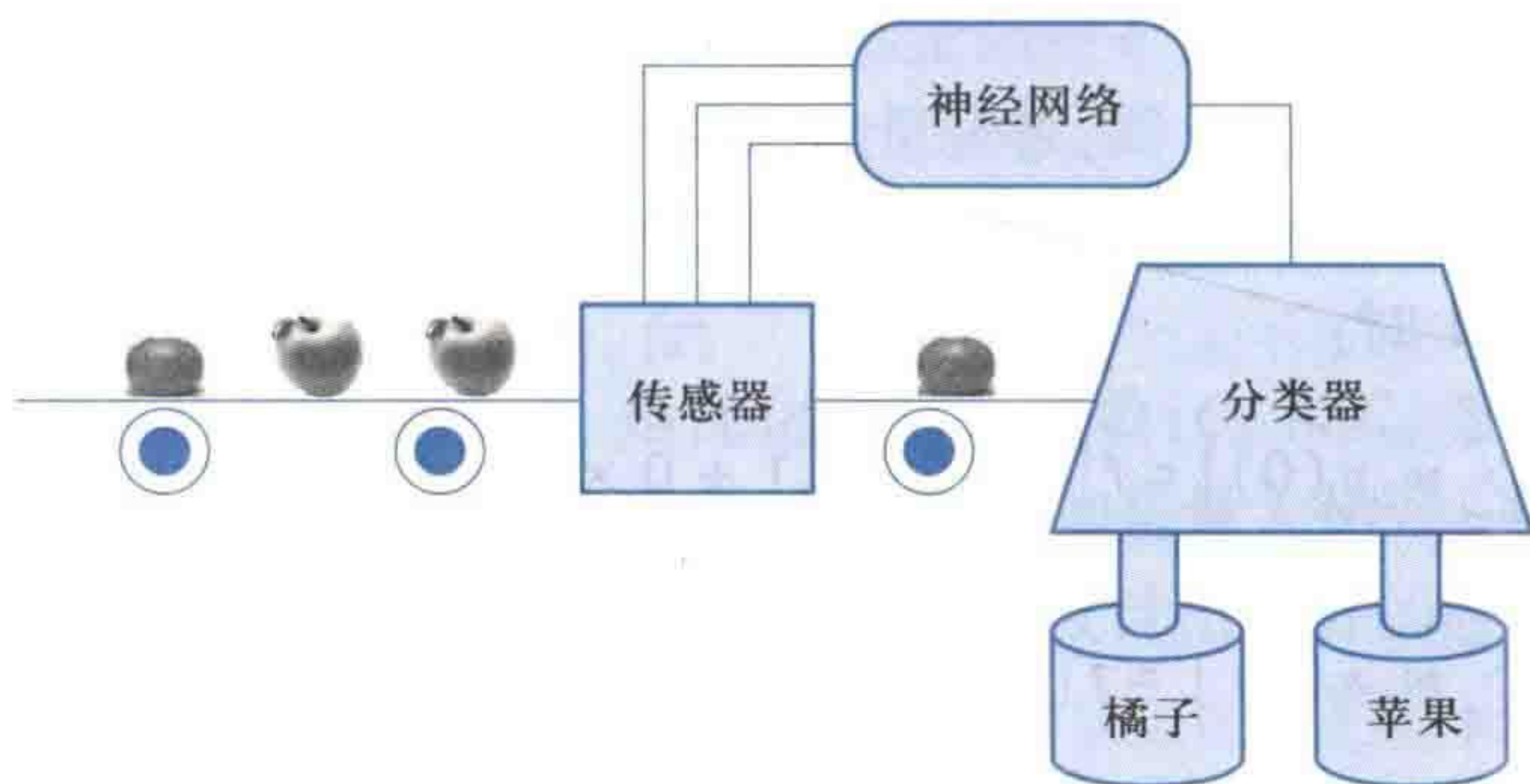


图 8.8 基于 Hopfield 神经网络的分类器

传感器检测结果为 1 或者 0,其意义如表 8.1 所示。

表 8.1 分类特征编码

	1	0
外形	圆	椭圆
质地	光滑	粗糙
质量	<300 g	>300 g

三个传感器输出表示为[外形 质地 质量],则标准橘子表示为: $\mathbf{x}^{(1)} = [1 \ 0 \ 1]^T$ 。标准苹果表示为: $\mathbf{x}^{(2)} = [0 \ 1 \ 0]^T$ 。

(1) 设计 DHNN 结构

设计有 3 个神经元的 Hopfield 神经网络如图 8.9 所示。3 个神经元的阈值都为 0。

(2) 设计连接权矩阵

$$w_{ij} = \begin{cases} \sum_{k=1}^2 (2x_i^{(k)} - 1)(2x_j^{(k)} - 1) & i \neq j \\ 0 & i = j \end{cases}$$

$$w_{ji} = w_{ij} \quad i = 1, 2, \dots, n \quad j = 1, 2, \dots, n$$

$$w_{12} = (2 \times 1 - 1) \times (2 \times 0 - 1) + (2 \times 0 - 1) \times (2 \times 1 - 1) = -1 - 1 = -2$$

$$w_{21} = w_{12} = -2$$

$$w_{23} = (2 \times 0 - 1) \times (2 \times 1 - 1) + (2 \times 1 - 1) \times (2 \times 0 - 1) = -1 - 1 = -2$$

$$w_{32} = w_{23} = -2$$

$$\mathbf{W} = \begin{bmatrix} 0 & -2 & 2 \\ -2 & 0 & -2 \\ 2 & -2 & 0 \end{bmatrix}$$

(3) 测试

测试用例: $[1 \ 1 \ 1]^T$,取初始状态: $v_1(0) = 1, v_2(0) = 1, v_3(0) = 1$ 。神经元状态调整次序取为:2→1→3。当 $k=1$ 时:

$$v_2(1) = f\left[\sum_{j=1}^3 w_{2j}v_j(0)\right] = f[(-2) \times 1 + 0 \times 1 + (-2) \times 1] = f(-4) = 0$$

$$v_1(2) = f\left[\sum_{j=1}^3 w_{1j}v_j(1)\right] = f[0 \times 1 + (-2) \times 0 + 2 \times 1] = f(2) = 1$$

神经元状态调整为: $v_1(1) = 1, v_2(1) = 0, v_3(1) = 1$ 。当 $k=2$ 时:

$$v_1(1) = v_1(0) = 1, \quad v_3(1) = v_3(0) = 1$$

$$v_2(2) = v_2(1) = 0, \quad v_3(2) = v_3(1) = 1$$

神经元状态调整为: $v_1(2) = 1, v_2(2) = 0, v_3(2) = 1$ 。

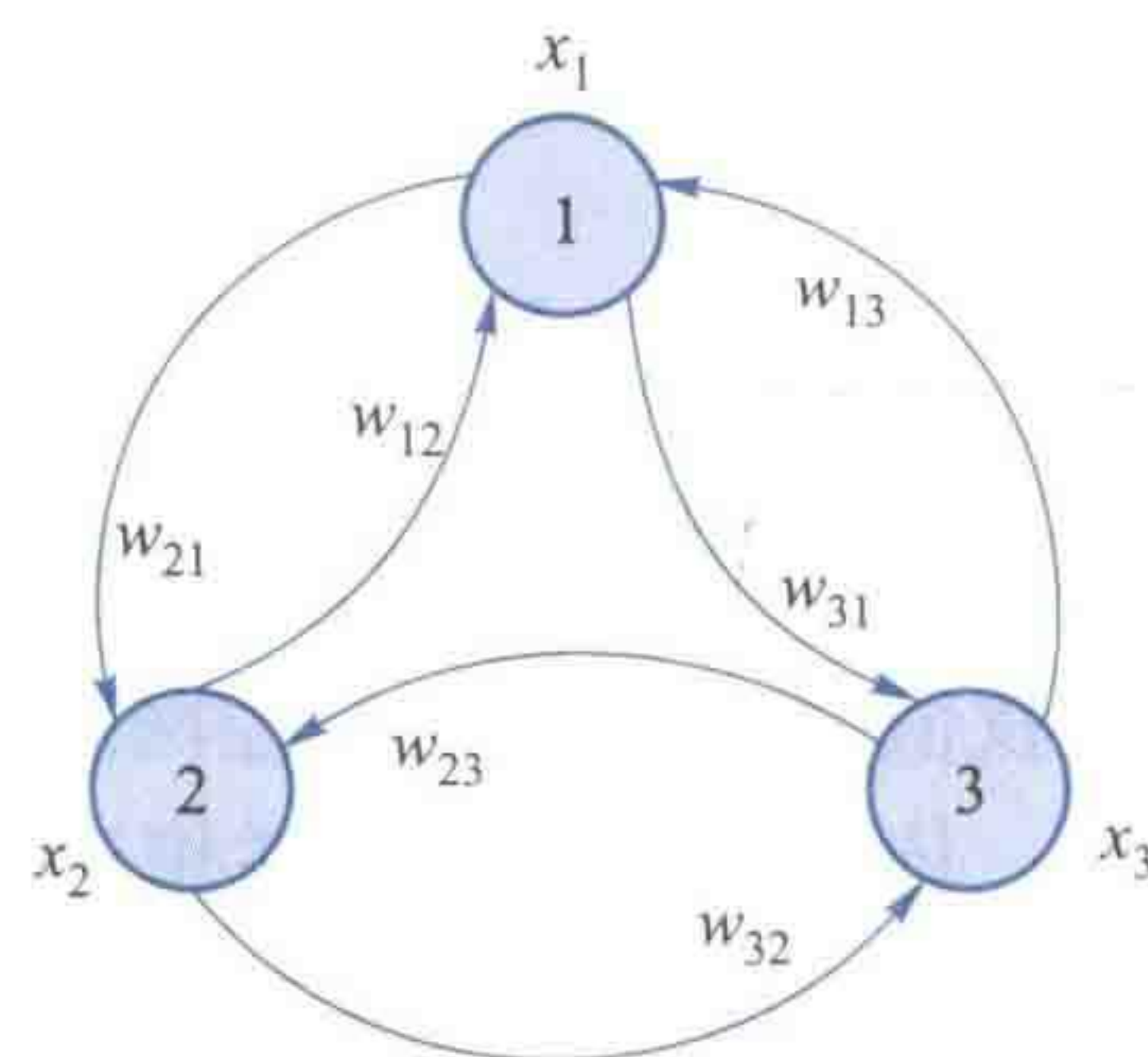


图 8.9 3 个神经元的 Hopfield 神经网络

类似地,有: $v_1(3)=1, v_2(3)=0, v_3(3)=1$ 。

可见输入 $[1 \ 1 \ 1]^T$,输出 $[1 \ 0 \ 1]^T$ 。

8.5.2 Hopfield 神经网络优化方法

如果将网络的稳态作为一个优化问题的目标函数的极小点,那么初态朝稳态的收敛过程便是优化计算的过程。该优化计算是在网络演化过程中自动完成的。

1985 年霍普菲尔德和塔克(D. W. Tank)应用神经网络方法求解旅行商问题这一著名的组合优化难题获得成功,引起了世界各国学者的广泛关注。他们的工作不仅对研究神经网络理论具有重要意义,也为组合优化问题求解开辟了新的途径。

1. 基本思想

无论是离散型 Hopfield 神经网络模型还是连续型 Hopfield 神经网络模型,能量函数都表征了网络的动力学演化过程,并揭示了该演化过程与网络稳定状态之间的内在联系。因此, Hopfield 能量函数极小化过程表示了神经网络从初始状态到稳定状态的演化过程。通常,约束优化问题的求解过程实际上就是目标函数的极小化过程,选择合适的能量函数使其最小值对应于问题的最优解。因此,目标函数达到局部极小或者全局最小,相应的解为约束优化问题的局部最优解或者全局最优解。如果将目标函数与能量函数相联系,并通过能量函数将约束优化问题的解映射到神经网络的一个稳定状态上去,那么,就可以利用神经网络的演化过程来实现优化计算。这就是应用神经网络解决优化问题的基本思想。

用神经网络方法求解优化问题的关键是如何把待求解的优化问题映射为一个神经网络。一般可以将求解的组合优化问题的每一个可行解用换位矩阵(permutation matrix)表示。

另一个关键问题是构造能量函数,使其最小值对应于问题的最优解。它决定了一个特定问题是否能够用神经网络方法求解。目前还没有直接将约束优化问题映射为神经网络的方法,通常采用优化理论中的拉格朗日(Lagrange)函数和乘子法,即利用优化问题的目标函数和约束条件构造相应的能量函数。一般使用在计算能量函数 E 中添加一些违反约束条件的惩罚项的简单方法,用罚函数法写出求解优化问题的能量函数,即

$$E = \sum_{i=1}^m C_i E_i + C_0 E_0 \quad (8.44)$$

其中, E_i 是违背约束条件的惩罚函数; E_0 是优化的目标函数; C_i 和 C_0 为平衡 E_i 和 E_0 在总能量函数中的作用的比例常数,且如果最小化 E_0 ,则 $C_0 > 0$,如果最大化 E_0 ,则 $C_0 < 0$ 。

显然,神经网络和能量函数的形式不是唯一的,如何设计出更好的映射方法及其能量函数构造方法是一个重要的课题。

2. 一般步骤

基于上述基本思想,用神经网络方法求解优化问题的一般步骤如下:

- ① 将求解的优化问题的每一个可行解用换位矩阵表示;
- ② 将换位矩阵与由 n 个神经元构成的神经网络相对应,每一个可行解所对应的换位矩阵

的各元素与相应的神经元稳态输出相对应;

③ 构造能量函数,使其最小值对应于优化问题的最优解,并满足约束条件;

④ 用罚函数法构造目标函数,与 Hopfield 神经网络的计算能量函数表达式相等,确定各连接权 w_{ij} 和偏置参数 I_i 等;

⑤ 给定网络初始状态和网络参数 A, B, C, D 等,使网络(可以是计算机模拟)按动态方程运行,直到达到稳定状态,并将它解释为优化问题的解。

Hopfield 神经网络模型与初始状态有关,例如在 TSP 问题中,原则上可选 $v_{xi}^0 = 1/n$ 为每个神经元的初始值,因为它满足 $\sum_x \sum_i v_{xi}^0 = n$ 。但由于相同长度的等价路径有 $2n$ 条,系统无法从中抉择,所以要加一定的噪声值($\pm 0.1v_{xi}^0$)来打破这种平衡。

当在计算机上实现上述演化时,需要离散化。此时, Δt 的选择很重要, Δt 太大可能导致离散后的算法与原连续算法有很大差异,甚至不收敛; Δt 太小则迭代次数太多,计算时间长。可以证明,只要 Δt 选取合理,上述算法一定收敛。但如果不加其他条件,收敛到的解不一定对应有效的访问路径,可能是不可行解。

鉴于神经网络动力学性质的复杂性,神经网络优化计算方面仍有许多问题需要进一步深入研究。目前存在的主要问题如下:

- ① 计算结果的不稳定性;
- ② 系数 A, B, C, D 的确定;
- ③ 能量函数存在大量局部极小值,求解结果不能保证为最优解。

3. Hopfield 神经网络优化方法求解 TSP

旅行商问题(traveling salesman problem, TSP)或者邮递员路径问题:有 n 个城市,其相互间的距离,或者旅行成本为已知,求合理的路线使每个城市都访问一次,且总路径(或者总成本)为最短。

TSP 是数学家、经济学家 Karl Menger 在 19 世纪 20 年代首先提出来的,但直到 20 世纪 40 年代 Merrill Flood 和他在 RAND 公司的同事们讨论后,TSP 才开始普及起来。那时人们对组合问题特别感兴趣。数学家们非常喜欢研究 TSP,这是因为 TSP 描述起来很简单,但要解决却是非常困难的。当城市数目不断增加时,求解问题所需要的计算量呈指数级增长。TSP 现在仍然被广泛用作新的组合优化方法的测试问题。

对于 TSP,一条访问路径可以用一个换位矩阵表示。以 5 个城市为例,如表 8.2 所示用换位矩阵表示访问 5 个城市的路径顺序为 $C_3 \rightarrow C_1 \rightarrow C_5 \rightarrow C_2 \rightarrow C_4 \rightarrow C_3$,其路径总长度为

$$l = d_{C_3C_1} + d_{C_1C_5} + d_{C_5C_2} + d_{C_2C_4} + d_{C_4C_3}$$

如果下标 x, y 表示城市, i 表示第 i 次访问,则路径长度可以表示为下列一般形式:

$$\begin{aligned} l &= \frac{1}{2} \sum_x \sum_{y \neq x} \sum_i d_{xy} v_{xi} v_{y, i+1} + \frac{1}{2} \sum_x \sum_{y \neq x} \sum_i d_{xy} v_{xi} v_{y, i-1} \\ &= \frac{1}{2} \sum_x \sum_{y \neq x} \sum_i d_{xy} v_{xi} (v_{y, i+1} + v_{y, i-1}) \end{aligned} \quad (8.45)$$

其中, d_{xy} 表示城市 x, y 之间的距离; v_{xi} 表示换位矩阵中的第 x 行第 i 列的元素, 其值为 1 时表示第 i 步访问城市 x , 其值为 0 时表示第 i 步不访问城市 x 。

表 8.2 用换位矩阵表示访问次序

城市	1	2	3	4	5
C_1	0	1	0	0	0
C_2	0	0	0	1	0
C_3	1	0	0	0	0
C_4	0	0	0	0	1
C_5	0	0	1	0	0

在表 8.2 中, 各行各列只能有一个元素为 1, 其余都是 0, 否则它表示一条无效的路径。每列中只有一个元素为 1, 表示每次只访问一个城市, 可以表示为

$$\sum_x v_{xi} = 1 \quad \forall i \quad (8.46)$$

每行中只有一个元素为 1, 表示每个城市必须且只能访问一次, 可以表示为

$$\sum_i v_{xi} = 1 \quad \forall x \quad (8.47)$$

综合上述讨论, TSP 可以表示为如下优化问题:

$$\begin{aligned} \min l &= \frac{1}{2} \sum_x \sum_{y \neq x} \sum_i d_{xy} v_{xi} (v_{y,i+1} + v_{y,i-1}) \\ \text{st.} \quad &\sum_x v_{xi} = 1 \quad \forall i \quad (\text{每个城市必须访问一次}) \\ &\sum_i v_{xi} = 1 \quad \forall x \quad (\text{每个城市只能访问一次}) \end{aligned} \quad (8.48)$$

用罚函数法, 将上述约束优化问题表示为下列无约束优化问题:

$$\begin{aligned} J &= \frac{A}{2} \sum_x \sum_i \sum_{j \neq i} v_{xi} v_{xj} + \frac{B}{2} \sum_i \sum_x \sum_{y \neq x} v_{xi} v_{yi} + \frac{C}{2} \left(\sum_x \sum_i v_{xi} - n \right)^2 \\ &+ \frac{D}{2} \sum_x \sum_{y \neq x} \sum_i d_{xy} v_{xi} (v_{y,i+1} + v_{y,i-1}) \end{aligned} \quad (8.49)$$

令式(8.49)与 Hopfield 神经网络的计算能量函数相等, 比较同一变量两端的系数, 可得第 x 行第 i 列位置上的神经元与第 y 行第 j 列位置上的神经元之间的连接权值为

$$\begin{aligned} W_{xi,yj} &= -A\delta_{xy}(1 - \delta_{ij}) - B\delta_{ij}(1 - \delta_{xy}) - C - Dd_{xy}(\delta_{j,i+1} + \delta_{j,i-1}) \\ I_{xi} &= Cn \end{aligned} \quad (8.50)$$

其中

$$\delta_{ij} = \begin{cases} 1 & i = j \\ 0 & \text{其他} \end{cases}$$

Hopfield 神经网络的动态方程为

$$\begin{aligned}\frac{du_{xi}}{dt} &= -\frac{u_{xi}}{\tau} - \frac{\partial E}{\partial v_{xi}} \\ &= -\frac{u_{xi}}{\tau} - A \sum_{j \neq i} v_{xj} - B \sum_{y \neq x} v_{yi} - C \left(\sum_x \sum_i v_{xi} - n \right) - D \sum_{y \neq x} d_{xy} (v_{y,i+1} + v_{y,i-1}) \quad (8.51) \\ v_{xi} &= f(u_{xi}) = \frac{1}{2} \left[1 + \tanh \left(\frac{u_{xi}}{u_0} \right) \right]\end{aligned}$$

求解上式,直到收敛,可以得到神经网络的稳态解。在演化过程中,有些神经元的输出 v_{xi} 逐渐增大到 1,而有些神经元的输出 v_{xi} 逐渐减小到 0,最后收敛到稳定状态,所以,神经元输出 0 或者 1。

8.6 卷积神经网络及其应用

8.6.1 动物视觉机理与深度学习的提出

BP 神经网络以数值作为输入。如果要处理图像相关的信息,则要先从图像中提取特征。BP 学习算法于 1991 年被指出存在梯度消失 (vanishing gradient) 问题,即网络的输出结果通过反向传播,当到达最前面几层时,梯度会逐渐消失,不能指引网络权值的训练,从而导致网络训练过程不能收敛。因此,BP 学习算法一般只能用于浅层网络结构的学习,这就限制了 BP 学习算法的数据表征能力,影响了其在诸多工程领域中的应用。成为影响 BP 神经网络发展的主要原因,使人工神经网络的研究进入第二次低潮。

可以利用很多数学和工程技巧用来增加神经网络的层数,即深度,所以多层的神经网络称为深度神经网络 (deep neural networks, DNN),相应的学习算法被称为深度学习 (deep learning, DL)。对于深度神经网络,如何减少神经网络的权值数量从而减少训练量,是一个非常重要的问题。

1958 年,著名神经生物学家大卫·休伯尔 (David Hunter Hubel) 与托斯坦·威泽尔 (Torsten N. Wiesel) 在研究猫的脑皮层中不同视觉神经元与瞳孔所受到刺激之间存在某种对应关系时,发现了一种方向性选择细胞的神经元细胞,从而揭示了“视觉系统信息分层处理”的机制,动物大脑皮层对视觉信息的处理是分级、分层进行的。正是由于这个重要的生理学发现,二人获得了 1981 年的诺贝尔生理学或医学奖。这个重要科学发现的意义并不限于生理学领域,它也间接促成了五十年后人工智能的突破性发展。休伯尔和威泽尔等人对大脑的“深入认识”,启迪了计算机科学家,为科研人员从“观察大脑”到“重现大脑”搭起了桥梁。

1980 年,日本学者福岛邦彦 (Kunihiko Fukushima) 模拟生物视觉系统,提出了一种带有卷积和池化操作的多层人工神经网络,称为“神经认知机” (neocognitron) 模型,没有采用 BP 算法进行训练,而是采用无监督学习方式训练。神经认知机是卷积神经网络的雏形,但当时没

有引起重视。自此之后,很多计算机科学家先后对“神经认知机”做了深入研究和改进,但效果不尽如人意。直到1990年,在AT&T贝尔实验室工作的美国纽约大学Yann LeCun等,将BP学习算法应用于福岛邦彦等人提出的架构,从而奠定了现代卷积神经网络的结构,被认为是第一个真正意义上的卷积神经网络。直到20世纪,大规模的训练数据和高效的计算能力这两个制约CNN应用与发展的瓶颈得以大大缓解。因此,深度卷积神经网络(deep convolutional neural network, DCNN)的研究再次火爆起来。

2006年,Hinton教授和他的同事们提出了一种称为深度信念网络(deep belief network, DBN)的多层网络并进行了有效训练。根据生物学的重要发现,又提出了一种通过多层神经网络进行数据降维的方法,正式提出了深度学习的概念,发表在著名学术刊物*Science*上,指出梯度消失的问题可以采用“无监督训练对权值进行初始化,再使用有监督训练进行微调”的方法解决。2012年,Hinton和他的学生Alex Krizhevsky等,提出了8层神经网络AlexNet,它引入了ReLU激活函数,使用GPU进行加速。为了证明卷积神经网络的效果,Hinton和他的学生Alex Krizhevsky参加ImageNet图像识别大赛(ILSVRC),使用了深度卷积网络AlexNet进行图像分类,一举夺得分类任务冠军,其错误率比采用传统浅层模型的第二名降低了惊人的10.9%。这些成功应用掀起了深度学习的新浪潮,各种深度学习模型先后涌现。

本节和后面几节将分别介绍应用广泛的卷积神经网络、胶囊网络以及生成对抗网络。

8.6.2 卷积神经网络的结构

卷积神经网络是一种具有局部连接、权值共享等特点的前馈神经网络,每层由多个二维平面组成,而每个平面由多个独立神经元组成。

卷积神经网络的结构如图8.10所示。

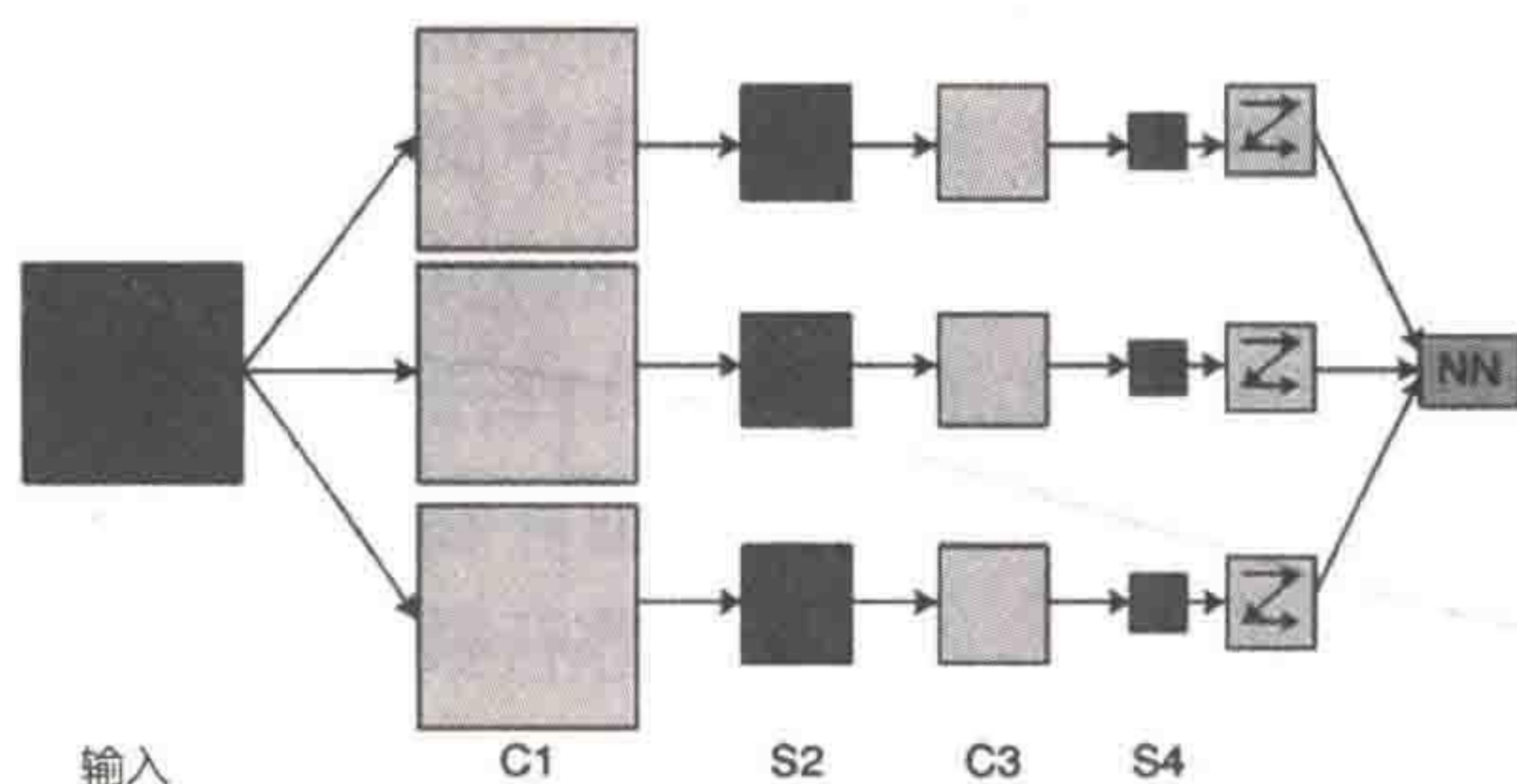


图 8.10 卷积神经网络的结构示例

C层为特征提取层,称为卷积层,代表对输入图像进行卷积后得到的所有组成的层。每个神经元的输入与前一层的局部感受野(local receptive field)相连,并提取该局部的特征。

S层是特征映射层,称为池化层(pooling),也称下采样层(subsampling),代表对输入图像进行池化得到的层。网络的每个计算层由多个特征映射组成,每个特征映射为一个平面,平面上所有神经元的权值相等。特征映射结构采用Sigmoid函数作为卷积网络的激活函数,使得特

征映射具有位移不变性。

CNN 中的每一个特征提取层 C 都紧跟着一个用来求局部平均与二次提取的计算层 S。C、S 层中的每一层都由多个二维平面组成,每一个二维平面是一个特征图(feature map)。这种特有的两次特征提取结构能够容许识别过程中输入样本有较严重的畸变。卷积层同时对其输入使用多个卷积核,使之能检测到输入的多个特征。

在图 8.10 中,输入图像通过和三个卷积核和可加偏置进行卷积,卷积后在 C1 层产生三个特征图。C1 层的三个特征图分别通过池化,对特征图中每组的四个像素再进行求和、加权值、加偏置,得到 S2 层的三个特征图。这三个特征图通过一个卷积核卷积得到 C3 层的三个特征图。与前面类似,池化得到 S4 层的三个特征图。最后,S4 层的特征图光栅化后,变成向量。这个向量输入到传统的全连接神经网络 NN 中进行进一步的分类,得到输出。在同一个特征图中,所有的神经元具有相同的参数(权值或阈值),但不同的特征图可能有不同的参数。

图 8.10 中的 C1、S2、C3、S4 层中的所有特征图都可以用“像素×像素”定义图像大小。由于这些特征图组成了神经网络的卷积层和池化层,这些特征图中的每一个像素恰恰就代表了一个神经元。每一层所有特征图的像素个数,就是这层网络的神经元个数。

8.6.3 卷积神经网络的卷积运算

卷积(convolution)源自拉丁文“convolvere”,其含义就是“卷在一起(roll together)”,是数学上的一个重要的运算,由于其具有丰富的物理、生物、生态等意义,所以具有非常广泛的应用。卷积运算分为离散卷积运算与连续卷积运算、二维离散卷积运算与二维连续卷积运算。

下面介绍卷积神经网络中用到的二维离散卷积运算方法。

在图像处理中,采用卷积运算对输入图像或 CNN 上一层的特征图进行变换,也就是特征抽取,得到新的特征。这就是为什么卷积之后的结果被称为“特征图”的原因。

一幅灰度图片可以用一个像素矩阵表示。矩阵中的每个数字的取值范围为 $[0, 255]$ 。0 表示黑色,255 表示白色,其他灰度为介于 0 到 255 的整数。如果是彩色图片,则用 RGB 三个像素矩阵共同表示。如(255,0,0)表示红色,(218,112,214)表示淡紫色。每个像素矩阵称之为通道,因此,灰度图像为单通道,彩色图像为 3 通道。在数学上,把这样的 3 通道数据矩阵称为三阶张量(tensor)。张量的长度和宽度即像素矩阵行数和列数分别为图像的分辨率,通道数称为高度。

人类通过长期的进化,当眼睛看到图像时,大脑就自动提取出很多用以识别类别的特征。但对计算机而言,从一系列的数字矩阵提取特征不是一件简单的事情。在计算机的“眼睛”里,图像是数字矩阵,那么提取图像的特征实际上就是对数字矩阵进行运算,其中非常重要的运算就是卷积。

为计算简单起见,考虑一个给定 5×5 的像素值的矩阵,它的像素值仅为 0 或 1(实际灰度图像的像素值的范围是 0~255)。卷积核是一个 3×3 的矩阵,其中的值也是 0 或 1(实际上可以是其他值),如图 8.11 所示。

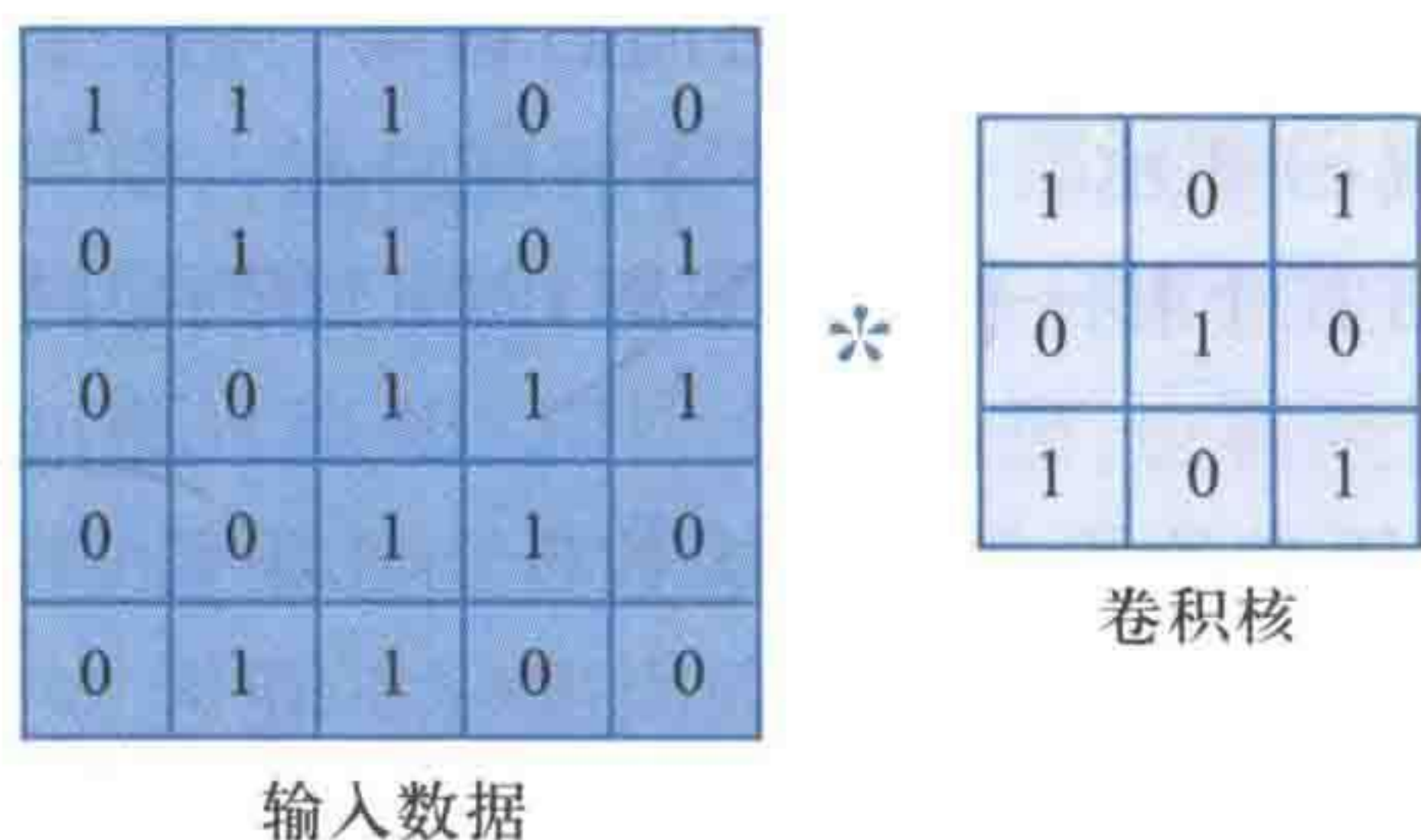


图 8.11 输入矩阵和卷积核

用卷积核矩阵在输入矩阵上从左到右、从上到下滑动,每次滑动 s 个像素,滑动的距离 s 称为步幅(stride)。卷积特征矩阵是输入矩阵和卷积核矩阵重合部分的内积,即卷积特征矩阵每个位置上的值是重合部分两个矩阵间的相应元素乘积之和。因此,卷积特征矩阵称为特征图。

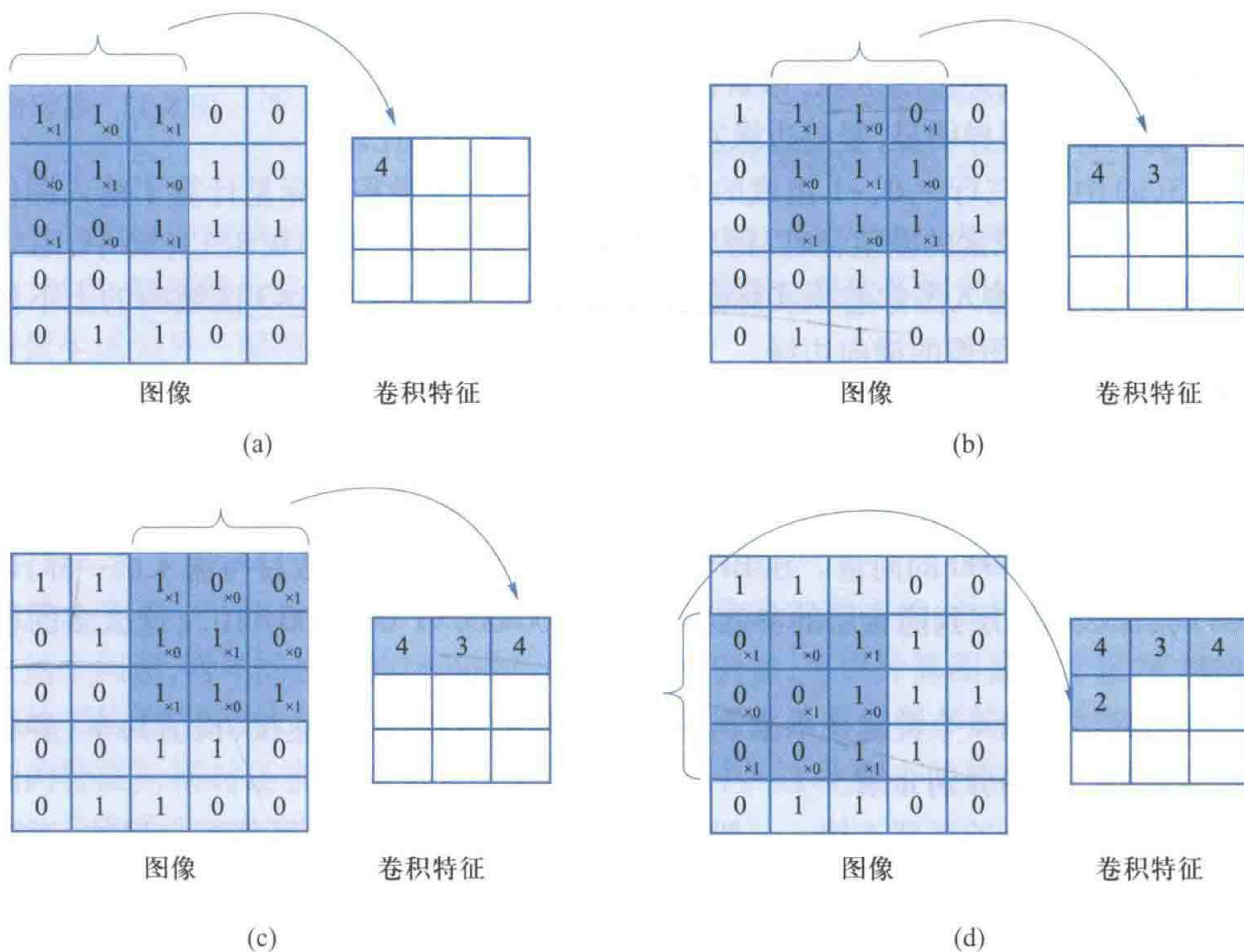


图 8.12 矩阵卷积运算

例如,卷积特征矩阵中第一行的第一个元素为: $(1 \times 1 + 1 \times 0 + 1 \times 1) + (0 \times 0 + 1 \times 1 + 1 \times 0) + (0 \times 1 + 0 \times 0 + 1 \times 1) = 2 + 1 + 1 = 4$,如图 8.12(a)所示。然后把卷积核矩阵向右移动一格(设步幅 $s = 1$),可以得到卷积特征矩阵中第一行的第二个元素为: $(1 \times 1 + 1 \times 0 + 0 \times 1) + (1 \times 0 + 1 \times 1 + 1 \times 0) + (0 \times 1 +$

$1 \times 0 + 1 \times 1 = 1 + 1 + 1 = 3$, 如图 8.12(b) 所示。再向右移动一格, 可以得到卷积特征矩阵第一行的第 3 个元素为 4, 如图 8.12(c) 所示。

卷积核矩阵移至最左边并往下移一格, 如图 8.12(d) 所示, 同样计算可以得到 2。以此类推, 这里不赘述。

很显然, 卷积特征矩阵比原来的输入矩阵维数低。如果希望得到的卷积特征矩阵维数和原来的输入矩阵维数一样, 可以在原输入矩阵四周进行 0 填充, 扩大输入矩阵的维数。这种操作称为补零 (zero padding)。例如, 在上面这个例子中, 可以在原输入矩阵四周进行 0 填充, 扩大输入矩阵的维数为 7×7 , 这样经过 3×3 的卷积核卷积后, 可以得到维数为 5×5 的卷积特征矩阵。

在卷积核和输入图像矩阵进行卷积过程中, 图像中的所有像素点会被线性变换组合, 得到图像的一些特征。例如, 分别用图 8.13(a)、(b) 两个卷积核, 可以分别得到输入图像的竖向边缘和横向边缘。

$$\begin{array}{cc} \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix} & \begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix} \\ \text{(a)} & \text{(b)} \end{array}$$

图 8.13 卷积核

在没有边缘的比较平坦的区域, 像素值的变化比较小, 而横向边缘上下两侧的像素差异明显, 竖向边缘左右两侧的像素差异比较大。在 8.13(a) 中, 用三行 $1, 0, -1$ 组成的卷积核和输入图像卷积实际是计算了输入图像中每个 3×3 区域内的左右像素的差值, 所以得到了输入图像的竖向边缘。在 8.13(b) 中, 用三列 $1, 0, -1$ 组成的卷积核和输入图像卷积实际是计算了输入图像中每个 3×3 区域内的上下像素的差值, 所以得到了输入图像的横向边缘。

8.6.4 卷积神经网络中的关键技术

在图像处理中, 往往把图像表示为像素的向量。如图 8.14(a) 所示 $1\,000 \times 1\,000$ 的图像, 可以表示为一个 $1\,000\,000$ 的向量。在 BP 神经网络中, 如果隐含层数目与输入层一样, 即也是 $1\,000\,000$ 时, 那么输入层到隐含层的参数数据为 $1\,000\,000 \times 1\,000\,000 = 10^{12}$, 这么多的权值参数很难训练。

卷积神经网络使用 4 个关键技术来利用自然信号的属性: 局部连接、权值共享、多卷积核以及池化的使用。下面分别介绍。

(1) 卷积神经网络的局部连接。一般认为人对外界的认知是从局部到全局的, 而图像的空间联系也是局部的像素联系较为紧密, 而距离较远的像素相关性则较弱。视觉皮层的神经元就是局部接受信息的, 这些神经元只受某些特定区域刺激的响应。因而, 每个神经元只对局部进行感知, 然后在更高层将局部的信息综合起来得到全局信息。这样, 就可以减少神经元之间的连接数, 从而减少神经网络需要训练的权值参数的个数。

局部连接如图 8.14(b) 所示。假如局部感受域是 10×10 , 隐含层每个感受域只需要和这 10×10 的局部图像相连接, 所以 $1\,000\,000$ 个隐含层神经元就只有 $100\,000\,000$ 个连接, 即 10^8 个参数, 比原来减少了 4 个数量级, 但需要训练的参数仍然很多, 需要进一步简化。

(2) 卷积神经网络的权值共享。CNN 受生物学里面的视觉系统结构启发,图像的一部分的统计特性与其他部分是一样的。这也意味着在这一部分学习的特征也能用在另一部分上,所以每个映射面上的神经元共享权值,因而减少了网络自由参数的个数。

隐含层的每一个神经元如果只和 10×10 个像素连接,也就是说每一个神经元存在 $10 \times 10 = 100$ 个连接权值参数。如果将每个神经元的 100 个参数设置成相同,就只有 100 个参数了。不管隐含层的神经元个数有多少,两层间的连接都只有 100 个参数。上述讨论未考虑每个神经元的偏置部分,所以共享权值个数需要加 1,这也是同一种卷积核所共享的。

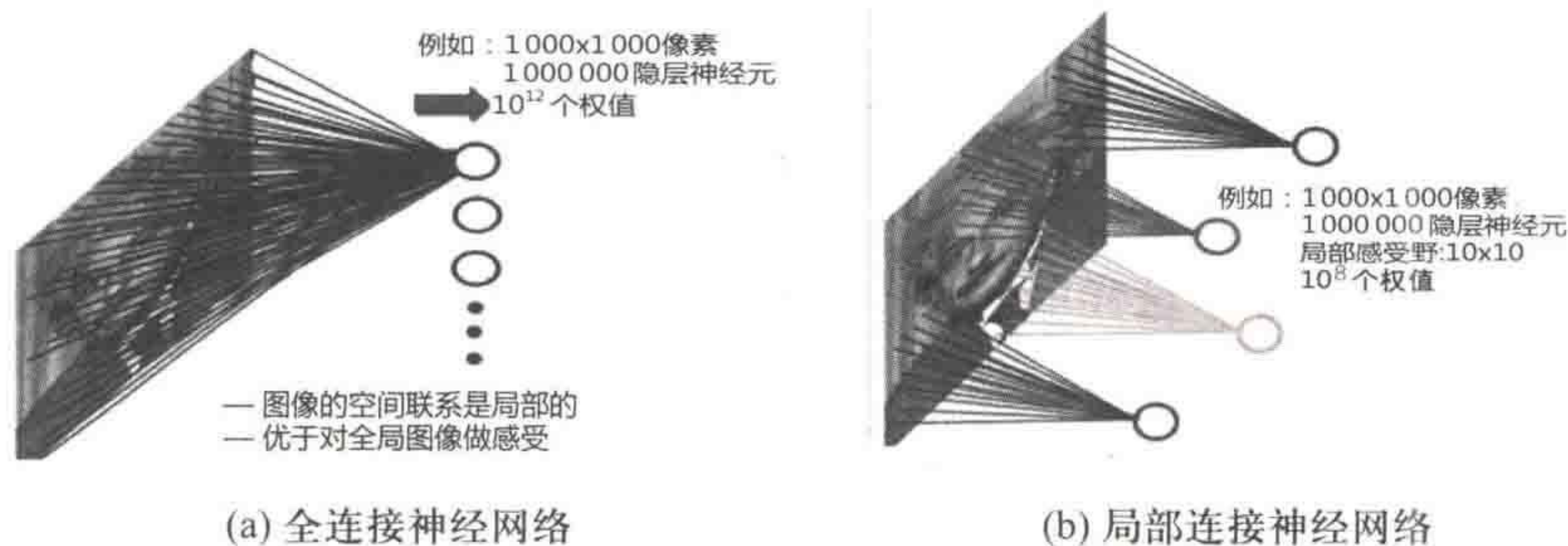


图 8.14 卷积神经网络的局部连接

(3) 卷积神经网络的多卷积核。上面所述只有 100 个参数时,表明只有 1 个 100×100 的卷积核,显然,特征提取是不充分的。可以添加多个卷积核提取不同的特征。每个卷积核都会将原图像生成为另一幅图像。

(4) 卷积神经网络的池化。通过卷积获得了特征之后,如果直接利用这些特征训练分类器,计算量仍然非常大。例如:对于一个 96×96 像素的图像,假设已经学习得到了 400 个定义在 8×8 输入上的特征,每一个特征和图像卷积都会得到一个 $(96-8+1) \times (96-8+1) = 7\,921$ 维的卷积特征,由于有 400 个特征,所以每个样本都会得到一个 $7\,921 \times 400 = 3\,168\,400$ 维的卷积特征向量。学习一个超过 300 万特征输入的分类器是非常困难的,并且容易出现过拟合。为了解决这个问题,对不同位置的特征进行聚合统计,例如,可以计算图像一个区域上的某个特定特征的平均值(或最大值)。这些聚合的统计特征不仅具有低得多的维度(相比使用所有提取得到的特征),同时还会改善结果(不容易过拟合)。这种聚合的操作称为池化(pooling)。有平均池化和最大池化两种方法。

卷积神经网络在池化层丢失一些信息,从而降低了空间分辨率,导致输入有微小变化时输出几乎不变。

8.6.5 卷积神经网络的应用

下面介绍一种典型的手写数字识别系统的卷积网络:Yann LeCun 的 LeNet-5。美国大多数银行当年用它识别支票上面的手写数字,达到了商用程度,具有很高的准确性。

LeNet-5 共有 7 层,不包含输入层,每层都包含可训练参数(连接权重),如图 8.15 所示。